

# MyFlowSim

## 1.2

Generated by Doxygen 1.17.0



|                                              |    |
|----------------------------------------------|----|
| 1 MyVensim                                   | 1  |
| 1.1 Descrição                                | 1  |
| 1.2 Estrutura do Projeto                     | 1  |
| 1.3 Compilação e Execução                    | 2  |
| 1.4 Testes                                   | 2  |
| 1.4.1 Unitários                              | 2  |
| 1.4.2 Funcionais                             | 2  |
| 1.5 Licença                                  | 2  |
| 2 Directory Hierarchy                        | 3  |
| 2.1 Directories                              | 3  |
| 3 Hierarchical Index                         | 5  |
| 3.1 Class Hierarchy                          | 5  |
| 4 Data Structure Index                       | 7  |
| 4.1 Data Structures                          | 7  |
| 5 File Index                                 | 9  |
| 5.1 File List                                | 9  |
| 6 Directory Documentation                    | 11 |
| 6.1 test/functional Directory Reference      | 11 |
| 6.2 include Directory Reference              | 12 |
| 6.3 src Directory Reference                  | 12 |
| 6.4 test Directory Reference                 | 13 |
| 6.5 test/unit Directory Reference            | 13 |
| 7 Data Structure Documentation               | 15 |
| 7.1 Flow Class Reference                     | 15 |
| 7.1.1 Detailed Description                   | 16 |
| 7.1.2 Constructor & Destructor Documentation | 16 |
| 7.1.2.1 ~Flow()                              | 16 |
| 7.1.3 Member Function Documentation          | 17 |
| 7.1.3.1 execute()                            | 17 |
| 7.1.3.2 getName()                            | 17 |
| 7.1.3.3 getSource()                          | 17 |
| 7.1.3.4 getTarget()                          | 17 |
| 7.1.3.5 setName()                            | 17 |
| 7.1.3.6 setSource()                          | 18 |
| 7.1.3.7 setTarget()                          | 18 |
| 7.2 Flow_Impl Class Reference                | 18 |
| 7.2.1 Constructor & Destructor Documentation | 21 |
| 7.2.1.1 Flow_Impl() [1/3]                    | 21 |
| 7.2.1.2 Flow_Impl() [2/3]                    | 22 |

|                                              |    |
|----------------------------------------------|----|
| 7.2.1.3 Flow_Impl() [3/3]                    | 23 |
| 7.2.1.4 ~Flow_Impl()                         | 23 |
| 7.2.2 Member Function Documentation          | 23 |
| 7.2.2.1 execute()                            | 23 |
| 7.2.2.2 getName()                            | 23 |
| 7.2.2.3 getSource()                          | 24 |
| 7.2.2.4 getTarget()                          | 24 |
| 7.2.2.5 operator=()                          | 25 |
| 7.2.2.6 setName()                            | 26 |
| 7.2.2.7 setSource()                          | 26 |
| 7.2.2.8 setTarget()                          | 26 |
| 7.2.3 Field Documentation                    | 27 |
| 7.2.3.1 name                                 | 27 |
| 7.2.3.2 source                               | 27 |
| 7.2.3.3 target                               | 27 |
| 7.3 Flow_Test Class Reference                | 27 |
| 7.3.1 Detailed Description                   | 30 |
| 7.3.2 Constructor & Destructor Documentation | 30 |
| 7.3.2.1 Flow_Test() [1/2]                    | 30 |
| 7.3.2.2 Flow_Test() [2/2]                    | 31 |
| 7.3.3 Member Function Documentation          | 31 |
| 7.3.3.1 execute()                            | 31 |
| 7.4 FlowComplexo Class Reference             | 32 |
| 7.4.1 Detailed Description                   | 34 |
| 7.4.2 Constructor & Destructor Documentation | 35 |
| 7.4.2.1 FlowComplexo() [1/3]                 | 35 |
| 7.4.2.2 FlowComplexo() [2/3]                 | 35 |
| 7.4.2.3 FlowComplexo() [3/3]                 | 36 |
| 7.4.2.4 ~FlowComplexo()                      | 36 |
| 7.4.3 Member Function Documentation          | 36 |
| 7.4.3.1 execute()                            | 36 |
| 7.4.3.2 operator=()                          | 37 |
| 7.5 FlowExponencial Class Reference          | 38 |
| 7.5.1 Detailed Description                   | 40 |
| 7.5.2 Constructor & Destructor Documentation | 41 |
| 7.5.2.1 FlowExponencial() [1/3]              | 41 |
| 7.5.2.2 FlowExponencial() [2/3]              | 41 |
| 7.5.2.3 FlowExponencial() [3/3]              | 42 |
| 7.5.2.4 ~FlowExponencial()                   | 42 |
| 7.5.3 Member Function Documentation          | 42 |
| 7.5.3.1 execute()                            | 42 |
| 7.5.3.2 operator=()                          | 43 |
| 7.6 FlowLogistico Class Reference            | 44 |

|                                              |    |
|----------------------------------------------|----|
| 7.6.1 Detailed Description                   | 46 |
| 7.6.2 Constructor & Destructor Documentation | 47 |
| 7.6.2.1 FlowLogistico() [1/3]                | 47 |
| 7.6.2.2 FlowLogistico() [2/3]                | 47 |
| 7.6.2.3 FlowLogistico() [3/3]                | 48 |
| 7.6.2.4 ~FlowLogistico()                     | 48 |
| 7.6.3 Member Function Documentation          | 48 |
| 7.6.3.1 execute()                            | 48 |
| 7.6.3.2 operator=()                          | 49 |
| 7.7 FlowTest Class Reference                 | 50 |
| 7.7.1 Detailed Description                   | 52 |
| 7.7.2 Constructor & Destructor Documentation | 52 |
| 7.7.2.1 FlowTest() [1/2]                     | 52 |
| 7.7.2.2 FlowTest() [2/2]                     | 53 |
| 7.7.3 Member Function Documentation          | 53 |
| 7.7.3.1 execute()                            | 53 |
| 7.8 Model Class Reference                    | 53 |
| 7.8.1 Detailed Description                   | 56 |
| 7.8.2 Constructor & Destructor Documentation | 56 |
| 7.8.2.1 ~Model()                             | 56 |
| 7.8.3 Member Function Documentation          | 56 |
| 7.8.3.1 add() [1/2]                          | 56 |
| 7.8.3.2 add() [2/2]                          | 56 |
| 7.8.3.3 clearSource()                        | 57 |
| 7.8.3.4 clearTarget()                        | 57 |
| 7.8.3.5 createFlow()                         | 57 |
| 7.8.3.6 createModel()                        | 58 |
| 7.8.3.7 createSystem()                       | 59 |
| 7.8.3.8 deleteFlow()                         | 60 |
| 7.8.3.9 deleteSystem()                       | 60 |
| 7.8.3.10 run()                               | 60 |
| 7.8.3.11 setSource()                         | 61 |
| 7.8.3.12 setTarget()                         | 61 |
| 7.8.3.13 showModel()                         | 62 |
| 7.9 Model_Impl Class Reference               | 62 |
| 7.9.1 Detailed Description                   | 66 |
| 7.9.2 Constructor & Destructor Documentation | 66 |
| 7.9.2.1 Model_Impl() [1/2]                   | 66 |
| 7.9.2.2 Model_Impl() [2/2]                   | 66 |
| 7.9.2.3 ~Model_Impl()                        | 67 |
| 7.9.3 Member Function Documentation          | 67 |
| 7.9.3.1 add() [1/4]                          | 67 |
| 7.9.3.2 add() [2/4]                          | 67 |

|                                               |    |
|-----------------------------------------------|----|
| 7.9.3.3 add() [3/4]                           | 68 |
| 7.9.3.4 add() [4/4]                           | 68 |
| 7.9.3.5 clearSource()                         | 68 |
| 7.9.3.6 clearTarget()                         | 69 |
| 7.9.3.7 createModel()                         | 69 |
| 7.9.3.8 createSystem()                        | 70 |
| 7.9.3.9 deleteFlow()                          | 71 |
| 7.9.3.10 deleteSystem()                       | 71 |
| 7.9.3.11 operator=()                          | 72 |
| 7.9.3.12 run()                                | 73 |
| 7.9.3.13 setSource()                          | 74 |
| 7.9.3.14 setTarget()                          | 74 |
| 7.9.3.15 showModel()                          | 75 |
| 7.9.4 Field Documentation                     | 75 |
| 7.9.4.1 flows                                 | 75 |
| 7.9.4.2 id_                                   | 75 |
| 7.9.4.3 models                                | 75 |
| 7.9.4.4 systems                               | 75 |
| 7.10 System Class Reference                   | 76 |
| 7.10.1 Detailed Description                   | 77 |
| 7.10.2 Constructor & Destructor Documentation | 77 |
| 7.10.2.1 ~System()                            | 77 |
| 7.10.3 Member Function Documentation          | 77 |
| 7.10.3.1 getName()                            | 77 |
| 7.10.3.2 getValue()                           | 78 |
| 7.10.3.3 setName()                            | 78 |
| 7.10.3.4 setValue()                           | 78 |
| 7.11 System_Impl Class Reference              | 79 |
| 7.11.1 Detailed Description                   | 82 |
| 7.11.2 Constructor & Destructor Documentation | 82 |
| 7.11.2.1 System_Impl() [1/3]                  | 82 |
| 7.11.2.2 System_Impl() [2/3]                  | 82 |
| 7.11.2.3 System_Impl() [3/3]                  | 83 |
| 7.11.2.4 ~System_Impl()                       | 83 |
| 7.11.3 Member Function Documentation          | 83 |
| 7.11.3.1 getName()                            | 83 |
| 7.11.3.2 getValue()                           | 83 |
| 7.11.3.3 operator=()                          | 84 |
| 7.11.3.4 setName()                            | 85 |
| 7.11.3.5 setValue()                           | 85 |
| 7.11.4 Field Documentation                    | 85 |
| 7.11.4.1 id_                                  | 85 |
| 7.11.4.2 name                                 | 85 |

|                                               |     |
|-----------------------------------------------|-----|
| 7.11.4.3 value                                | 85  |
| 8 File Documentation                          | 87  |
| 8.1 include/flow.h File Reference             | 87  |
| 8.1.1 Detailed Description                    | 87  |
| 8.2 flow.h                                    | 88  |
| 8.3 include/model.h File Reference            | 88  |
| 8.3.1 Detailed Description                    | 89  |
| 8.4 model.h                                   | 89  |
| 8.5 include/system.h File Reference           | 90  |
| 8.5.1 Detailed Description                    | 90  |
| 8.6 system.h                                  | 90  |
| 8.7 README.md File Reference                  | 91  |
| 8.8 src/flow_impl.cpp File Reference          | 91  |
| 8.8.1 Detailed Description                    | 91  |
| 8.9 src/flow_impl.h File Reference            | 92  |
| 8.10 flow_impl.h                              | 92  |
| 8.11 src/main.cpp File Reference              | 93  |
| 8.11.1 Detailed Description                   | 94  |
| 8.11.2 Macro Definition Documentation         | 94  |
| 8.11.2.1 MAIN                                 | 94  |
| 8.11.3 Function Documentation                 | 94  |
| 8.11.3.1 main()                               | 94  |
| 8.12 test/functional/main.cpp File Reference  | 95  |
| 8.12.1 Detailed Description                   | 95  |
| 8.12.2 Macro Definition Documentation         | 96  |
| 8.12.2.1 MAIN_FUNCIONAL_TESTS                 | 96  |
| 8.12.3 Function Documentation                 | 96  |
| 8.12.3.1 main()                               | 96  |
| 8.13 test/unit/main.cpp File Reference        | 96  |
| 8.13.1 Detailed Description                   | 97  |
| 8.13.2 Function Documentation                 | 97  |
| 8.13.2.1 main()                               | 97  |
| 8.14 src/model_impl.cpp File Reference        | 98  |
| 8.14.1 Detailed Description                   | 99  |
| 8.15 src/model_impl.h File Reference          | 99  |
| 8.16 model_impl.h                             | 100 |
| 8.17 src/system_impl.cpp File Reference       | 101 |
| 8.17.1 Detailed Description                   | 102 |
| 8.18 src/system_impl.h File Reference         | 102 |
| 8.19 system_impl.h                            | 103 |
| 8.20 test/functional/flows.cpp File Reference | 103 |
| 8.20.1 Detailed Description                   | 104 |

|                                                                   |     |
|-------------------------------------------------------------------|-----|
| 8.21 test/functional/flows.h File Reference . . . . .             | 104 |
| 8.21.1 Detailed Description . . . . .                             | 106 |
| 8.22 flows.h . . . . .                                            | 106 |
| 8.23 test/functional/functional_test.cpp File Reference . . . . . | 107 |
| 8.23.1 Detailed Description . . . . .                             | 107 |
| 8.23.2 Function Documentation . . . . .                           | 108 |
| 8.23.2.1 complexFuncionalTest() . . . . .                         | 108 |
| 8.23.2.2 exponentialFuncionalTest() . . . . .                     | 108 |
| 8.23.2.3 floatingPointComparison() . . . . .                      | 109 |
| 8.23.2.4 logisticalFuncionalTest() . . . . .                      | 110 |
| 8.24 test/functional/functional_test.h File Reference . . . . .   | 111 |
| 8.24.1 Detailed Description . . . . .                             | 111 |
| 8.24.2 Function Documentation . . . . .                           | 112 |
| 8.24.2.1 complexFuncionalTest() . . . . .                         | 112 |
| 8.24.2.2 exponentialFuncionalTest() . . . . .                     | 112 |
| 8.24.2.3 logisticalFuncionalTest() . . . . .                      | 113 |
| 8.25 functional_test.h . . . . .                                  | 114 |
| 8.26 test/unit/unit_flow.cpp File Reference . . . . .             | 114 |
| 8.26.1 Detailed Description . . . . .                             | 115 |
| 8.26.2 Function Documentation . . . . .                           | 115 |
| 8.26.2.1 run_unit_tests_Flow() . . . . .                          | 115 |
| 8.26.2.2 unit_Flow_constructor() . . . . .                        | 116 |
| 8.26.2.3 unit_Flow_destructor() . . . . .                         | 117 |
| 8.26.2.4 unit_Flow_execute() . . . . .                            | 117 |
| 8.26.2.5 unit_Flow_getName() . . . . .                            | 118 |
| 8.26.2.6 unit_Flow_getSource() . . . . .                          | 118 |
| 8.26.2.7 unit_Flow_getTarget() . . . . .                          | 119 |
| 8.26.2.8 unit_Flow_setName() . . . . .                            | 119 |
| 8.26.2.9 unit_Flow_setSource() . . . . .                          | 120 |
| 8.26.2.10 unit_Flow_setTarget() . . . . .                         | 121 |
| 8.27 test/unit/unit_flow.h File Reference . . . . .               | 121 |
| 8.27.1 Detailed Description . . . . .                             | 122 |
| 8.27.2 Function Documentation . . . . .                           | 122 |
| 8.27.2.1 run_unit_tests_Flow() . . . . .                          | 122 |
| 8.27.2.2 unit_Flow_constructor() . . . . .                        | 123 |
| 8.27.2.3 unit_Flow_destructor() . . . . .                         | 124 |
| 8.27.2.4 unit_Flow_execute() . . . . .                            | 124 |
| 8.27.2.5 unit_Flow_getName() . . . . .                            | 125 |
| 8.27.2.6 unit_Flow_getSource() . . . . .                          | 125 |
| 8.27.2.7 unit_Flow_getTarget() . . . . .                          | 126 |
| 8.27.2.8 unit_Flow_setName() . . . . .                            | 126 |
| 8.27.2.9 unit_Flow_setSource() . . . . .                          | 127 |
| 8.27.2.10 unit_Flow_setTarget() . . . . .                         | 128 |



|          |                                                          |     |
|----------|----------------------------------------------------------|-----|
| 8.28     | <a href="#">unit_flow.h</a>                              | 128 |
| 8.29     | <a href="#">test/unit/unit_model.cpp</a> File Reference  | 128 |
| 8.29.1   | Detailed Description                                     | 129 |
| 8.29.2   | Function Documentation                                   | 130 |
| 8.29.2.1 | <a href="#">run_unit_tests_Model()</a>                   | 130 |
| 8.29.2.2 | <a href="#">unit_Model_addFlow()</a>                     | 130 |
| 8.29.2.3 | <a href="#">unit_Model_addSystem()</a>                   | 131 |
| 8.29.2.4 | <a href="#">unit_Model_constructor()</a>                 | 131 |
| 8.29.2.5 | <a href="#">unit_Model_destructor()</a>                  | 132 |
| 8.29.2.6 | <a href="#">unit_Model_run()</a>                         | 132 |
| 8.29.2.7 | <a href="#">unit_Model_showModel()</a>                   | 133 |
| 8.30     | <a href="#">test/unit/unit_model.h</a> File Reference    | 133 |
| 8.30.1   | Detailed Description                                     | 134 |
| 8.30.2   | Function Documentation                                   | 134 |
| 8.30.2.1 | <a href="#">run_unit_tests_Model()</a>                   | 134 |
| 8.30.2.2 | <a href="#">unit_Model_addFlow()</a>                     | 135 |
| 8.30.2.3 | <a href="#">unit_Model_addSystem()</a>                   | 136 |
| 8.30.2.4 | <a href="#">unit_Model_constructor()</a>                 | 136 |
| 8.30.2.5 | <a href="#">unit_Model_destructor()</a>                  | 137 |
| 8.30.2.6 | <a href="#">unit_Model_run()</a>                         | 137 |
| 8.30.2.7 | <a href="#">unit_Model_showModel()</a>                   | 138 |
| 8.31     | <a href="#">unit_model.h</a>                             | 138 |
| 8.32     | <a href="#">test/unit/unit_system.cpp</a> File Reference | 139 |
| 8.32.1   | Detailed Description                                     | 140 |
| 8.32.2   | Function Documentation                                   | 140 |
| 8.32.2.1 | <a href="#">run_unit_tests_System()</a>                  | 140 |
| 8.32.2.2 | <a href="#">unit_System_constructor()</a>                | 140 |
| 8.32.2.3 | <a href="#">unit_System_destructor()</a>                 | 141 |
| 8.32.2.4 | <a href="#">unit_System_getValue()</a>                   | 141 |
| 8.32.2.5 | <a href="#">unit_System_setValue()</a>                   | 141 |
| 8.33     | <a href="#">test/unit/unit_system.h</a> File Reference   | 142 |
| 8.33.1   | Detailed Description                                     | 143 |
| 8.33.2   | Function Documentation                                   | 143 |
| 8.33.2.1 | <a href="#">run_unit_tests_System()</a>                  | 143 |
| 8.33.2.2 | <a href="#">unit_System_constructor()</a>                | 144 |
| 8.33.2.3 | <a href="#">unit_System_destructor()</a>                 | 144 |
| 8.33.2.4 | <a href="#">unit_System_getValue()</a>                   | 145 |
| 8.33.2.5 | <a href="#">unit_System_setValue()</a>                   | 145 |
| 8.34     | <a href="#">unit_system.h</a>                            | 146 |
| 8.35     | <a href="#">test/unit/unit_tests.cpp</a> File Reference  | 146 |
| 8.35.1   | Detailed Description                                     | 147 |
| 8.35.2   | Function Documentation                                   | 147 |
| 8.35.2.1 | <a href="#">run_unit_tests_globals()</a>                 | 147 |

|                                                      |     |
|------------------------------------------------------|-----|
| 8.36 test/unit/unit_tests.h File Reference . . . . . | 148 |
| 8.36.1 Detailed Description . . . . .                | 149 |
| 8.36.2 Function Documentation . . . . .              | 150 |
| 8.36.2.1 run_unit_tests_globals() . . . . .          | 150 |
| 8.37 unit_tests.h . . . . .                          | 150 |
| Index . . . . .                                      | 153 |

# Chapter 1

## MyVensim

Framework em C++ para construção e execução de modelos de simulação baseados em Dinâmica de Sistemas.

Disciplina: BCC322 — Engenharia de Software I · UFOP

Professor: Tiago Garcia de Senna Carneiro

---

### 1.1 Descrição

API inspirada na linguagem DYNAMO e no simulador Vensim. Permite modelar e executar sistemas dinâmicos compostos por sistemas (estoques), fluxos (equações de transferência) e modelos (orquestradores da simulação).

---

### 1.2 Estrutura do Projeto

```
MyVensim/  
  include/  
    flow.h  
    flow_impl.h  
    flows.h  
    functional_test.h  
    model.h  
    model_impl.h  
    system.h  
    system_impl.h  
  
  src/  
    main.cpp  
    flow_impl.cpp  
    flows.cpp  
    model_impl.cpp  
    system_impl.cpp  
  
  test/  
    functional/  
      main.cpp  
      functional_test.cpp  
    unit/  
      main.cpp  
      unit_tests.h  
      unit_tests.cpp  
      unit_flow.h  
      unit_flow.cpp  
      unit_model.h  
      unit_model.cpp  
      unit_system.h  
      unit_system.cpp  
  
  bin/  
  Makefile  
  README.md
```

---

## 1.3 Compilação e Execução

Requer g++ com suporte a C++17.

| Comando               | Descrição                              |
|-----------------------|----------------------------------------|
| make                  | Compila tudo (simulador + testes)      |
| make unit_tests       | Compila e executa os testes unitários  |
| make functional_tests | Compila e executa os testes funcionais |
| make run              | Executa testes unitários e funcionais  |
| make clean            | Remove binários e objetos gerados      |

---

## 1.4 Testes

### 1.4.1 Unitários

Verificam individualmente os métodos de [System](#), [Flow](#) e [Model](#).

Executável: bin/progTestUnit

### 1.4.2 Funcionais

Verificam a simulação completa contra valores analíticos esperados para três modelos:

| Modelo      | Descrição                                               |
|-------------|---------------------------------------------------------|
| Exponencial | Crescimento proporcional ao estoque de origem (taxa 1%) |
| Logístico   | Crescimento com capacidade limite de 70                 |
| Complexo    | Rede com 5 sistemas e 6 fluxos interligados             |

Executável: bin/progTestFuncional

---

## 1.5 Licença

Desenvolvido exclusivamente para fins acadêmicos na disciplina BCC322 — UFOP.

Uso comercial não permitido sem autorização prévia do autor.

## Chapter 2

# Directory Hierarchy

### 2.1 Directories

|                     |     |
|---------------------|-----|
| functional          | 11  |
| flows.cpp           | 103 |
| flows.h             | 104 |
| functional_test.cpp | 107 |
| functional_test.h   | 111 |
| main.cpp            | 95  |
| include             | 12  |
| flow.h              | 87  |
| model.h             | 88  |
| system.h            | 90  |
| src                 | 12  |
| flow_impl.cpp       | 91  |
| flow_impl.h         | 92  |
| main.cpp            | 93  |
| model_impl.cpp      | 98  |
| model_impl.h        | 99  |
| system_impl.cpp     | 101 |
| system_impl.h       | 102 |
| test                | 13  |
| functional          | 11  |
| flows.cpp           | 103 |
| flows.h             | 104 |
| functional_test.cpp | 107 |
| functional_test.h   | 111 |
| main.cpp            | 95  |
| unit                | 13  |
| main.cpp            | 96  |
| unit_flow.cpp       | 114 |
| unit_flow.h         | 121 |
| unit_model.cpp      | 128 |
| unit_model.h        | 133 |
| unit_system.cpp     | 139 |
| unit_system.h       | 142 |
| unit_tests.cpp      | 146 |
| unit_tests.h        | 148 |
| unit                | 13  |
| main.cpp            | 96  |
| unit_flow.cpp       | 114 |
| unit_flow.h         | 121 |
| unit_model.cpp      | 128 |

|                           |     |
|---------------------------|-----|
| unit_model.h . . . . .    | 133 |
| unit_system.cpp . . . . . | 139 |
| unit_system.h . . . . .   | 142 |
| unit_tests.cpp . . . . .  | 146 |
| unit_tests.h . . . . .    | 148 |

## Chapter 3

# Hierarchical Index

### 3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

|                 |    |
|-----------------|----|
| Flow            | 15 |
| Flow_Impl       | 18 |
| FlowComplexo    | 32 |
| FlowExponencial | 38 |
| FlowLogistico   | 44 |
| FlowTest        | 50 |
| Flow_Test       | 27 |
| Model           | 53 |
| Model_Impl      | 62 |
| System          | 76 |
| System_Impl     | 79 |





## Chapter 4

# Data Structure Index

### 4.1 Data Structures

Here are the data structures with brief descriptions:

|                                 |                                                                                                             |    |
|---------------------------------|-------------------------------------------------------------------------------------------------------------|----|
| <a href="#">Flow</a>            | Interface abstrata que representa um fluxo entre dois sistemas . . . . .                                    | 15 |
| <a href="#">Flow_Impl</a>       | . . . . .                                                                                                   | 18 |
| <a href="#">Flow_Test</a>       | Implementação concreta de <a href="#">Flow_Impl</a> utilizada nos testes unitários . . . . .                | 27 |
| <a href="#">FlowComplexo</a>    | Fluxo com equação de múltiplas interações entre sistemas . . . . .                                          | 32 |
| <a href="#">FlowExponencial</a> | Fluxo baseado em crescimento exponencial . . . . .                                                          | 38 |
| <a href="#">FlowLogistico</a>   | Fluxo baseado em crescimento logístico . . . . .                                                            | 44 |
| <a href="#">FlowTest</a>        | Implementação concreta de <a href="#">Flow_Impl</a> utilizada nos testes do <a href="#">Model</a> . . . . . | 50 |
| <a href="#">Model</a>           | Interface abstrata que representa um modelo de dinâmica de sistemas . . . . .                               | 53 |
| <a href="#">Model_Impl</a>      | Classe responsável por representar um modelo de simulação . . . . .                                         | 62 |
| <a href="#">System</a>          | Interface abstrata que representa um sistema em um modelo de simulação . . . . .                            | 76 |
| <a href="#">System_Impl</a>     | Classe responsável por representar um sistema . . . . .                                                     | 79 |



# Chapter 5

## File Index

### 5.1 File List

Here is a list of all files with brief descriptions:

|                                                      |                                                                                        |     |
|------------------------------------------------------|----------------------------------------------------------------------------------------|-----|
| include/ <a href="#">flow.h</a>                      | Declaração da interface abstrata <a href="#">Flow</a> . . . . .                        | 87  |
| include/ <a href="#">model.h</a>                     | Declaração da interface abstrata <a href="#">Model</a> . . . . .                       | 88  |
| include/ <a href="#">system.h</a>                    | Declaração da interface abstrata <a href="#">System</a> . . . . .                      | 90  |
| src/ <a href="#">flow_impl.cpp</a>                   | Implementação da classe <a href="#">Flow_Impl</a> . . . . .                            | 91  |
| src/ <a href="#">flow_impl.h</a>                     | . . . . .                                                                              | 92  |
| src/ <a href="#">main.cpp</a>                        | Ponto de entrada principal do simulador . . . . .                                      | 93  |
| src/ <a href="#">model_impl.cpp</a>                  | Implementação da classe <a href="#">Model_Impl</a> . . . . .                           | 98  |
| src/ <a href="#">model_impl.h</a>                    | . . . . .                                                                              | 99  |
| src/ <a href="#">system_impl.cpp</a>                 | Implementação da classe <a href="#">System_Impl</a> . . . . .                          | 101 |
| src/ <a href="#">system_impl.h</a>                   | . . . . .                                                                              | 102 |
| test/functional/ <a href="#">flows.cpp</a>           | Implementação dos fluxos do simulador . . . . .                                        | 103 |
| test/functional/ <a href="#">flows.h</a>             | Declaração das classes de fluxo concretas: exponencial, logístico e complexo . . . . . | 104 |
| test/functional/ <a href="#">functional_test.cpp</a> | Implementação dos testes funcionais do simulador . . . . .                             | 107 |
| test/functional/ <a href="#">functional_test.h</a>   | Declaração dos testes funcionais dos modelos de simulação . . . . .                    | 111 |
| test/functional/ <a href="#">main.cpp</a>            | Ponto de entrada dos testes funcionais do simulador . . . . .                          | 95  |
| test/unit/ <a href="#">main.cpp</a>                  | Ponto de entrada da suíte de testes unitários . . . . .                                | 96  |
| test/unit/ <a href="#">unit_flow.cpp</a>             | Implementação dos testes unitários da classe <a href="#">Flow</a> . . . . .            | 114 |
| test/unit/ <a href="#">unit_flow.h</a>               | Declaração dos testes unitários da classe <a href="#">Flow</a> . . . . .               | 121 |
| test/unit/ <a href="#">unit_model.cpp</a>            | Implementação dos testes unitários da classe <a href="#">Model</a> . . . . .           | 128 |
| test/unit/ <a href="#">unit_model.h</a>              | Declaração dos testes unitários da classe <a href="#">Model</a> . . . . .              | 133 |
| test/unit/ <a href="#">unit_system.cpp</a>           | Implementação dos testes unitários da classe <a href="#">System</a> . . . . .          | 139 |

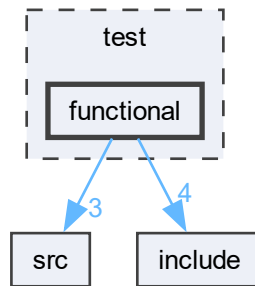
|                                                                    |     |
|--------------------------------------------------------------------|-----|
| test/unit/ <a href="#">unit_system.h</a>                           |     |
| Declaração dos testes unitários da classe <a href="#">System</a>   | 142 |
| test/unit/ <a href="#">unit_tests.cpp</a>                          |     |
| Implementação da função orquestradora dos testes unitários globais | 146 |
| test/unit/ <a href="#">unit_tests.h</a>                            |     |
| Declaração da função de entrada dos testes unitários globais       | 148 |

## Chapter 6

# Directory Documentation

### 6.1 test/functional Directory Reference

Directory dependency graph for functional:

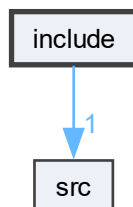


#### Files

- file [flows.cpp](#)  
Implementação dos fluxos do simulador.
- file [flows.h](#)  
Declaração das classes de fluxo concretas: exponencial, logístico e complexo.
- file [functional\\_test.cpp](#)  
Implementação dos testes funcionais do simulador.
- file [functional\\_test.h](#)  
Declaração dos testes funcionais dos modelos de simulação.
- file [main.cpp](#)  
Ponto de entrada dos testes funcionais do simulador.

## 6.2 include Directory Reference

Directory dependency graph for include:

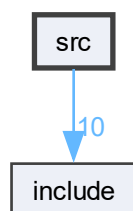


Files

- file [flow.h](#)  
Declaração da interface abstrata [Flow](#).
- file [model.h](#)  
Declaração da interface abstrata [Model](#).
- file [system.h](#)  
Declaração da interface abstrata [System](#).

## 6.3 src Directory Reference

Directory dependency graph for src:



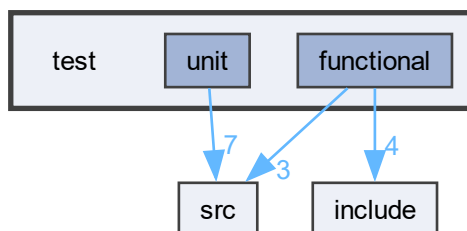
Files

- file [flow\\_impl.cpp](#)  
Implementação da classe [Flow\\_Impl](#).
- file [flow\\_impl.h](#)
- file [main.cpp](#)  
Ponto de entrada principal do simulador.
- file [model\\_impl.cpp](#)  
Implementação da classe [Model\\_Impl](#).
- file [model\\_impl.h](#)

- file [system\\_impl.cpp](#)  
Implementação da classe [System\\_Impl](#).
- file [system\\_impl.h](#)

## 6.4 test Directory Reference

Directory dependency graph for test:

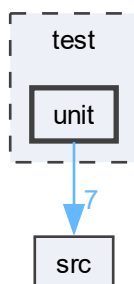


Directories

- directory [functional](#)
- directory [unit](#)

## 6.5 test/unit Directory Reference

Directory dependency graph for unit:



Files

- file [main.cpp](#)  
Ponto de entrada da suíte de testes unitários.
- file [unit\\_flow.cpp](#)  
Implementação dos testes unitários da classe [Flow](#).

- file [unit\\_flow.h](#)  
Declaração dos testes unitários da classe [Flow](#).
- file [unit\\_model.cpp](#)  
Implementação dos testes unitários da classe [Model](#).
- file [unit\\_model.h](#)  
Declaração dos testes unitários da classe [Model](#).
- file [unit\\_system.cpp](#)  
Implementação dos testes unitários da classe [System](#).
- file [unit\\_system.h](#)  
Declaração dos testes unitários da classe [System](#).
- file [unit\\_tests.cpp](#)  
Implementação da função orquestradora dos testes unitários globais.
- file [unit\\_tests.h](#)  
Declaração da função de entrada dos testes unitários globais.



## Chapter 7

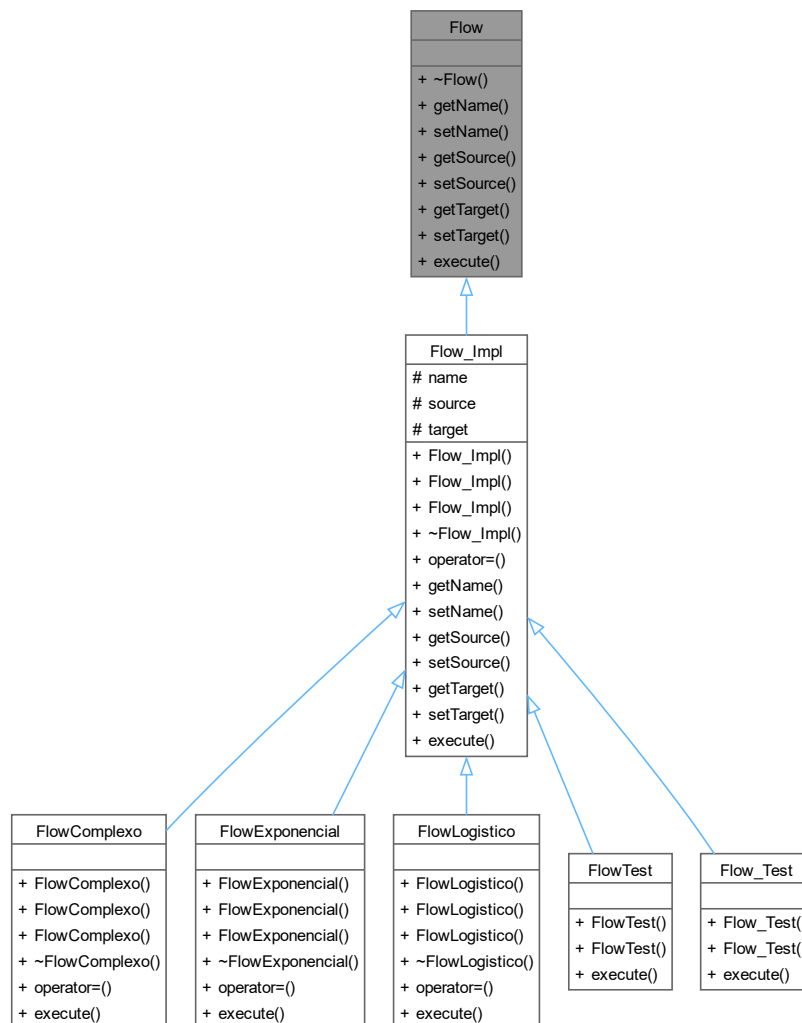
# Data Structure Documentation

### 7.1 Flow Class Reference

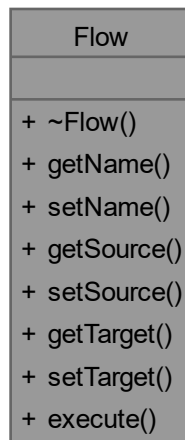
Interface abstrata que representa um fluxo entre dois sistemas.

```
#include <flow.h>
```

Inheritance diagram for Flow:



Collaboration diagram for Flow:



#### Public Member Functions

- virtual `~Flow()` = default  
Destrutor virtual da classe `Flow`.
- virtual `string getName()` const = 0  
Retorna o nome do fluxo.
- virtual `void setName(const string &name)` = 0  
Define o nome do fluxo.
- virtual `System * getSource()` const = 0  
Retorna o sistema de origem do fluxo.
- virtual `void setSource(System *source)` = 0  
Define o sistema de origem do fluxo.
- virtual `System * getTarget()` const = 0  
Retorna o sistema de destino do fluxo.
- virtual `void setTarget(System *target)` = 0  
Define o sistema de destino do fluxo.
- virtual `double execute()` = 0  
Método abstrato.

#### 7.1.1 Detailed Description

Interface abstrata que representa um fluxo entre dois sistemas.

A classe `Flow` define a interface para os fluxos de um modelo de dinâmica de sistemas. Um fluxo conecta dois sistemas (origem e destino) e define uma equação que determina a quantidade transferida por unidade de tempo. Subclasses devem implementar o método `execute()`.

#### 7.1.2 Constructor & Destructor Documentation

##### 7.1.2.1 ~Flow()

`virtual Flow::~~Flow()` [virtual], [default]

Destrutor virtual da classe `Flow`.

### 7.1.3 Member Function Documentation

#### 7.1.3.1 execute()

virtual double Flow::execute () [pure virtual]

Método abstrato.

Executa o cálculo do fluxo para um instante de tempo.

Método puramente virtual implementado pelas subclasses. Calcula a quantidade a ser transferida do sistema de origem para o sistema de destino com base na equação específica do fluxo.

Returns

Valor calculado pelo fluxo.

Implemented in [Flow\\_Impl](#), [Flow\\_Test](#), [FlowComplexo](#), [FlowExponencial](#), [FlowLogistico](#), and [FlowTest](#).

#### 7.1.3.2 getName()

virtual string Flow::getName () const [pure virtual]

Retorna o nome do fluxo.

Returns

Nome do fluxo.

Implemented in [Flow\\_Impl](#).

#### 7.1.3.3 getSource()

virtual [System](#) \* Flow::getSource () const [pure virtual]

Retorna o sistema de origem do fluxo.

Returns

Ponteiro para o sistema de origem.

Implemented in [Flow\\_Impl](#).

#### 7.1.3.4 getTarget()

virtual [System](#) \* Flow::getTarget () const [pure virtual]

Retorna o sistema de destino do fluxo.

Returns

Ponteiro para o sistema de destino.

Implemented in [Flow\\_Impl](#).

#### 7.1.3.5 setName()

virtual void Flow::setName (  
const string & name) [pure virtual]

Define o nome do fluxo.

Parameters

|      |                     |
|------|---------------------|
| name | Novo nome do fluxo. |
|------|---------------------|

Implemented in [Flow\\_Impl](#).

#### 7.1.3.6 setSource()

```
virtual void Flow::setSource (  
    System * source) [pure virtual]
```

Define o sistema de origem do fluxo.

Parameters

|        |                                         |
|--------|-----------------------------------------|
| source | Ponteiro para o novo sistema de origem. |
|--------|-----------------------------------------|

Implemented in [Flow\\_Impl](#).

#### 7.1.3.7 setTarget()

```
virtual void Flow::setTarget (  
    System * target) [pure virtual]
```

Define o sistema de destino do fluxo.

Parameters

|        |                                          |
|--------|------------------------------------------|
| target | Ponteiro para o novo sistema de destino. |
|--------|------------------------------------------|

Implemented in [Flow\\_Impl](#).

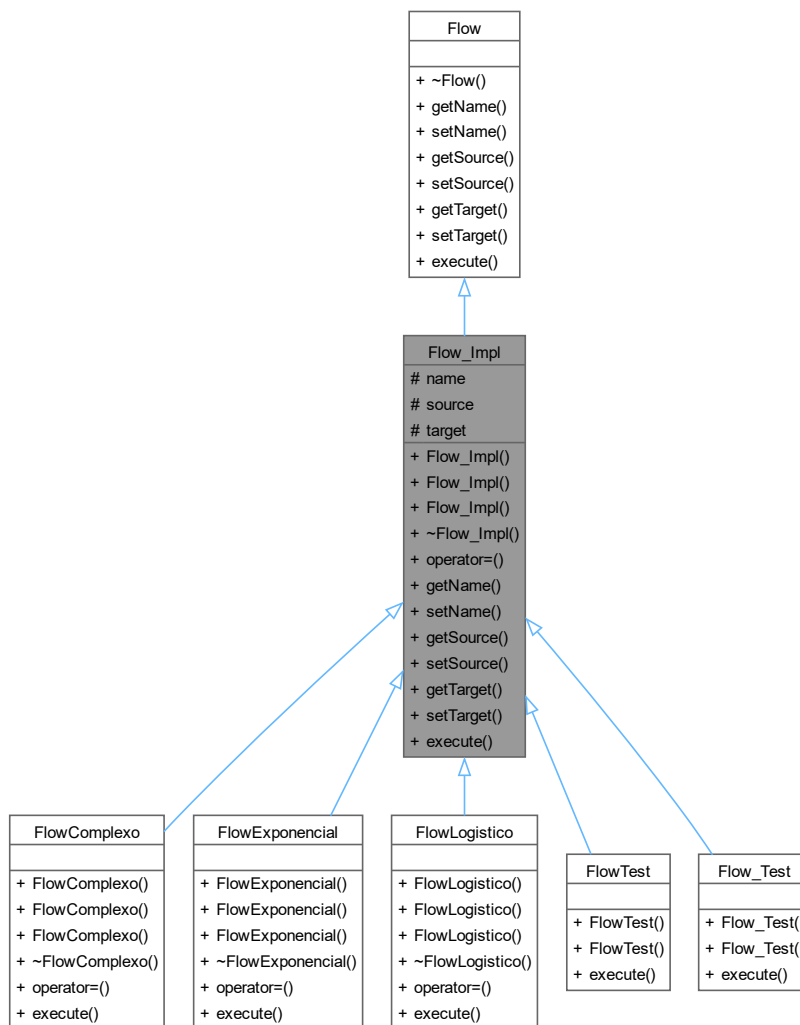
The documentation for this class was generated from the following file:

- include/[flow.h](#)

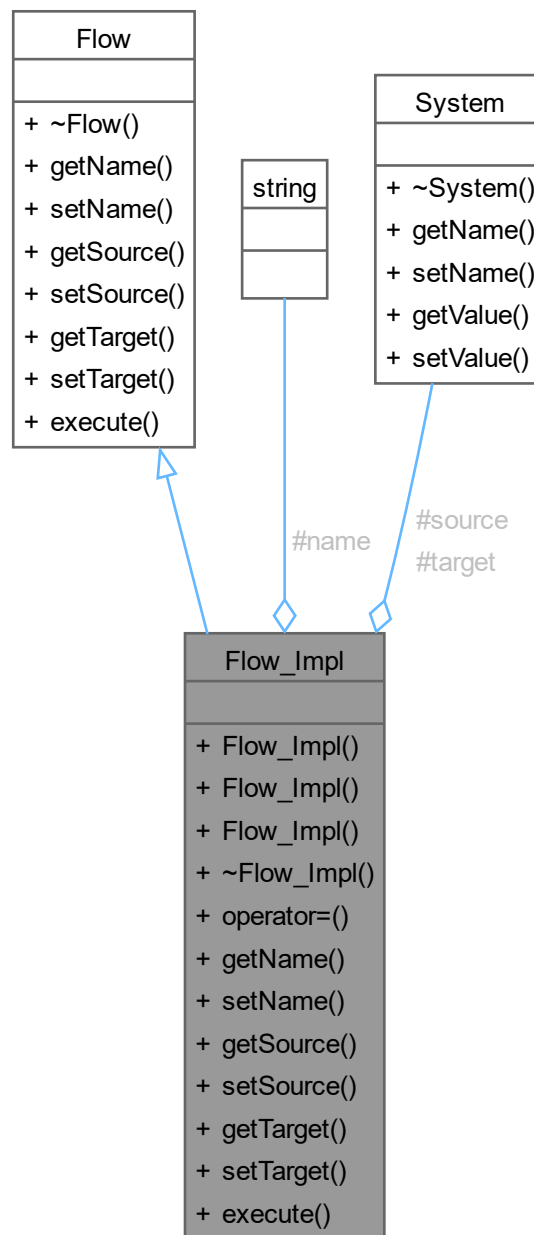
## 7.2 Flow\_Impl Class Reference

```
#include <flow_impl.h>
```

Inheritance diagram for Flow\_Impl:



Collaboration diagram for Flow\_Impl:



#### Public Member Functions

- [Flow\\_Impl \(\)](#)  
Forma canônica.
- [Flow\\_Impl \(const string &name, System \\*source, System \\*target\)](#)  
Construtor parametrizado da classe [Flow](#).
- [Flow\\_Impl \(const Flow\\_Impl &other\)](#)  
Construtor de cópia da classe [Flow](#).
- virtual [~Flow\\_Impl \(\)](#)

- Destrutor da classe [Flow](#).
- [Flow\\_Impl](#) & [operator=](#) (const [Flow\\_Impl](#) &other)  
Operador de atribuição da classe [Flow](#).
- string [getName](#) () const  
Getters e setters.
- void [setName](#) (const string &name)  
Define o nome do fluxo.
- [System](#) \* [getSource](#) () const  
Retorna o sistema de origem.
- void [setSource](#) ([System](#) \*source)  
Define o sistema de origem.
- [System](#) \* [getTarget](#) () const  
Retorna o sistema de destino.
- void [setTarget](#) ([System](#) \*target)  
Define o sistema de destino.
- virtual double [execute](#) ()=0  
Método abstrato.

### Public Member Functions inherited from [Flow](#)

- virtual [~Flow](#) ()=default  
Destrutor virtual da classe [Flow](#).

### Protected Attributes

- string [name](#)
- [System](#) \* [source](#)
- [System](#) \* [target](#)

## 7.2.1 Constructor & Destructor Documentation

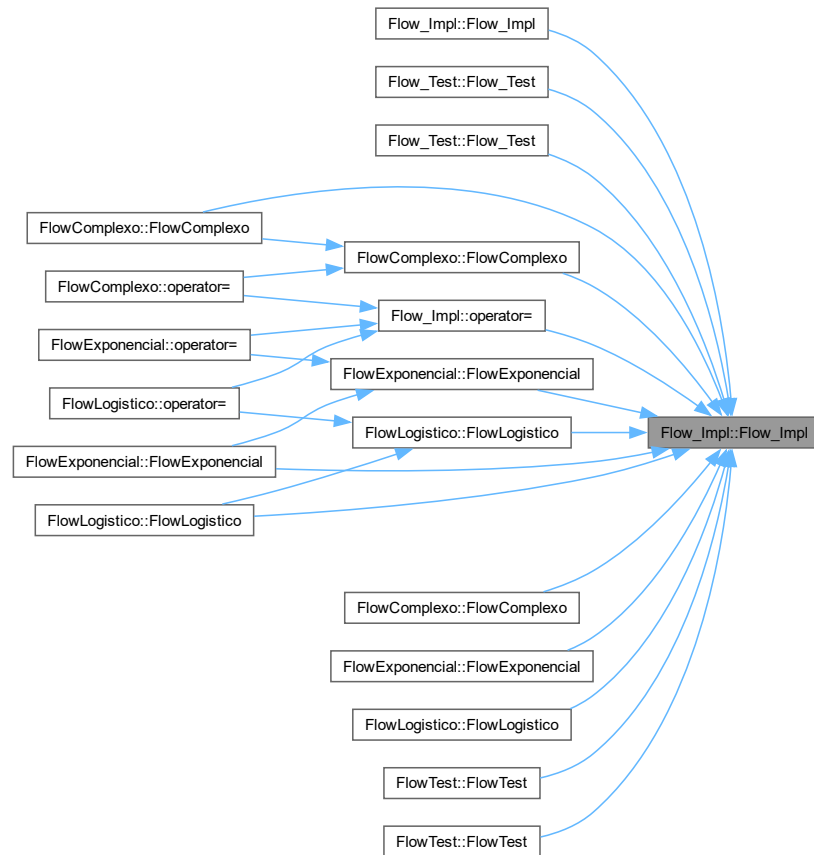
### 7.2.1.1 [Flow\\_Impl](#)() [1/3]

[Flow\\_Impl](#)::[Flow\\_Impl](#) ()

Forma canônica.

Construtor padrão da classe [Flow\\_Impl](#).

Inicializa o fluxo com nome vazio, source nulo e target nulo. Here is the caller graph for this function:



#### 7.2.1.2 Flow\_Impl() [2/3]

```
Flow_Impl::Flow_Impl (
    const string & name,
    System * source,
    System * target)
```

Construtor parametrizado da classe [Flow](#).

Construtor parametrizado da classe [Flow\\_Impl](#).

#### Parameters

|        |                                     |
|--------|-------------------------------------|
| name   | Nome do fluxo.                      |
| source | Sistema de origem.                  |
| target | Sistema de destino.                 |
| name   | Nome do fluxo.                      |
| source | Ponteiro para o sistema de origem.  |
| target | Ponteiro para o sistema de destino. |



## 7.2.1.3 Flow\_Impl() [3/3]

Flow\_Impl::Flow\_Impl (   
                     const [Flow\\_Impl](#) & other)   
 Construtor de cópia da classe [Flow](#).   
 Construtor de cópia da classe [Flow\\_Impl](#).

## Parameters

|       |                                                 |
|-------|-------------------------------------------------|
| other | Outro objeto <a href="#">Flow</a> .             |
| other | Objeto <a href="#">Flow_Impl</a> a ser copiado. |

Here is the call graph for this function:



## 7.2.1.4 ~Flow\_Impl()

Flow\_Impl::~~Flow\_Impl () [virtual]   
 Destrutor da classe [Flow](#).   
 Destrutor da classe [Flow\\_Impl](#).

## 7.2.2 Member Function Documentation

## 7.2.2.1 execute()

virtual double Flow\_Impl::execute () [pure virtual]   
 Método abstrato.   
 Executa o cálculo do fluxo.   
 Método abstrato implementado pelas subclasses.

## Returns

Valor calculado pelo fluxo.

Implements [Flow](#).

Implemented in [Flow\\_Test](#), [FlowComplexo](#), [FlowExponencial](#), [FlowLogistico](#), and [FlowTest](#).

## 7.2.2.2 getName()

string Flow\_Impl::getName () const [virtual]   
 Getters e setters.   
 Retorna o nome do fluxo.   
 Retorna o nome do fluxo.

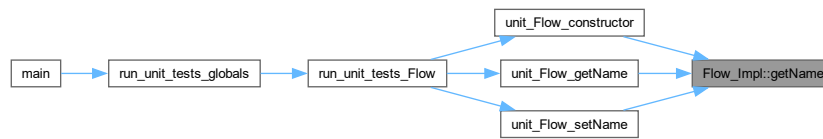
## Returns

Nome do fluxo.

Nome do fluxo.

Implements [Flow](#).

Here is the caller graph for this function:



#### 7.2.2.3 getSource()

**System** \* `Flow_Impl::getSource () const` [virtual]

Retorna o sistema de origem.

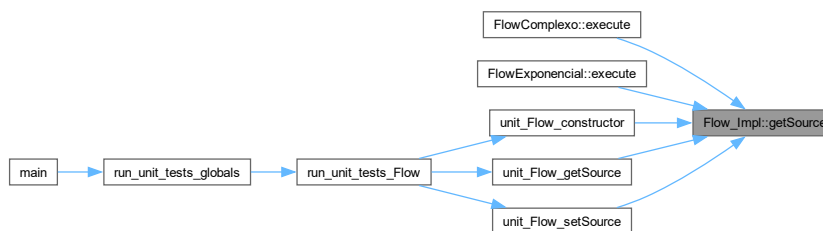
Retorna o ponteiro para o sistema de origem.

Returns

Ponteiro para o sistema de origem.

Implements [Flow](#).

Here is the caller graph for this function:



#### 7.2.2.4 getTarget()

**System** \* `Flow_Impl::getTarget () const` [virtual]

Retorna o sistema de destino.

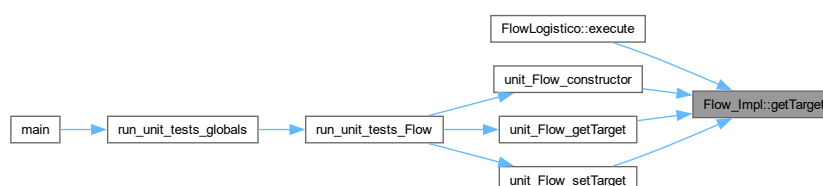
Retorna o ponteiro para o sistema de destino.

Returns

Ponteiro para o sistema de destino.

Implements [Flow](#).

Here is the caller graph for this function:



## 7.2.2.5 operator=()

```
Flow_Impl & Flow_Impl::operator= (
    const Flow_Impl & other)
```

Operador de atribuição da classe [Flow](#).

Operador de atribuição da classe [Flow\\_Impl](#).

Copia os atributos de outro fluxo.

## Parameters

|       |                                     |
|-------|-------------------------------------|
| other | Outro objeto <a href="#">Flow</a> . |
|-------|-------------------------------------|

## Returns

Referência para o objeto atual.

Copia nome, source e target de outro objeto [Flow\\_Impl](#).

## Parameters

|       |                                                   |
|-------|---------------------------------------------------|
| other | Objeto <a href="#">Flow_Impl</a> a ser atribuído. |
|-------|---------------------------------------------------|

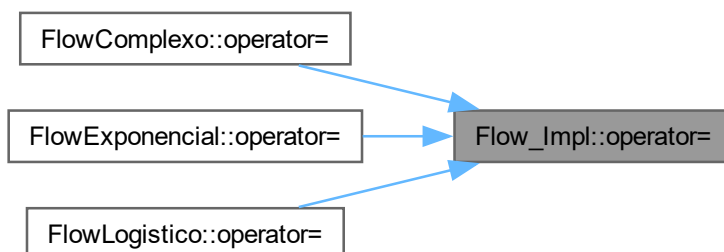
## Returns

Referência para o objeto atual.

Here is the call graph for this function:



Here is the caller graph for this function:



### 7.2.2.6 setName()

```
void Flow_Impl::setName (
    const string & name) [virtual]
```

Define o nome do fluxo.

Parameters

|      |                     |
|------|---------------------|
| name | Novo nome do fluxo. |
|------|---------------------|

Implements [Flow](#).

Here is the caller graph for this function:



### 7.2.2.7 setSource()

```
void Flow_Impl::setSource (
    System * source) [virtual]
```

Define o sistema de origem.

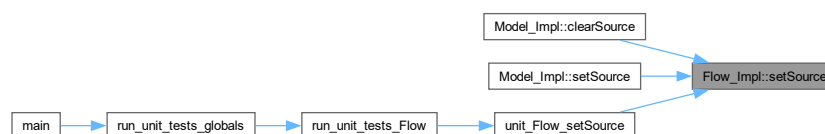
Define o sistema de origem do fluxo.

Parameters

|        |                                         |
|--------|-----------------------------------------|
| source | Novo sistema de origem.                 |
| source | Ponteiro para o novo sistema de origem. |

Implements [Flow](#).

Here is the caller graph for this function:



### 7.2.2.8 setTarget()

```
void Flow_Impl::setTarget (
    System * target) [virtual]
```

Define o sistema de destino.

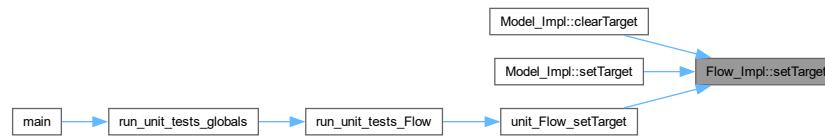
Define o sistema de destino do fluxo.

Parameters

|        |                                          |
|--------|------------------------------------------|
| target | Novo sistema de destino.                 |
| target | Ponteiro para o novo sistema de destino. |

Implements [Flow](#).

Here is the caller graph for this function:



## 7.2.3 Field Documentation

### 7.2.3.1 name

`string Flow_Impl::name` [protected]

### 7.2.3.2 source

`System* Flow_Impl::source` [protected]

### 7.2.3.3 target

`System* Flow_Impl::target` [protected]

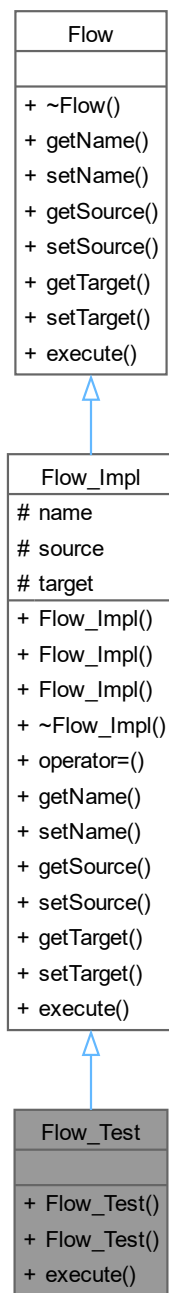
The documentation for this class was generated from the following files:

- [src/flow\\_impl.h](#)
- [src/flow\\_impl.cpp](#)

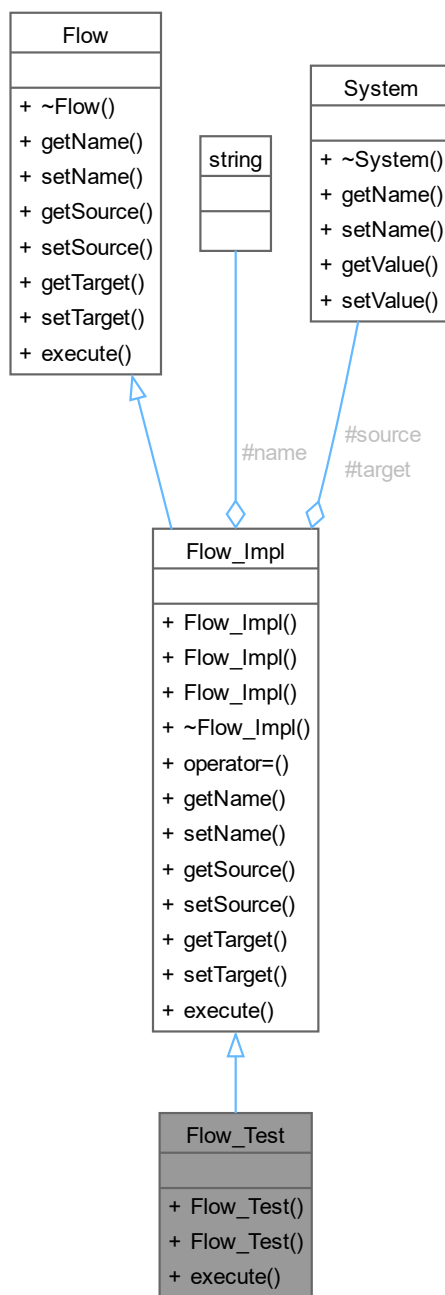
## 7.3 Flow\_Test Class Reference

Implementação concreta de [Flow\\_Impl](#) utilizada nos testes unitários.

Inheritance diagram for Flow\_Test:



Collaboration diagram for Flow\_Test:



#### Public Member Functions

- [Flow\\_Test](#) ()  
Construtor padrão de [Flow\\_Test](#).
- [Flow\\_Test](#) (string name, [System](#) \*source, [System](#) \*target)  
Construtor parametrizado de [Flow\\_Test](#).
- double [execute](#) () override  
Implementação trivial de [execute\(\)](#) para fins de teste.

## Public Member Functions inherited from [Flow\\_Impl](#)

- [Flow\\_Impl](#) ()  
Forma canônica.
- [Flow\\_Impl](#) (const string &name, [System](#) \*source, [System](#) \*target)  
Construtor parametrizado da classe [Flow](#).
- [Flow\\_Impl](#) (const [Flow\\_Impl](#) &other)  
Construtor de cópia da classe [Flow](#).
- virtual [~Flow\\_Impl](#) ()  
Destrutor da classe [Flow](#).
- [Flow\\_Impl](#) & operator= (const [Flow\\_Impl](#) &other)  
Operador de atribuição da classe [Flow](#).
- string [getName](#) () const  
Getters e setters.
- void [setName](#) (const string &name)  
Define o nome do fluxo.
- [System](#) \* [getSource](#) () const  
Retorna o sistema de origem.
- void [setSource](#) ([System](#) \*source)  
Define o sistema de origem.
- [System](#) \* [getTarget](#) () const  
Retorna o sistema de destino.
- void [setTarget](#) ([System](#) \*target)  
Define o sistema de destino.

## Public Member Functions inherited from [Flow](#)

- virtual [~Flow](#) ()=default  
Destrutor virtual da classe [Flow](#).

## Additional Inherited Members

## Protected Attributes inherited from [Flow\\_Impl](#)

- string [name](#)
- [System](#) \* [source](#)
- [System](#) \* [target](#)

### 7.3.1 Detailed Description

Implementação concreta de [Flow\\_Impl](#) utilizada nos testes unitários.

Classe auxiliar de teste que herda de [Flow\\_Impl](#) e fornece uma implementação trivial de [execute\(\)](#), retornando sempre 1.0. Permite testar os métodos de [Flow\\_Impl](#) sem depender de uma equação de fluxo real.

### 7.3.2 Constructor & Destructor Documentation

#### 7.3.2.1 [Flow\\_Test](#)() [1/2]

[Flow\\_Test::Flow\\_Test](#) () [inline]

Construtor padrão de [Flow\\_Test](#).



Here is the call graph for this function:



### 7.3.2.2 Flow\_Test() [2/2]

```
Flow_Test::Flow_Test (
    string name,
    System * source,
    System * target) [inline]
```

Construtor parametrizado de [Flow\\_Test](#).

Parameters

|        |                                     |
|--------|-------------------------------------|
| name   | Nome do fluxo.                      |
| source | Ponteiro para o sistema de origem.  |
| target | Ponteiro para o sistema de destino. |

Here is the call graph for this function:



## 7.3.3 Member Function Documentation

### 7.3.3.1 execute()

```
double Flow_Test::execute () [inline], [override], [virtual]
```

Implementação trivial de [execute\(\)](#) para fins de teste.

Returns

Sempre retorna 1.0.

Implements [Flow\\_Impl](#).

Here is the caller graph for this function:



The documentation for this class was generated from the following file:

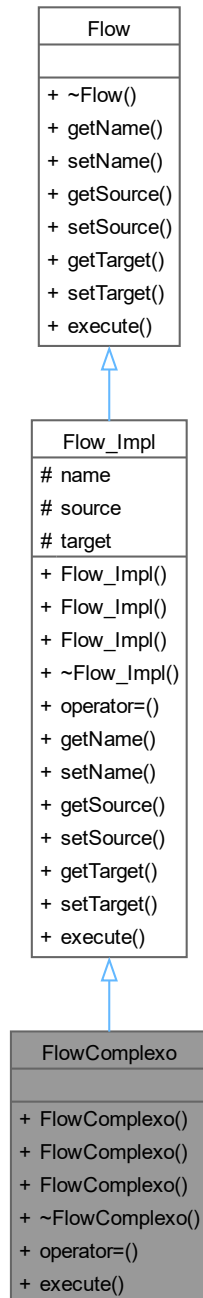
- test/unit/[unit\\_flow.cpp](#)

## 7.4 FlowComplexo Class Reference

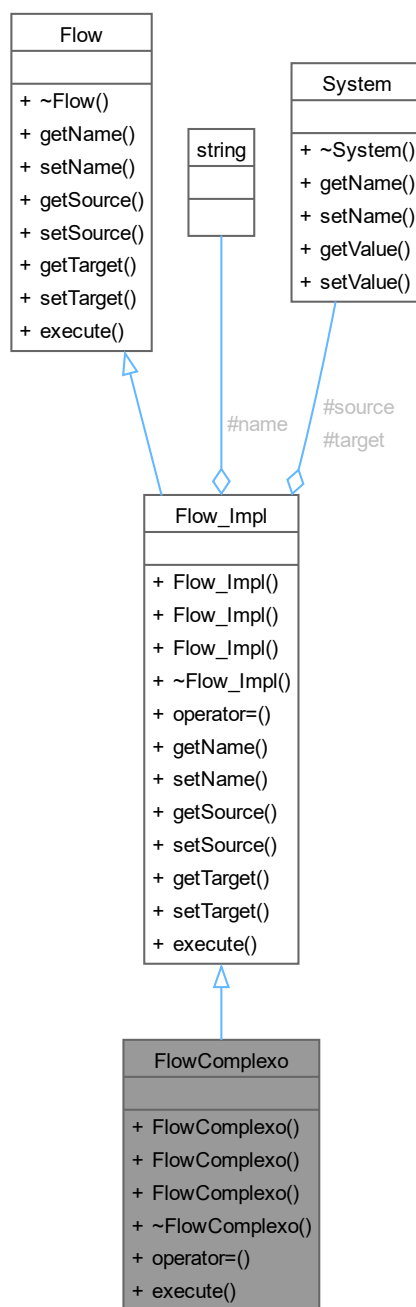
Fluxo com equação de múltiplas interações entre sistemas.

```
#include <flows.h>
```

Inheritance diagram for FlowComplexo:



Collaboration diagram for FlowComplexo:



#### Public Member Functions

- [FlowComplexo](#) ()  
Construtor padrão da classe [FlowComplexo](#).
- [FlowComplexo](#) (const string &name, [System](#) \*source, [System](#) \*target)  
Construtor parametrizado da classe [FlowComplexo](#).
- [FlowComplexo](#) (const [FlowComplexo](#) &other)  
Construtor de cópia da classe [FlowComplexo](#).

- virtual `~FlowComplexo ()`  
Destrutor virtual da classe `FlowComplexo`.
- `FlowComplexo & operator= (const FlowComplexo &other)`  
Operador de atribuição da classe `FlowComplexo`.
- double `execute ()` override  
Executa o cálculo do fluxo complexo.

#### Public Member Functions inherited from `Flow_Impl`

- `Flow_Impl ()`  
Forma canônica.
- `Flow_Impl (const string &name, System *source, System *target)`  
Construtor parametrizado da classe `Flow`.
- `Flow_Impl (const Flow_Impl &other)`  
Construtor de cópia da classe `Flow`.
- virtual `~Flow_Impl ()`  
Destrutor da classe `Flow`.
- `Flow_Impl & operator= (const Flow_Impl &other)`  
Operador de atribuição da classe `Flow`.
- string `getName ()` const  
Getters e setters.
- void `setName (const string &name)`  
Define o nome do fluxo.
- `System * getSource ()` const  
Retorna o sistema de origem.
- void `setSource (System *source)`  
Define o sistema de origem.
- `System * getTarget ()` const  
Retorna o sistema de destino.
- void `setTarget (System *target)`  
Define o sistema de destino.

#### Public Member Functions inherited from `Flow`

- virtual `~Flow ()=default`  
Destrutor virtual da classe `Flow`.

#### Additional Inherited Members

#### Protected Attributes inherited from `Flow_Impl`

- string `name`
- `System * source`
- `System * target`

### 7.4.1 Detailed Description

Fluxo com equação de múltiplas interações entre sistemas.

Implementa um fluxo mais elaborado que combina interações entre o sistema de origem e o sistema de destino, permitindo modelar dinâmicas com dependências cruzadas entre compartimentos, como modelos predador-presa ou transferências bidirecionalmente influenciadas.

## 7.4.2 Constructor & Destructor Documentation

### 7.4.2.1 FlowComplexo() [1/3]

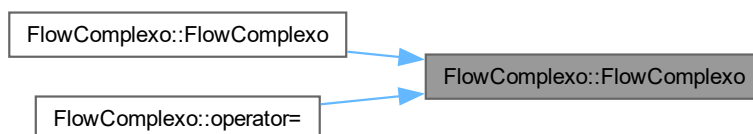
FlowComplexo::FlowComplexo ()

Construtor padrão da classe [FlowComplexo](#).

Here is the call graph for this function:



Here is the caller graph for this function:



### 7.4.2.2 FlowComplexo() [2/3]

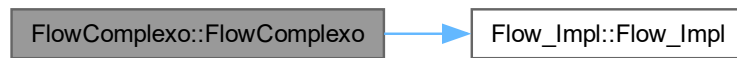
FlowComplexo::FlowComplexo (  
     const string & name,  
     [System](#) \* source,  
     [System](#) \* target)

Construtor parametrizado da classe [FlowComplexo](#).

#### Parameters

|        |                                     |
|--------|-------------------------------------|
| name   | Nome do fluxo.                      |
| source | Ponteiro para o sistema de origem.  |
| target | Ponteiro para o sistema de destino. |
| name   | Nome do fluxo.                      |
| source | Sistema de origem.                  |
| target | Sistema de destino.                 |

Here is the call graph for this function:



#### 7.4.2.3 FlowComplexo() [3/3]

FlowComplexo::FlowComplexo (  
     const FlowComplexo & other)  
 Construtor de cópia da classe FlowComplexo.

Parameters

|       |                                    |
|-------|------------------------------------|
| other | Objeto FlowComplexo a ser copiado. |
| other | Outro objeto FlowComplexo.         |

Here is the call graph for this function:



#### 7.4.2.4 ~FlowComplexo()

FlowComplexo::~~FlowComplexo () [virtual]  
 Destrutor virtual da classe FlowComplexo.  
 Destrutor da classe FlowComplexo.

### 7.4.3 Member Function Documentation

#### 7.4.3.1 execute()

double FlowComplexo::execute () [override], [virtual]  
 Executa o cálculo do fluxo complexo.

Calcula a quantidade transferida entre os sistemas com base em uma equação que combina os valores do sistema de origem e do sistema de destino, modelando interações mais elaboradas entre os compartimentos do modelo.

Returns

Valor calculado pelo fluxo complexo.

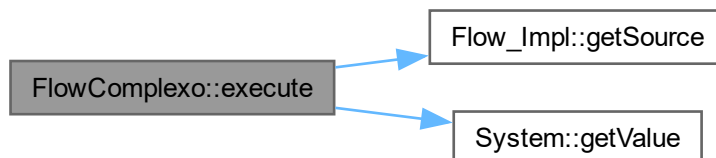
Calcula a transferência entre sistemas utilizando o modelo complexo.

## Returns

Valor calculado do fluxo.

Implements [Flow\\_Impl](#).

Here is the call graph for this function:



## 7.4.3.2 operator=()

[FlowComplexo](#) & `FlowComplexo::operator=` (  
const [FlowComplexo](#) & other)

Operador de atribuição da classe [FlowComplexo](#).

## Parameters

|       |                                                      |
|-------|------------------------------------------------------|
| other | Objeto <a href="#">FlowComplexo</a> a ser atribuído. |
|-------|------------------------------------------------------|

## Returns

Referência para o objeto atual.

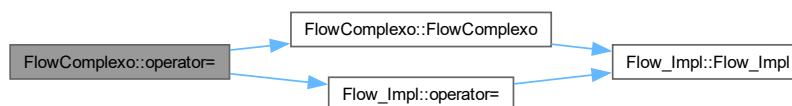
## Parameters

|       |                                             |
|-------|---------------------------------------------|
| other | Outro objeto <a href="#">FlowComplexo</a> . |
|-------|---------------------------------------------|

## Returns

Referência para o objeto atual.

Here is the call graph for this function:



The documentation for this class was generated from the following files:

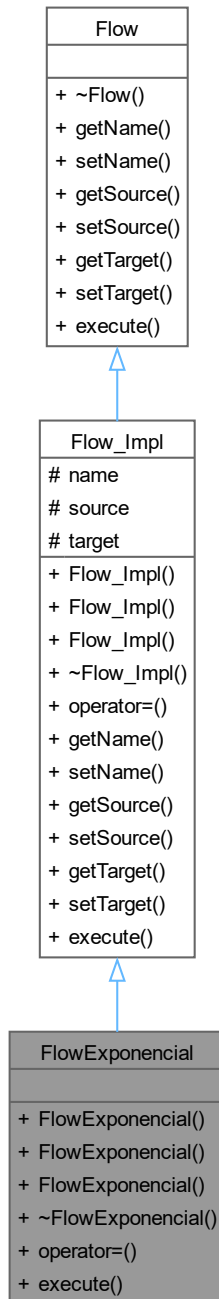
- test/functional/[flows.h](#)
- test/functional/[flows.cpp](#)

## 7.5 FlowExponencial Class Reference

Fluxo baseado em crescimento exponencial.

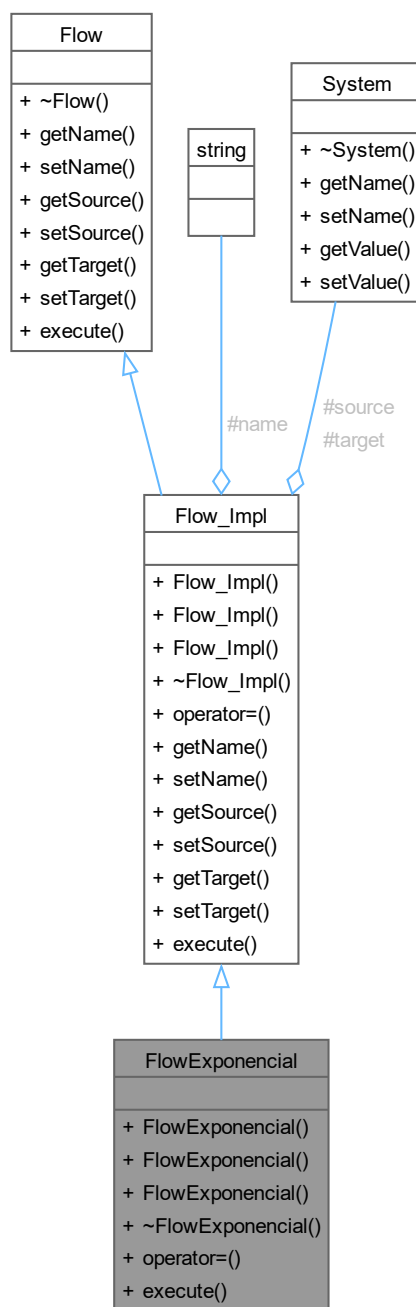
```
#include <flows.h>
```

Inheritance diagram for FlowExponencial:





Collaboration diagram for FlowExponential:



#### Public Member Functions

- [FlowExponential](#) ()  
Construtor padrão da classe [FlowExponential](#).
- [FlowExponential](#) (const string &name, [System](#) \*source, [System](#) \*target)  
Construtor parametrizado da classe [FlowExponential](#).
- [FlowExponential](#) (const [FlowExponential](#) &other)  
Construtor de cópia da classe [FlowExponential](#).

- virtual `~FlowExponencial ()`  
Destrutor virtual da classe `FlowExponencial`.
- `FlowExponencial & operator= (const FlowExponencial &other)`  
Operador de atribuição da classe `FlowExponencial`.
- double `execute ()` override  
Executa o cálculo do fluxo exponencial.

#### Public Member Functions inherited from `Flow_Impl`

- `Flow_Impl ()`  
Forma canônica.
- `Flow_Impl (const string &name, System *source, System *target)`  
Construtor parametrizado da classe `Flow`.
- `Flow_Impl (const Flow_Impl &other)`  
Construtor de cópia da classe `Flow`.
- virtual `~Flow_Impl ()`  
Destrutor da classe `Flow`.
- `Flow_Impl & operator= (const Flow_Impl &other)`  
Operador de atribuição da classe `Flow`.
- string `getName ()` const  
Getters e setters.
- void `setName (const string &name)`  
Define o nome do fluxo.
- `System * getSource ()` const  
Retorna o sistema de origem.
- void `setSource (System *source)`  
Define o sistema de origem.
- `System * getTarget ()` const  
Retorna o sistema de destino.
- void `setTarget (System *target)`  
Define o sistema de destino.

#### Public Member Functions inherited from `Flow`

- virtual `~Flow ()=default`  
Destrutor virtual da classe `Flow`.

#### Additional Inherited Members

#### Protected Attributes inherited from `Flow_Impl`

- string `name`
- `System * source`
- `System * target`

### 7.5.1 Detailed Description

Fluxo baseado em crescimento exponencial.

Implementa uma equação de fluxo exponencial entre dois sistemas, onde a taxa de transferência é proporcional ao valor atual do sistema de origem. Tipicamente usada para modelar crescimento ou decaimento exponencial em populações e recursos.

## 7.5.2 Constructor & Destructor Documentation

### 7.5.2.1 FlowExponencial() [1/3]

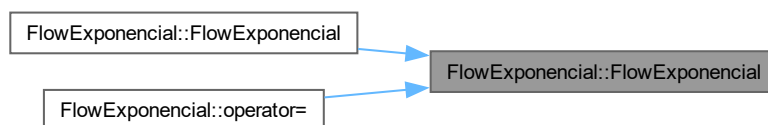
FlowExponencial::FlowExponencial ()

Construtor padrão da classe [FlowExponencial](#).

Here is the call graph for this function:



Here is the caller graph for this function:



### 7.5.2.2 FlowExponencial() [2/3]

FlowExponencial::FlowExponencial (  
     const string & name,  
     [System](#) \* source,  
     [System](#) \* target)

Construtor parametrizado da classe [FlowExponencial](#).

Parameters

|        |                                     |
|--------|-------------------------------------|
| name   | Nome do fluxo.                      |
| source | Ponteiro para o sistema de origem.  |
| target | Ponteiro para o sistema de destino. |
| name   | Nome do fluxo.                      |
| source | Sistema de origem.                  |
| target | Sistema de destino.                 |

Here is the call graph for this function:



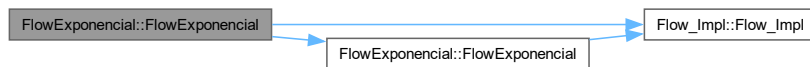
### 7.5.2.3 FlowExponencial() [3/3]

FlowExponencial::FlowExponencial (  
     const FlowExponencial & other)  
 Construtor de cópia da classe FlowExponencial.

Parameters

|       |                                       |
|-------|---------------------------------------|
| other | Objeto FlowExponencial a ser copiado. |
| other | Outro objeto FlowExponencial.         |

Here is the call graph for this function:



### 7.5.2.4 ~FlowExponencial()

FlowExponencial::~FlowExponencial () [virtual]  
 Destrutor virtual da classe FlowExponencial.  
 Destrutor da classe FlowExponencial.

## 7.5.3 Member Function Documentation

### 7.5.3.1 execute()

double FlowExponencial::execute () [override], [virtual]

Executa o cálculo do fluxo exponencial.

Calcula a quantidade transferida do sistema de origem para o sistema de destino com base em uma equação de crescimento exponencial aplicada ao valor atual do sistema de origem.

Returns

Valor calculado pelo fluxo exponencial.

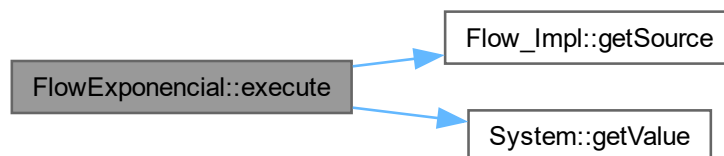
O valor transferido corresponde a 1% do valor do sistema de origem.

Returns

Valor calculado do fluxo.

Implements [Flow\\_Impl](#).

Here is the call graph for this function:



### 7.5.3.2 operator=()

[FlowExponencial](#) & FlowExponencial::operator= (  
     const [FlowExponencial](#) & other)

Operador de atribuição da classe [FlowExponencial](#).

Parameters

|       |                                                         |
|-------|---------------------------------------------------------|
| other | Objeto <a href="#">FlowExponencial</a> a ser atribuído. |
|-------|---------------------------------------------------------|

Returns

Referência para o objeto atual.

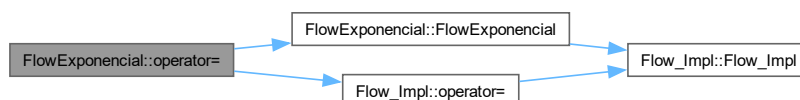
Parameters

|       |                                                |
|-------|------------------------------------------------|
| other | Outro objeto <a href="#">FlowExponencial</a> . |
|-------|------------------------------------------------|

Returns

Referência para o objeto atual.

Here is the call graph for this function:



The documentation for this class was generated from the following files:

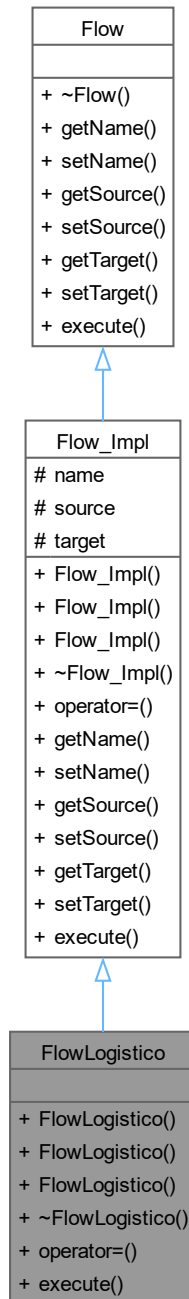
- test/functional/[flows.h](#)
- test/functional/[flows.cpp](#)

## 7.6 FlowLogistico Class Reference

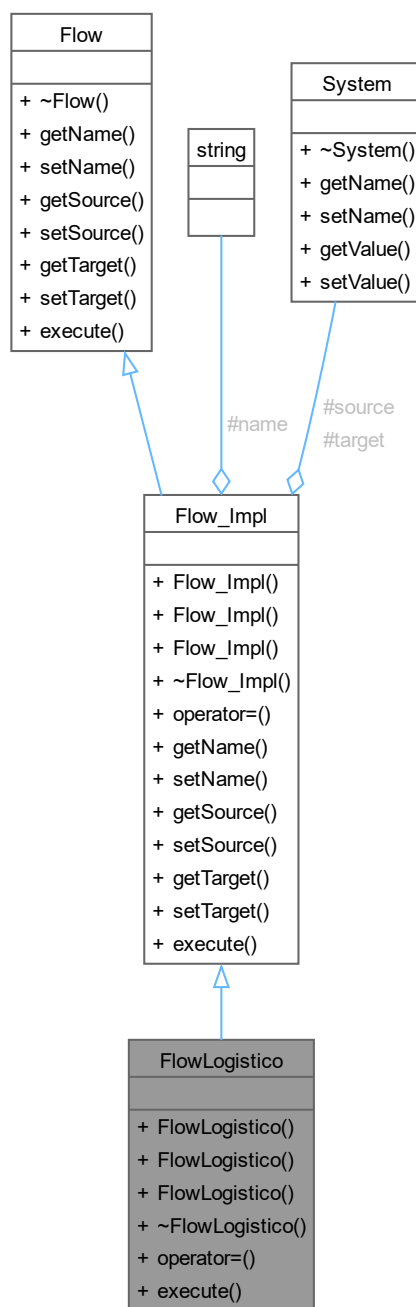
Fluxo baseado em crescimento logístico.

```
#include <flows.h>
```

Inheritance diagram for FlowLogistico:



Collaboration diagram for FlowLogistico:



#### Public Member Functions

- [FlowLogistico](#) ()  
Construtor padrão da classe [FlowLogistico](#).
- [FlowLogistico](#) (const string &name, [System](#) \*source, [System](#) \*target)  
Construtor parametrizado da classe [FlowLogistico](#).
- [FlowLogistico](#) (const [FlowLogistico](#) &other)  
Construtor de cópia da classe [FlowLogistico](#).

- virtual `~FlowLogistico ()`  
Destrutor virtual da classe `FlowLogistico`.
- `FlowLogistico & operator= (const FlowLogistico &other)`  
Operador de atribuição da classe `FlowLogistico`.
- double `execute ()` override  
Executa o cálculo do fluxo logístico.

#### Public Member Functions inherited from `Flow_Impl`

- `Flow_Impl ()`  
Forma canônica.
- `Flow_Impl (const string &name, System *source, System *target)`  
Construtor parametrizado da classe `Flow`.
- `Flow_Impl (const Flow_Impl &other)`  
Construtor de cópia da classe `Flow`.
- virtual `~Flow_Impl ()`  
Destrutor da classe `Flow`.
- `Flow_Impl & operator= (const Flow_Impl &other)`  
Operador de atribuição da classe `Flow`.
- string `getName ()` const  
Getters e setters.
- void `setName (const string &name)`  
Define o nome do fluxo.
- `System * getSource ()` const  
Retorna o sistema de origem.
- void `setSource (System *source)`  
Define o sistema de origem.
- `System * getTarget ()` const  
Retorna o sistema de destino.
- void `setTarget (System *target)`  
Define o sistema de destino.

#### Public Member Functions inherited from `Flow`

- virtual `~Flow ()=default`  
Destrutor virtual da classe `Flow`.

#### Additional Inherited Members

#### Protected Attributes inherited from `Flow_Impl`

- string `name`
- `System * source`
- `System * target`

### 7.6.1 Detailed Description

Fluxo baseado em crescimento logístico.

Implementa uma equação de fluxo logístico entre dois sistemas, onde a taxa de transferência desacelera conforme o sistema de origem se aproxima de uma capacidade limite. Usada para modelar crescimento populacional com capacidade de suporte.



## 7.6.2 Constructor & Destructor Documentation

### 7.6.2.1 FlowLogistico() [1/3]

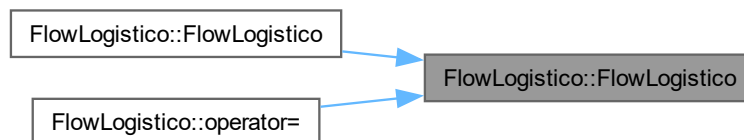
FlowLogistico::FlowLogistico ()

Construtor padrão da classe [FlowLogistico](#).

Here is the call graph for this function:



Here is the caller graph for this function:



### 7.6.2.2 FlowLogistico() [2/3]

FlowLogistico::FlowLogistico (  
     const string & name,  
     [System](#) \* source,  
     [System](#) \* target)

Construtor parametrizado da classe [FlowLogistico](#).

#### Parameters

|        |                                     |
|--------|-------------------------------------|
| name   | Nome do fluxo.                      |
| source | Ponteiro para o sistema de origem.  |
| target | Ponteiro para o sistema de destino. |
| name   | Nome do fluxo.                      |
| source | Sistema de origem.                  |
| target | Sistema de destino.                 |

Here is the call graph for this function:



#### 7.6.2.3 FlowLogistico() [3/3]

FlowLogistico::FlowLogistico (   
                   const FlowLogistico & other)   
 Construtor de cópia da classe FlowLogistico.

##### Parameters

|       |                                     |
|-------|-------------------------------------|
| other | Objeto FlowLogistico a ser copiado. |
| other | Outro objeto FlowLogistico.         |

Here is the call graph for this function:



#### 7.6.2.4 ~FlowLogistico()

FlowLogistico::~FlowLogistico () [virtual]   
 Destrutor virtual da classe FlowLogistico.   
 Destrutor da classe FlowLogistico.

### 7.6.3 Member Function Documentation

#### 7.6.3.1 execute()

double FlowLogistico::execute () [override], [virtual]

Executa o cálculo do fluxo logístico.

Calcula a quantidade transferida do sistema de origem para o sistema de destino com base em uma equação de crescimento logístico, levando em conta os valores atuais de ambos os sistemas.

##### Returns

Valor calculado pelo fluxo logístico.

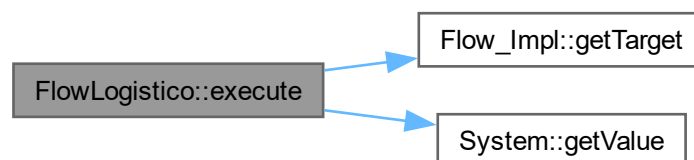
Calcula o crescimento logístico considerando a capacidade limite do sistema.

## Returns

Valor calculado do fluxo.

Implements [Flow\\_Impl](#).

Here is the call graph for this function:



## 7.6.3.2 operator=()

[FlowLogistico](#) & `FlowLogistico::operator=` (  
     const [FlowLogistico](#) & other)

Operador de atribuição da classe [FlowLogistico](#).

## Parameters

|       |                                                       |
|-------|-------------------------------------------------------|
| other | Objeto <a href="#">FlowLogistico</a> a ser atribuído. |
|-------|-------------------------------------------------------|

## Returns

Referência para o objeto atual.

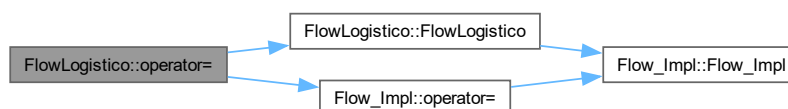
## Parameters

|       |                                              |
|-------|----------------------------------------------|
| other | Outro objeto <a href="#">FlowLogistico</a> . |
|-------|----------------------------------------------|

## Returns

Referência para o objeto atual.

Here is the call graph for this function:



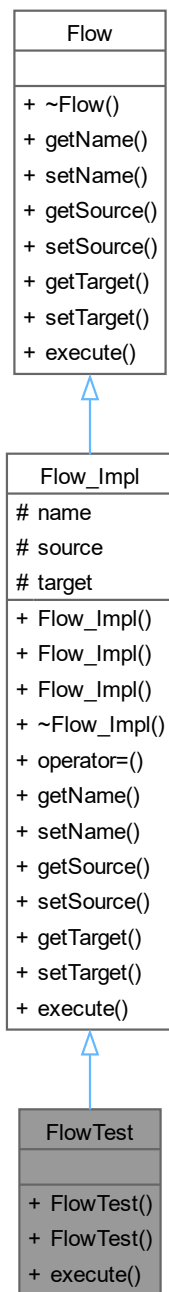
The documentation for this class was generated from the following files:

- test/functional/[flows.h](#)
- test/functional/[flows.cpp](#)

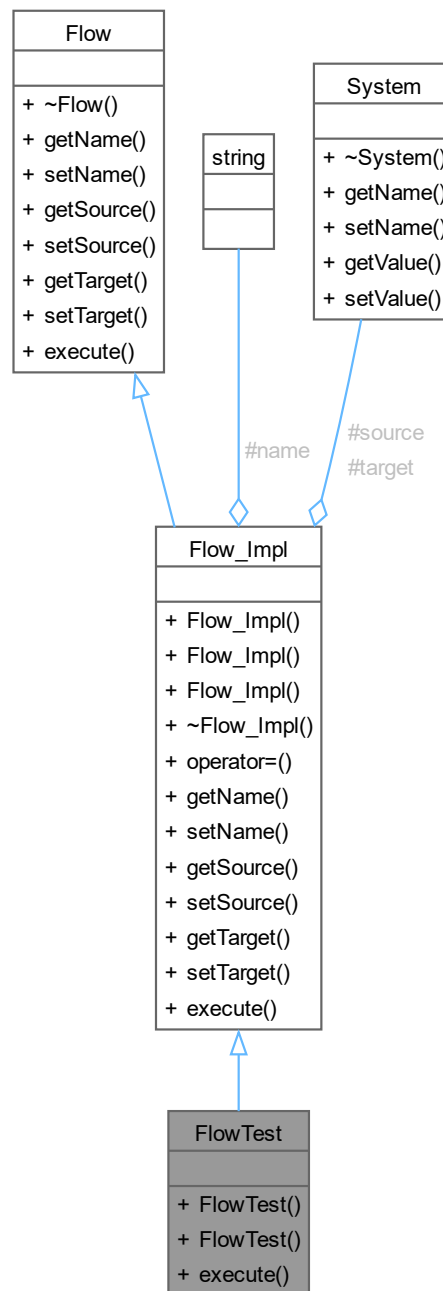
## 7.7 FlowTest Class Reference

Implementação concreta de [Flow\\_Impl](#) utilizada nos testes do [Model](#).

Inheritance diagram for FlowTest:



Collaboration diagram for FlowTest:



#### Public Member Functions

- [FlowTest](#) ()  
Construtor padrão de [FlowTest](#).
- [FlowTest](#) (string name, [System](#) \*source, [System](#) \*target)  
Construtor parametrizado de [FlowTest](#).
- double [execute](#) () override  
Implementação trivial de [execute\(\)](#) para fins de teste.

## Public Member Functions inherited from [Flow\\_Impl](#)

- [Flow\\_Impl](#) ()  
Forma canônica.
- [Flow\\_Impl](#) (const string &name, [System](#) \*source, [System](#) \*target)  
Construtor parametrizado da classe [Flow](#).
- [Flow\\_Impl](#) (const [Flow\\_Impl](#) &other)  
Construtor de cópia da classe [Flow](#).
- virtual [~Flow\\_Impl](#) ()  
Destrutor da classe [Flow](#).
- [Flow\\_Impl](#) & operator= (const [Flow\\_Impl](#) &other)  
Operador de atribuição da classe [Flow](#).
- string [getName](#) () const  
Getters e setters.
- void [setName](#) (const string &name)  
Define o nome do fluxo.
- [System](#) \* [getSource](#) () const  
Retorna o sistema de origem.
- void [setSource](#) ([System](#) \*source)  
Define o sistema de origem.
- [System](#) \* [getTarget](#) () const  
Retorna o sistema de destino.
- void [setTarget](#) ([System](#) \*target)  
Define o sistema de destino.

## Public Member Functions inherited from [Flow](#)

- virtual [~Flow](#) ()=default  
Destrutor virtual da classe [Flow](#).

## Additional Inherited Members

## Protected Attributes inherited from [Flow\\_Impl](#)

- string [name](#)
- [System](#) \* [source](#)
- [System](#) \* [target](#)

### 7.7.1 Detailed Description

Implementação concreta de [Flow\\_Impl](#) utilizada nos testes do [Model](#).

Classe auxiliar de teste que herda de [Flow\\_Impl](#) e fornece uma implementação trivial de [execute\(\)](#), retornando sempre 1.0. Permite testar a lógica de execução do [Model](#) sem depender de uma equação de fluxo real.

### 7.7.2 Constructor & Destructor Documentation

#### 7.7.2.1 FlowTest() [1/2]

[FlowTest::FlowTest](#) () [inline]

Construtor padrão de [FlowTest](#).

Here is the call graph for this function:



### 7.7.2.2 FlowTest() [2/2]

```
FlowTest::FlowTest (
    string name,
    System * source,
    System * target) [inline]
```

Construtor parametrizado de [FlowTest](#).

Parameters

|        |                                     |
|--------|-------------------------------------|
| name   | Nome do fluxo.                      |
| source | Ponteiro para o sistema de origem.  |
| target | Ponteiro para o sistema de destino. |

Here is the call graph for this function:



## 7.7.3 Member Function Documentation

### 7.7.3.1 execute()

```
double FlowTest::execute () [inline], [override], [virtual]
```

Implementação trivial de [execute\(\)](#) para fins de teste.

Returns

Sempre retorna 1.0.

Implements [Flow\\_Impl](#).

The documentation for this class was generated from the following file:

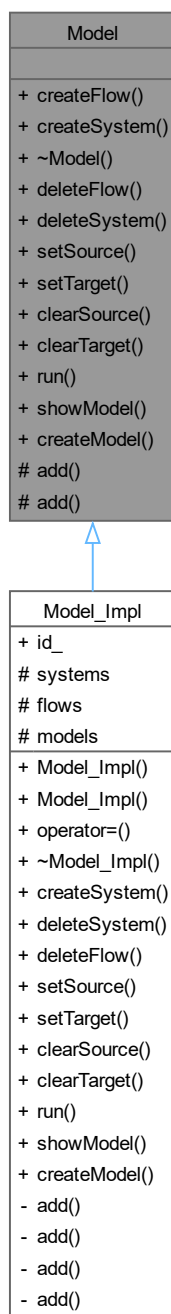
- test/unit/[unit\\_model.cpp](#)

## 7.8 Model Class Reference

Interface abstrata que representa um modelo de dinâmica de sistemas.

```
#include <model.h>
```

Inheritance diagram for Model:





Collaboration diagram for Model:

| Model                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>+ createFlow()</li> <li>+ createSystem()</li> <li>+ ~Model()</li> <li>+ deleteFlow()</li> <li>+ deleteSystem()</li> <li>+ setSource()</li> <li>+ setTarget()</li> <li>+ clearSource()</li> <li>+ clearTarget()</li> <li>+ run()</li> <li>+ showModel()</li> <li>+ createModel()</li> <li># add()</li> <li># add()</li> </ul> |

#### Public Member Functions

- `template<typename T_FLOW_IMPL>`  
`Flow & createFlow` (string id, `System *source=NULL`, `System *target=NULL`)  
 Cria e adiciona um fluxo concreto ao modelo.
- `virtual System & createSystem` (string id, double)=0  
 Cria e adiciona um sistema ao modelo.
- `virtual ~Model` ()=default  
 Destrutor virtual da classe `Model`.
- `virtual bool deleteFlow` (`Flow &`)=0  
 Remove um fluxo do modelo.
- `virtual bool deleteSystem` (`System &`)=0  
 Remove um sistema do modelo.
- `virtual void setSource` (`Flow &`, `System &`)=0  
 Define o sistema de origem de um fluxo.
- `virtual void setTarget` (`Flow &`, `System &`)=0  
 Define o sistema de destino de um fluxo.
- `virtual void clearSource` (`Flow &`)=0  
 Remove o sistema de origem de um fluxo, definindo-o como nulo.
- `virtual void clearTarget` (`Flow &`)=0  
 Remove o sistema de destino de um fluxo, definindo-o como nulo.
- `virtual bool run` (int t\_init, int t\_final)=0  
 Executa a simulação do modelo no intervalo de tempo especificado.

- virtual void [showModel](#) () const =0  
Exibe no console as informações dos sistemas e fluxos do modelo.

#### Static Public Member Functions

- static [Model](#) & [createModel](#) (string id=0)  
Cria e registra uma nova instância de [Model](#).

#### Protected Member Functions

- virtual void [add](#) ([Model](#) \*)=0  
Registra um modelo no vetor estático interno de modelos.
- virtual void [add](#) ([Flow](#) \*)=0  
Adiciona um fluxo à lista interna do modelo.

### 7.8.1 Detailed Description

Interface abstrata que representa um modelo de dinâmica de sistemas.

A classe [Model](#) define a interface para um modelo de simulação composto por sistemas (estoques) e fluxos (equações de transferência). O modelo é responsável por gerenciar esses elementos e conduzir a simulação ao longo de um intervalo de tempo discreto. Subclasses devem fornecer implementação concreta para todos os métodos virtuais puros.

### 7.8.2 Constructor & Destructor Documentation

#### 7.8.2.1 ~Model()

virtual [Model](#)::~[Model](#) () [virtual], [default]  
Destrutor virtual da classe [Model](#).

### 7.8.3 Member Function Documentation

#### 7.8.3.1 add() [1/2]

virtual void [Model](#)::[add](#) (  
[Flow](#) \* ) [protected], [pure virtual]  
Adiciona um fluxo à lista interna do modelo.

#### Parameters

|   |                                         |
|---|-----------------------------------------|
| f | Ponteiro para o fluxo a ser adicionado. |
|---|-----------------------------------------|

Implemented in [Model\\_Impl](#).

#### 7.8.3.2 add() [2/2]

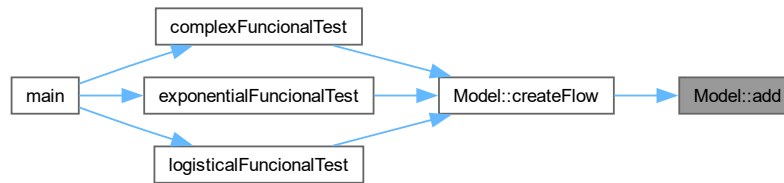
virtual void [Model](#)::[add](#) (  
[Model](#) \* ) [protected], [pure virtual]  
Registra um modelo no vetor estático interno de modelos.

#### Parameters

|   |                                          |
|---|------------------------------------------|
| m | Ponteiro para o modelo a ser registrado. |
|---|------------------------------------------|

Implemented in [Model\\_Impl](#).

Here is the caller graph for this function:



### 7.8.3.3 clearSource()

```
virtual void Model::clearSource (
    Flow & ) [pure virtual]
```

Remove o sistema de origem de um fluxo, definindo-o como nulo.

Parameters

|   |                                                 |
|---|-------------------------------------------------|
| f | Referência para o fluxo cujo source será limpo. |
|---|-------------------------------------------------|

Implemented in [Model\\_Impl](#).

### 7.8.3.4 clearTarget()

```
virtual void Model::clearTarget (
    Flow & ) [pure virtual]
```

Remove o sistema de destino de um fluxo, definindo-o como nulo.

Parameters

|   |                                                 |
|---|-------------------------------------------------|
| f | Referência para o fluxo cujo target será limpo. |
|---|-------------------------------------------------|

Implemented in [Model\\_Impl](#).

### 7.8.3.5 createFlow()

```
template<typename T_FLOW_IMPL>
Flow & Model::createFlow (
    string id,
    System * source = NULL,
    System * target = NULL) [inline]
```

Cria e adiciona um fluxo concreto ao modelo.

Método template que instancia qualquer subclasse de [Flow\\_Impl](#), vincula os sistemas de origem e destino fornecidos e registra o fluxo no modelo.

Template Parameters

|             |                                                                                  |
|-------------|----------------------------------------------------------------------------------|
| T_FLOW_IMPL | Tipo concreto do fluxo a ser criado (deve herdar de <a href="#">Flow_Impl</a> ). |
|-------------|----------------------------------------------------------------------------------|

## Parameters

|        |                                                             |
|--------|-------------------------------------------------------------|
| id     | Identificador do fluxo.                                     |
| source | Ponteiro para o sistema de origem (opcional, padrão NULL).  |
| target | Ponteiro para o sistema de destino (opcional, padrão NULL). |

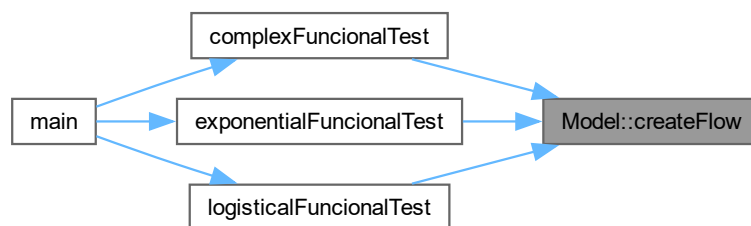
## Returns

Referência para o fluxo recém-criado.

Here is the call graph for this function:



Here is the caller graph for this function:



## 7.8.3.6 createModel()

`Model & Model::createModel (`  
     string id = 0) [static]

Cria e registra uma nova instância de `Model`.

Delega a criação de um modelo para `Model_Impl::createModel`.

Método estático de fábrica que instancia um `Model_Impl`, registra-o no vetor estático de modelos e o retorna por referência.

## Parameters

|    |                                |
|----|--------------------------------|
| id | Identificador único do modelo. |
|----|--------------------------------|

## Returns

Referência para o modelo recém-criado.

Ponto de entrada estático da interface [Model](#) que redireciona a chamada para a implementação concreta em [Model\\_Impl](#).

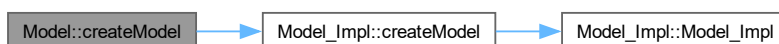
## Parameters

|    |                                |
|----|--------------------------------|
| id | Identificador único do modelo. |
|----|--------------------------------|

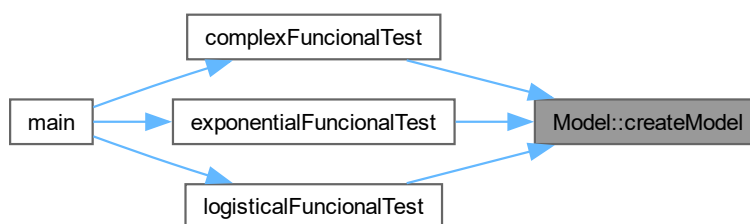
## Returns

Referência para o modelo recém-criado.

Here is the call graph for this function:



Here is the caller graph for this function:



## 7.8.3.7 createSystem()

```
virtual System & Model::createSystem (
    string id,
    double ) [pure virtual]
```

Cria e adiciona um sistema ao modelo.

Instancia um [System\\_Impl](#) com o identificador e valor inicial fornecidos e o registra no modelo.

## Parameters

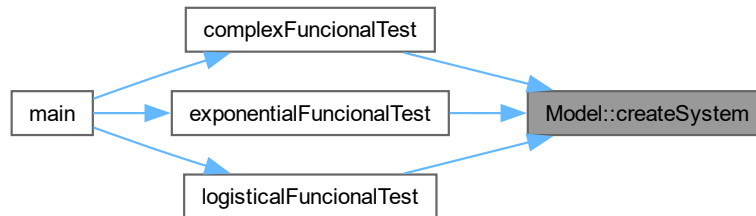
|       |                           |
|-------|---------------------------|
| id    | Identificador do sistema. |
| value | Valor inicial do sistema. |

## Returns

Referência para o sistema recém-criado.

Implemented in [Model\\_Impl](#).

Here is the caller graph for this function:

7.8.3.8 `deleteFlow()`

```
virtual bool Model::deleteFlow (
    Flow & ) [pure virtual]
```

Remove um fluxo do modelo.

## Parameters

|   |                                         |
|---|-----------------------------------------|
| f | Referência para o fluxo a ser removido. |
|---|-----------------------------------------|

## Returns

true se o fluxo foi removido com sucesso; false caso contrário.

Implemented in [Model\\_Impl](#).

7.8.3.9 `deleteSystem()`

```
virtual bool Model::deleteSystem (
    System & ) [pure virtual]
```

Remove um sistema do modelo.

## Parameters

|   |                                           |
|---|-------------------------------------------|
| s | Referência para o sistema a ser removido. |
|---|-------------------------------------------|

## Returns

true se o sistema foi removido com sucesso; false caso contrário.

Implemented in [Model\\_Impl](#).

7.8.3.10 `run()`

```
virtual bool Model::run (
    int t_init,
    int t_final) [pure virtual]
```

Executa a simulação do modelo no intervalo de tempo especificado.

A cada passo de tempo, todos os fluxos são calculados com base nos valores atuais dos sistemas, e os sistemas são atualizados em seguida.

#### Parameters

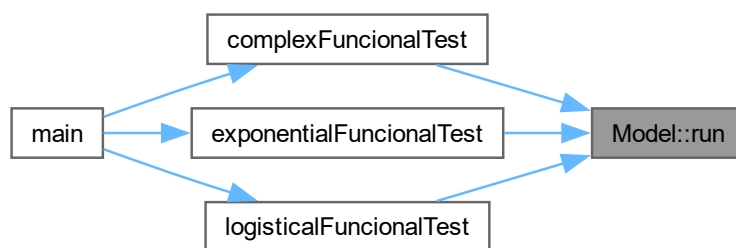
|         |                                         |
|---------|-----------------------------------------|
| t_init  | Tempo inicial da simulação (inclusive). |
| t_final | Tempo final da simulação (inclusive).   |

#### Returns

true se a execução foi concluída com sucesso; false caso contrário.

Implemented in [Model\\_Impl](#).

Here is the caller graph for this function:



#### 7.8.3.11 setSource()

```
virtual void Model::setSource (
    Flow & ,
    System & ) [pure virtual]
```

Define o sistema de origem de um fluxo.

#### Parameters

|   |                                              |
|---|----------------------------------------------|
| f | Referência para o fluxo a ser configurado.   |
| s | Referência para o sistema que será a origem. |

Implemented in [Model\\_Impl](#).

#### 7.8.3.12 setTarget()

```
virtual void Model::setTarget (
    Flow & ,
    System & ) [pure virtual]
```

Define o sistema de destino de um fluxo.

#### Parameters

|   |                                            |
|---|--------------------------------------------|
| f | Referência para o fluxo a ser configurado. |
|---|--------------------------------------------|

|   |                                               |
|---|-----------------------------------------------|
| s | Referência para o sistema que será o destino. |
|---|-----------------------------------------------|

Implemented in [Model\\_Impl](#).

#### 7.8.3.13 showModel()

virtual void Model::showModel () const [pure virtual]

Exibe no console as informações dos sistemas e fluxos do modelo.

Implemented in [Model\\_Impl](#).

The documentation for this class was generated from the following files:

- include/[model.h](#)
- src/[model\\_impl.cpp](#)

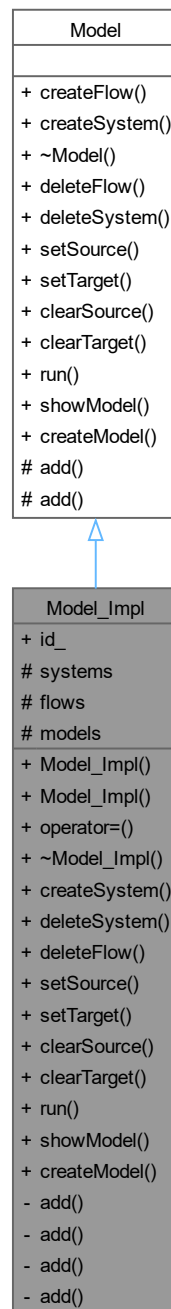
## 7.9 Model\_Impl Class Reference

Classe responsável por representar um modelo de simulação.

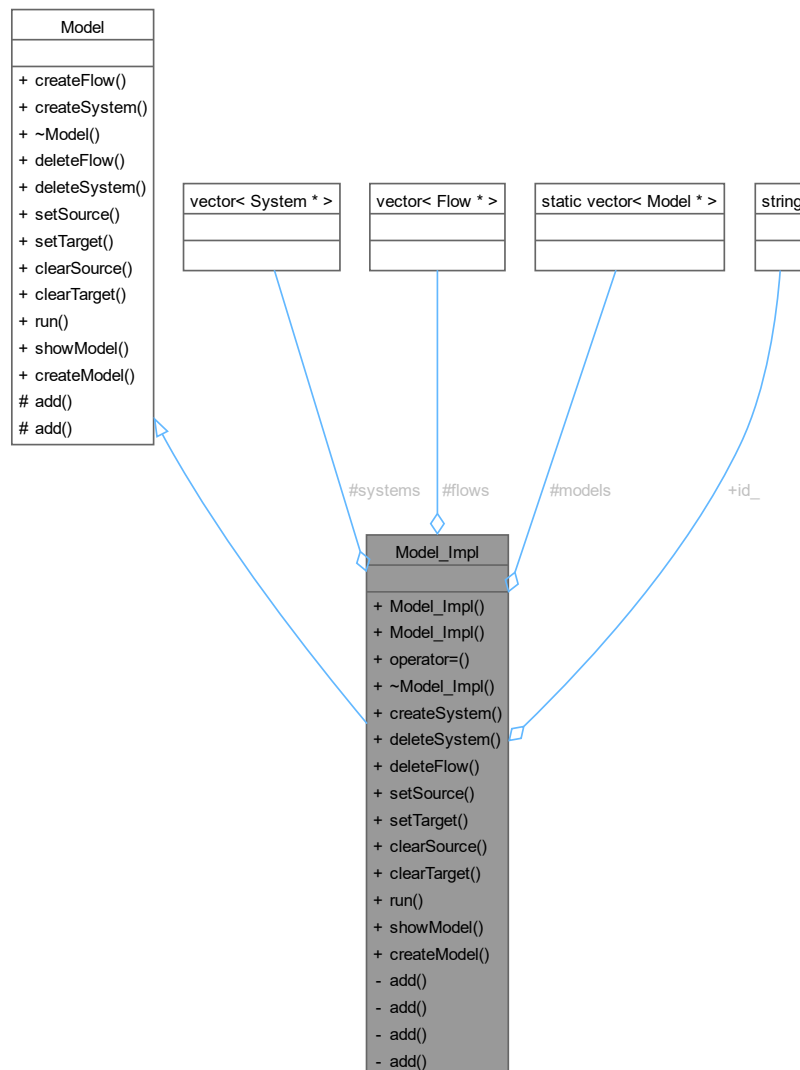
#include <model\_impl.h>



Inheritance diagram for Model\_Impl:



Collaboration diagram for Model\_Impl:



## Public Member Functions

- [Model\\_Impl](#) (string)  
Forma canônica.
- [Model\\_Impl](#) (const [Model\\_Impl](#) &other)  
Construtor de cópia privado.
- [Model\\_Impl](#) & [operator=](#) (const [Model\\_Impl](#) &other)  
Operador de atribuição privado.
- virtual [~Model\\_Impl](#) ()  
Destrutor virtual da classe [Model](#).
- [System](#) & [createSystem](#) (string id, double)  
Cria e adiciona um sistema ao modelo.
- bool [deleteSystem](#) ([System](#) &)  
Remove um sistema do modelo.
- bool [deleteFlow](#) ([Flow](#) &)

- Remove um fluxo do modelo.
- void `setSource` (`Flow` &, `System` &)
  - Define o sistema de origem de um fluxo.
- void `setTarget` (`Flow` &, `System` &)
  - Define o sistema de destino de um fluxo.
- void `clearSource` (`Flow` &)
  - Remove o sistema de origem de um fluxo, definindo-o como nulo.
- void `clearTarget` (`Flow` &)
  - Remove o sistema de destino de um fluxo, definindo-o como nulo.
- bool `run` (int t\_init, int t\_final)
  - Executa a simulação do modelo.
- void `showModel` () const
  - Exibe informações do modelo.

### Public Member Functions inherited from `Model`

- template<typename T\_FLOW\_IMPL>  
`Flow` & `createFlow` (string id, `System` \*source=NULL, `System` \*target=NULL)
  - Cria e adiciona um fluxo concreto ao modelo.
- virtual `~Model` ()=default
  - Destrutor virtual da classe `Model`.

### Static Public Member Functions

- static `Model` & `createModel` (string id)
  - Cria e registra uma nova instância de `Model_Impl`.

### Static Public Member Functions inherited from `Model`

- static `Model` & `createModel` (string id=0)
  - Cria e registra uma nova instância de `Model`.

### Data Fields

- string `id_`
  - Identificador único do modelo.

### Protected Attributes

- vector< `System` \* > `systems`
  - Vetor contendo os sistemas do modelo.
- vector< `Flow` \* > `flows`
  - Vetor contendo os fluxos do modelo.

### Static Protected Attributes

- static vector< `Model` \* > `models`
  - Vetor estático contendo todos os modelos criados.

## Private Member Functions

- void `add (Model *)`  
Registra um modelo no vetor estático interno.
- void `add (Flow *)`  
Adiciona um fluxo ao final do vetor de fluxos.
- void `add (System *)`  
Adiciona um sistema ao vetor de sistemas.
- void `add (Flow *, int i)`  
Insere um fluxo em posição específica do vetor de fluxos.

### 7.9.1 Detailed Description

Classe responsável por representar um modelo de simulação.

A classe `Model_Impl` armazena os sistemas e fluxos que compõem um modelo de dinâmica de sistemas, sendo responsável pela execução da simulação.

### 7.9.2 Constructor & Destructor Documentation

#### 7.9.2.1 `Model_Impl()` [1/2]

`Model_Impl::Model_Impl (`  
    string id)

Forma canônica.

Construtor parametrizado da classe `Model_Impl`.

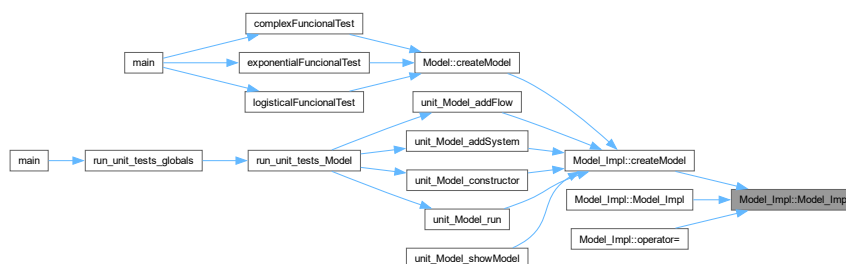
Construtor padrão da classe `Model`.

Inicializa o modelo com o identificador fornecido e vetores de sistemas e fluxos vazios.

#### Parameters

|    |                                |
|----|--------------------------------|
| id | Identificador único do modelo. |
|----|--------------------------------|

Here is the caller graph for this function:



#### 7.9.2.2 `Model_Impl()` [2/2]

`Model_Impl::Model_Impl (`  
    const `Model_Impl` & other)

Construtor de cópia privado.

Construtor de cópia da classe `Model_Impl`.

Impede cópia da classe `Model`.

## Parameters

|       |                                      |
|-------|--------------------------------------|
| other | Outro objeto <a href="#">Model</a> . |
|-------|--------------------------------------|

Copia os vetores de sistemas e fluxos de outro modelo.

## Parameters

|       |                                                  |
|-------|--------------------------------------------------|
| other | Objeto <a href="#">Model_Impl</a> a ser copiado. |
|-------|--------------------------------------------------|

Here is the call graph for this function:



## 7.9.2.3 ~Model\_Impl()

`Model_Impl::~~Model_Impl (`  
     `void )` [virtual]

Destrutor virtual da classe [Model](#).

Destrutor da classe [Model\\_Impl](#).

Limpa os vetores de sistemas e fluxos. Os objetos apontados pelos ponteiros não são destruídos (responsabilidade do código cliente).

## 7.9.3 Member Function Documentation

## 7.9.3.1 add() [1/4]

`void Model_Impl::add (`  
     `Flow * f)` [private], [virtual]

Adiciona um fluxo ao final do vetor de fluxos.

Adiciona um fluxo ao modelo.

## Parameters

|   |                                         |
|---|-----------------------------------------|
| f | Ponteiro para o fluxo a ser adicionado. |
|---|-----------------------------------------|

Implements [Model](#).

## 7.9.3.2 add() [2/4]

`void Model_Impl::add (`  
     `Flow * f,`  
     `int i)` [private]

Insere um fluxo em posição específica do vetor de fluxos.

## Parameters

|   |                                       |
|---|---------------------------------------|
| f | Ponteiro para o fluxo a ser inserido. |
| i | Posição de inserção (1-indexada).     |

## 7.9.3.3 add() [3/4]

```
void Model_Impl::add (
    Model * m) [private], [virtual]
```

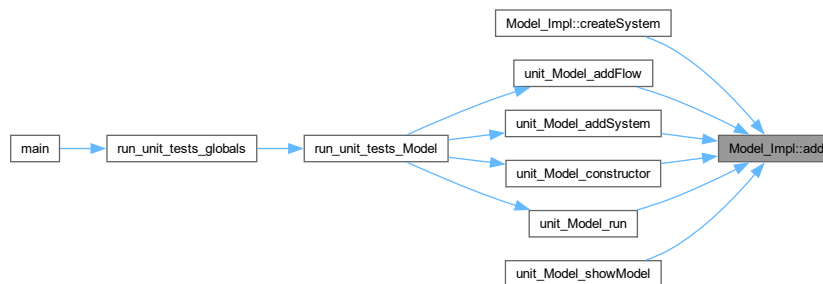
Registra um modelo no vetor estático interno.

Parameters

|   |                                          |
|---|------------------------------------------|
| m | Ponteiro para o modelo a ser registrado. |
|---|------------------------------------------|

Implements [Model](#).

Here is the caller graph for this function:



## 7.9.3.4 add() [4/4]

```
void Model_Impl::add (
    System * s) [private]
```

Adiciona um sistema ao vetor de sistemas.

Adiciona um sistema ao modelo.

Parameters

|   |                                           |
|---|-------------------------------------------|
| s | Ponteiro para o sistema a ser adicionado. |
|---|-------------------------------------------|

## 7.9.3.5 clearSource()

```
void Model_Impl::clearSource (
    Flow & f) [virtual]
```

Remove o sistema de origem de um fluxo, definindo-o como nulo.

Parameters

|   |                                                 |
|---|-------------------------------------------------|
| f | Referência para o fluxo cujo source será limpo. |
|---|-------------------------------------------------|

Implements [Model](#).

Here is the call graph for this function:



#### 7.9.3.6 clearTarget()

```
void Model_Impl::clearTarget (
    Flow & f) [virtual]
```

Remove o sistema de destino de um fluxo, definindo-o como nulo.

Parameters

|   |                                                 |
|---|-------------------------------------------------|
| f | Referência para o fluxo cujo target será limpo. |
|---|-------------------------------------------------|

Implements [Model](#).

Here is the call graph for this function:



#### 7.9.3.7 createModel()

```
Model & Model_Impl::createModel (
    string id) [static]
```

Cria e registra uma nova instância de [Model\\_Impl](#).

Método estático de fábrica que instancia um [Model\\_Impl](#) com o identificador fornecido, registra-o no vetor estático e o retorna.

Parameters

|    |                                |
|----|--------------------------------|
| id | Identificador único do modelo. |
|----|--------------------------------|

Returns

Referência para o modelo recém-criado.

Instancia um [Model\\_Impl](#) com o identificador fornecido, registra-o no vetor estático de modelos e retorna a referência.

## Parameters

|    |                                |
|----|--------------------------------|
| id | Identificador único do modelo. |
|----|--------------------------------|

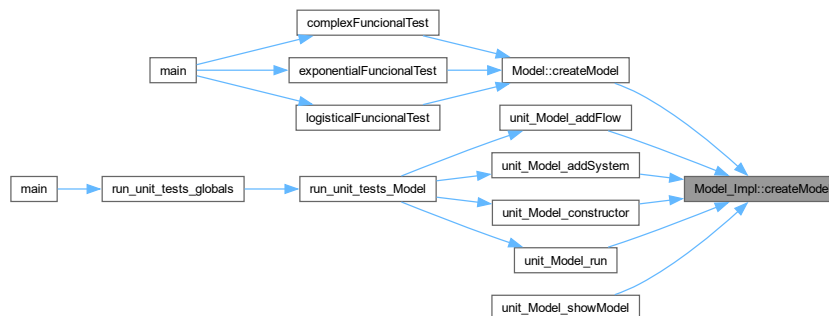
## Returns

Referência para o modelo recém-criado.

Here is the call graph for this function:



Here is the caller graph for this function:



## 7.9.3.8 createSystem()

`System & Model_Impl::createSystem (`  
     string id,  
     double value) [virtual]

Cria e adiciona um sistema ao modelo.

Instancia um `System_Impl` com o identificador e valor inicial fornecidos, adiciona-o ao vetor interno e retorna a referência.

## Parameters

|       |                           |
|-------|---------------------------|
| id    | Identificador do sistema. |
| value | Valor inicial do sistema. |

## Returns

Referência para o sistema recém-criado.

Instancia um `System_Impl` com o identificador e valor inicial fornecidos, adiciona-o ao vetor interno de sistemas e retorna a referência.



## Parameters

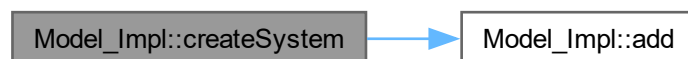
|       |                           |
|-------|---------------------------|
| id    | Identificador do sistema. |
| value | Valor inicial do sistema. |

## Returns

Referência para o sistema recém-criado.

Implements [Model](#).

Here is the call graph for this function:



## 7.9.3.9 deleteFlow()

```
bool Model_Impl::deleteFlow (  
    Flow & ) [virtual]
```

Remove um fluxo do modelo.

## Parameters

|   |                                         |
|---|-----------------------------------------|
| f | Referência para o fluxo a ser removido. |
|---|-----------------------------------------|

## Returns

true se o fluxo foi removido com sucesso; false caso contrário.

Implementação atual retorna sempre true (stub).

## Parameters

|   |                                         |
|---|-----------------------------------------|
| f | Referência para o fluxo a ser removido. |
|---|-----------------------------------------|

## Returns

true indicando sucesso.

Implements [Model](#).

## 7.9.3.10 deleteSystem()

```
bool Model_Impl::deleteSystem (  
    System & ) [virtual]
```

Remove um sistema do modelo.

## Parameters

|   |                                           |
|---|-------------------------------------------|
| s | Referência para o sistema a ser removido. |
|---|-------------------------------------------|

## Returns

true se o sistema foi removido com sucesso; false caso contrário.

Implementação atual retorna sempre true (stub).

## Parameters

|   |                                           |
|---|-------------------------------------------|
| s | Referência para o sistema a ser removido. |
|---|-------------------------------------------|

## Returns

true indicando sucesso.

Implements [Model](#).

## 7.9.3.11 operator=()

[Model\\_Impl](#) & [Model\\_Impl::operator=](#) (  
const [Model\\_Impl](#) & other)

Operador de atribuição privado.

Operador de atribuição da classe [Model\\_Impl](#).

Impede atribuição entre objetos [Model](#).

## Parameters

|       |                                      |
|-------|--------------------------------------|
| other | Outro objeto <a href="#">Model</a> . |
|-------|--------------------------------------|

## Returns

Referência para o próprio objeto.

Copia os vetores de sistemas e fluxos de outro modelo.

## Parameters

|       |                                                    |
|-------|----------------------------------------------------|
| other | Objeto <a href="#">Model_Impl</a> a ser atribuído. |
|-------|----------------------------------------------------|

## Returns

Referência para o objeto atual.

Here is the call graph for this function:



## 7.9.3.12 run()

```
bool Model_Impl::run (
    int t_init,
    int t_final) [virtual]
```

Executa a simulação do modelo.

Executa a simulação do modelo no intervalo de tempo especificado.

## Parameters

|         |                             |
|---------|-----------------------------|
| t_init  | Tempo inicial da simulação. |
| t_final | Tempo final da simulação.   |

## Returns

true caso a execução seja concluída com sucesso.

false caso ocorra falha na execução.

A cada passo de tempo, todos os fluxos são calculados com base nos valores atuais dos sistemas (passo síncrono), e em seguida os sistemas de origem e destino de cada fluxo são atualizados simultaneamente.

## Parameters

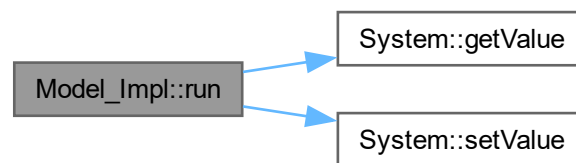
|         |                                         |
|---------|-----------------------------------------|
| t_init  | Tempo inicial da simulação (inclusive). |
| t_final | Tempo final da simulação (exclusive).   |

## Returns

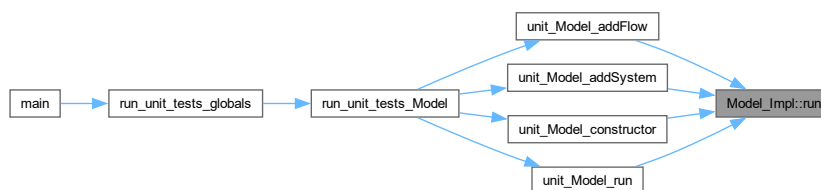
true se a execução foi concluída com sucesso.

Implements [Model](#).

Here is the call graph for this function:



Here is the caller graph for this function:



## 7.9.3.13 setSource()

```
void Model_Impl::setSource (
    Flow & f,
    System & s) [virtual]
```

Define o sistema de origem de um fluxo.

Realiza cast para [Flow\\_Impl](#) e invoca [setSource\(\)](#).

## Parameters

|   |                                              |
|---|----------------------------------------------|
| f | Referência para o fluxo a ser configurado.   |
| s | Referência para o sistema que será a origem. |

Realiza cast do fluxo para [Flow\\_Impl](#) e invoca [setSource\(\)](#).

## Parameters

|   |                                              |
|---|----------------------------------------------|
| f | Referência para o fluxo a ser configurado.   |
| s | Referência para o sistema que será a origem. |

Implements [Model](#).

Here is the call graph for this function:



## 7.9.3.14 setTarget()

```
void Model_Impl::setTarget (
    Flow & f,
    System & s) [virtual]
```

Define o sistema de destino de um fluxo.

Realiza cast para [Flow\\_Impl](#) e invoca [setTarget\(\)](#).

## Parameters

|   |                                               |
|---|-----------------------------------------------|
| f | Referência para o fluxo a ser configurado.    |
| s | Referência para o sistema que será o destino. |

Realiza cast do fluxo para [Flow\\_Impl](#) e invoca [setTarget\(\)](#).

## Parameters

|   |                                               |
|---|-----------------------------------------------|
| f | Referência para o fluxo a ser configurado.    |
| s | Referência para o sistema que será o destino. |

Implements [Model](#).

Here is the call graph for this function:



#### 7.9.3.15 showModel()

`void Model_Impl::showModel () const [virtual]`

Exibe informações do modelo.

Exibe no console os sistemas e fluxos do modelo.

Para cada sistema, imprime seu nome e valor atual. Para cada fluxo, imprime a conexão entre sistema de origem e sistema de destino no formato "origem -> destino".

Implements [Model](#).

Here is the caller graph for this function:



### 7.9.4 Field Documentation

#### 7.9.4.1 flows

`vector<Flow*> Model_Impl::flows [protected]`

Vetor contendo os fluxos do modelo.

#### 7.9.4.2 id\_\_

`string Model_Impl::id__`

Identificador único do modelo.

#### 7.9.4.3 models

`vector< Model * > Model_Impl::models [static], [protected]`

Vetor estático contendo todos os modelos criados.

Inicialização do vetor estático de modelos.

Armazena ponteiros para todas as instâncias de [Model\\_Impl](#) criadas via [createModel\(\)](#), permitindo gerenciamento global dos modelos.

#### 7.9.4.4 systems

`vector<System*> Model_Impl::systems [protected]`

Vetor contendo os sistemas do modelo.

The documentation for this class was generated from the following files:

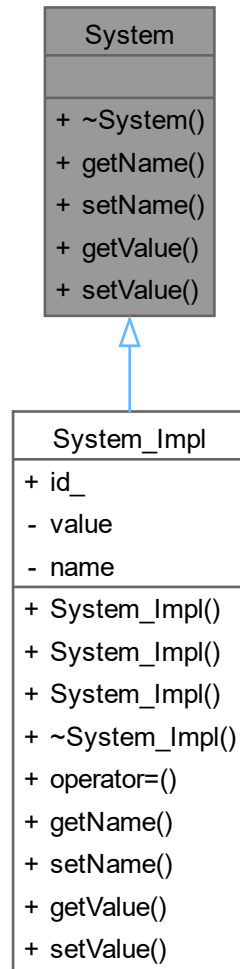
- [src/model\\_impl.h](#)
- [src/model\\_impl.cpp](#)

## 7.10 System Class Reference

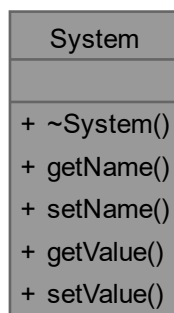
Interface abstrata que representa um sistema em um modelo de simulação.

#include <system.h>

Inheritance diagram for System:



Collaboration diagram for System:



### Public Member Functions

- virtual [~System](#) ()=default  
Destrutor virtual da classe [System](#).
- virtual string [getName](#) () const =0  
Retorna o nome do sistema.
- virtual void [setName](#) (const string &name)=0  
Define o nome do sistema.
- virtual double [getValue](#) () const =0  
Retorna o valor atual armazenado no sistema.
- virtual void [setValue](#) (double value)=0  
Define o valor armazenado no sistema.

### 7.10.1 Detailed Description

Interface abstrata que representa um sistema em um modelo de simulação.

A classe [System](#) define a interface para os compartimentos (estoques) de um modelo de dinâmica de sistemas. Um sistema armazena um valor numérico que é modificado ao longo do tempo pelos fluxos conectados a ele. Subclasses devem fornecer implementação concreta para todos os métodos virtuais puros.

### 7.10.2 Constructor & Destructor Documentation

#### 7.10.2.1 ~System()

virtual System::~~System () [virtual], [default]

Destrutor virtual da classe [System](#).

### 7.10.3 Member Function Documentation

#### 7.10.3.1 getName()

virtual string System::getName () const [pure virtual]

Retorna o nome do sistema.

Returns

Nome do sistema.

Implemented in [System\\_Impl](#).

### 7.10.3.2 getValue()

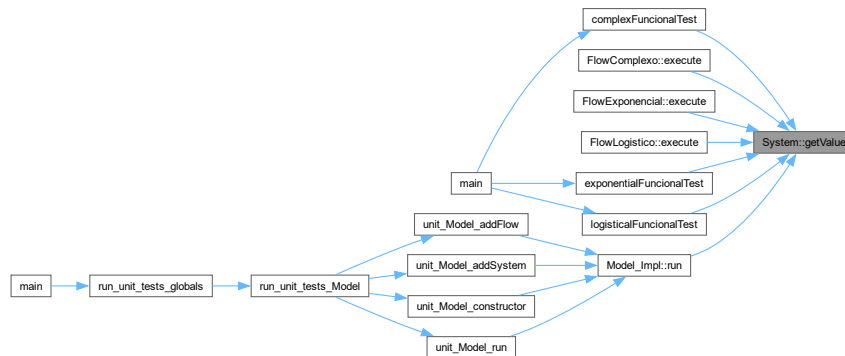
virtual double System::getValue () const [pure virtual]  
Retorna o valor atual armazenado no sistema.

Returns

Valor do sistema.

Implemented in [System\\_Impl](#).

Here is the caller graph for this function:



### 7.10.3.3 setName()

virtual void System::setName (  
                    const string & name) [pure virtual]  
Define o nome do sistema.

Parameters

|      |                       |
|------|-----------------------|
| name | Novo nome do sistema. |
|------|-----------------------|

Implemented in [System\\_Impl](#).

### 7.10.3.4 setValue()

virtual void System::setValue (  
                    double value) [pure virtual]  
Define o valor armazenado no sistema.

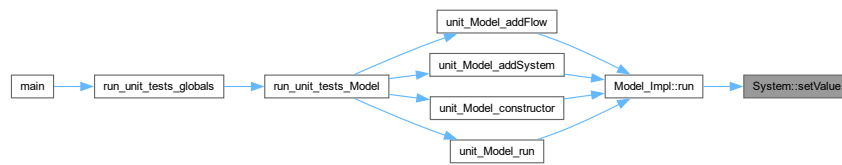
Parameters

|       |                        |
|-------|------------------------|
| value | Novo valor do sistema. |
|-------|------------------------|

Implemented in [System\\_Impl](#).



Here is the caller graph for this function:



The documentation for this class was generated from the following file:

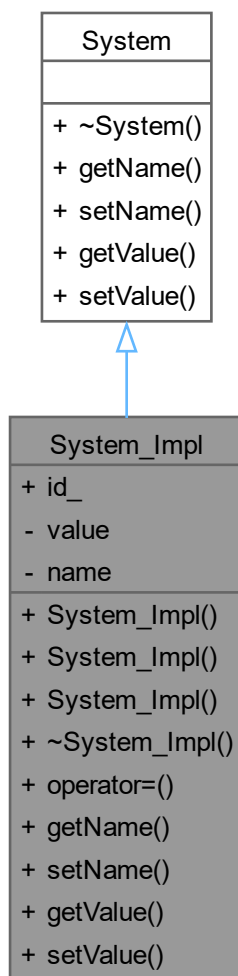
- `include/system.h`

## 7.11 System\_Impl Class Reference

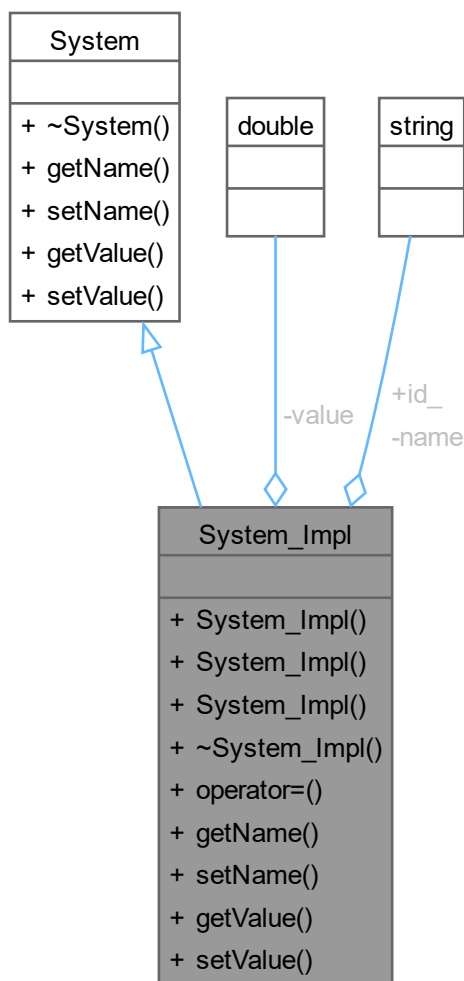
Classe responsável por representar um sistema.

`#include <system_impl.h>`

Inheritance diagram for System\_Impl:



Collaboration diagram for System\_Impl:



#### Public Member Functions

- [System\\_Impl](#) ()  
Forma canônica.
- [System\\_Impl](#) (const string &name, double value)  
Construtor da classe [System](#).
- [System\\_Impl](#) (const [System\\_Impl](#) &other)  
Construtor de cópia privado.
- virtual [~System\\_Impl](#) ()  
Destrutor virtual da classe [System](#).
- [System\\_Impl](#) & [operator=](#) (const [System\\_Impl](#) &other)  
Operador de atribuição privado.
- string [getName](#) () const  
Métodos da UML.
- void [setName](#) (const string &name)

- Define o nome do sistema.
- double `getValue` () const  
Retorna o valor atual do sistema.
- void `setValue` (double value)  
Define o valor do sistema.

## Public Member Functions inherited from `System`

- virtual `~System` ()=default  
Destrutor virtual da classe `System`.

## Data Fields

- string `id_`

## Private Attributes

- double `value`  
Valor armazenado pelo sistema.
- string `name`  
Nome do sistema.

### 7.11.1 Detailed Description

Classe responsável por representar um sistema.

Um sistema armazena um valor que pode ser alterado durante a execução da simulação.

### 7.11.2 Constructor & Destructor Documentation

#### 7.11.2.1 `System_Impl()` [1/3]

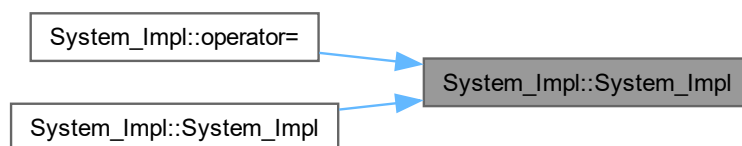
`System_Impl::System_Impl ()`

Forma canônica.

Construtor padrão da classe `System_Impl`.

Construtor padrão da classe `System`.

Inicializa o sistema com valor igual a 0.0 e nome vazio. Here is the caller graph for this function:



#### 7.11.2.2 `System_Impl()` [2/3]

`System_Impl::System_Impl (`  
     const string & name,  
     double value)

Construtor da classe `System`.

Construtor parametrizado da classe `System_Impl`.

## Parameters

|       |                           |
|-------|---------------------------|
| name  | Nome do sistema.          |
| value | Valor inicial do sistema. |

## 7.11.2.3 System\_Impl() [3/3]

System\_Impl::System\_Impl (  
     const [System\\_Impl](#) & other)

Construtor de cópia privado.

Construtor de cópia da classe [System\\_Impl](#).

Impede cópia da classe [System](#).

## Parameters

|       |                                                   |
|-------|---------------------------------------------------|
| other | Outro objeto <a href="#">System</a> .             |
| other | Objeto <a href="#">System_Impl</a> a ser copiado. |

Here is the call graph for this function:



## 7.11.2.4 ~System\_Impl()

System\_Impl::~System\_Impl () [virtual]

Destrutor virtual da classe [System](#).

Destrutor da classe [System\\_Impl](#).

## 7.11.3 Member Function Documentation

## 7.11.3.1 getName()

string System\_Impl::getName () const [virtual]

Métodos da UML.

Retorna o nome do sistema.

Retorna o nome do sistema.

## Returns

Nome do sistema.

Nome do sistema.

Implements [System](#).

## 7.11.3.2 getValue()

double System\_Impl::getValue () const [virtual]

Retorna o valor atual do sistema.

Retorna o valor atual armazenado no sistema.

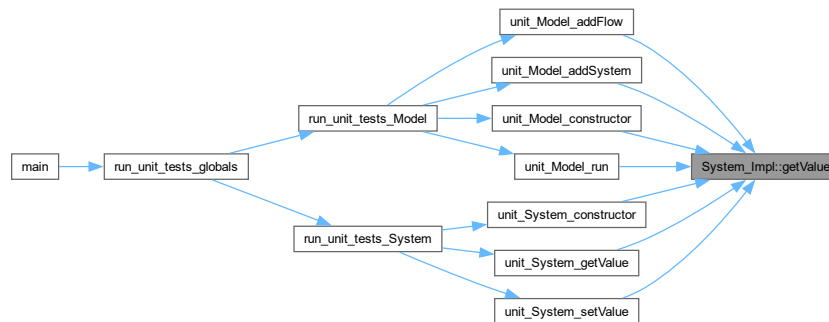
## Returns

Valor armazenado no sistema.

Valor do sistema.

Implements [System](#).

Here is the caller graph for this function:



## 7.11.3.3 operator=()

[System\\_Impl](#) & [System\\_Impl::operator=](#) (  
const [System\\_Impl](#) & other)

Operador de atribuição privado.

Operador de atribuição da classe [System\\_Impl](#).

Impede atribuição entre objetos [System](#).

## Parameters

|       |                                       |
|-------|---------------------------------------|
| other | Outro objeto <a href="#">System</a> . |
|-------|---------------------------------------|

## Returns

Referência para o próprio objeto.

Copia nome e valor de outro objeto [System\\_Impl](#).

## Parameters

|       |                                                     |
|-------|-----------------------------------------------------|
| other | Objeto <a href="#">System_Impl</a> a ser atribuído. |
|-------|-----------------------------------------------------|

## Returns

Referência para o objeto atual.

Here is the call graph for this function:



#### 7.11.3.4 setName()

```
void System_Impl::setName (  
    const string & name) [virtual]
```

Define o nome do sistema.

##### Parameters

|      |                       |
|------|-----------------------|
| name | Novo nome do sistema. |
|------|-----------------------|

Implements [System](#).

#### 7.11.3.5 setValue()

```
void System_Impl::setValue (  
    double value) [virtual]
```

Define o valor do sistema.

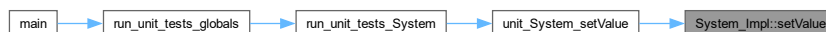
Define o valor armazenado no sistema.

##### Parameters

|       |                        |
|-------|------------------------|
| value | Novo valor do sistema. |
|-------|------------------------|

Implements [System](#).

Here is the caller graph for this function:



### 7.11.4 Field Documentation

#### 7.11.4.1 id\_

string System\_Impl::id\_

#### 7.11.4.2 name

string System\_Impl::name [private]

Nome do sistema.

#### 7.11.4.3 value

double System\_Impl::value [private]

Valor armazenado pelo sistema.

The documentation for this class was generated from the following files:

- [src/system\\_impl.h](#)
- [src/system\\_impl.cpp](#)





## Chapter 8

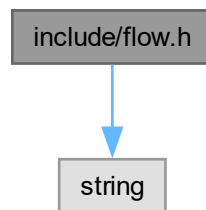
# File Documentation

### 8.1 include/flow.h File Reference

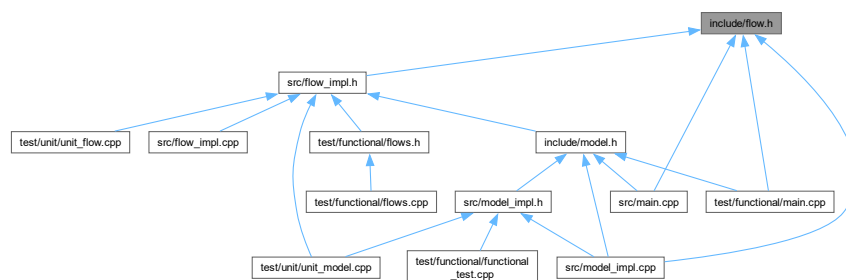
Declaração da interface abstrata [Flow](#).

```
#include <string>
```

Include dependency graph for flow.h:



This graph shows which files directly or indirectly include this file:



#### Data Structures

- class [Flow](#)  
Interface abstrata que representa um fluxo entre dois sistemas.

#### 8.1.1 Detailed Description

Declaração da interface abstrata [Flow](#).

## 8.2 flow.h

[Go to the documentation of this file.](#)

```

00001 #ifndef FLOW_H
00002 #define FLOW_H
00003
00004 #include <string>
00005
00006 using namespace std;
00007
00008 class System;
00009
00014
00023 class Flow {
00024 public:
00025
00029     virtual ~Flow() = default;
00030
00036     virtual string getName() const = 0;
00037
00043     virtual void setName(const string& name) = 0;
00044
00050     virtual System* getSource() const = 0;
00051
00057     virtual void setSource(System* source) = 0;
00058
00064     virtual System* getTarget() const = 0;
00065
00071     virtual void setTarget(System* target) = 0;
00072
00074
00083     virtual double execute() = 0;
00084 };
00085
00086 #endif

```

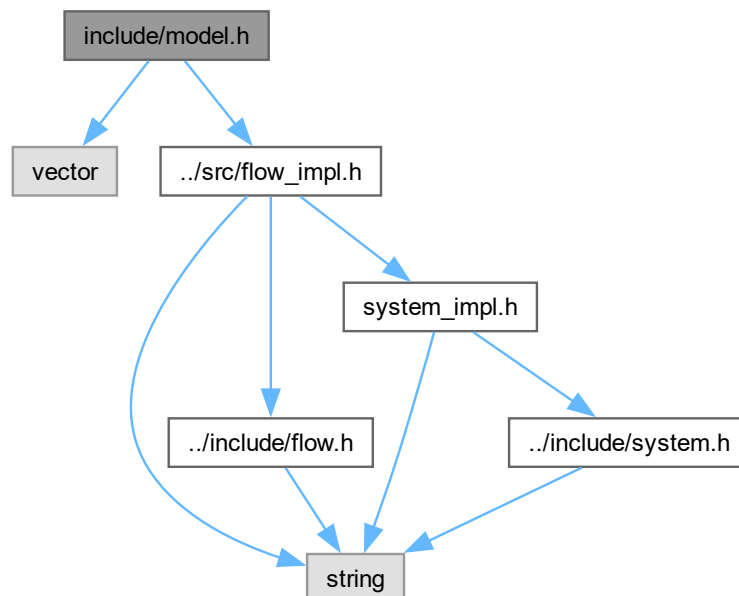
## 8.3 include/model.h File Reference

Declaração da interface abstrata [Model](#).

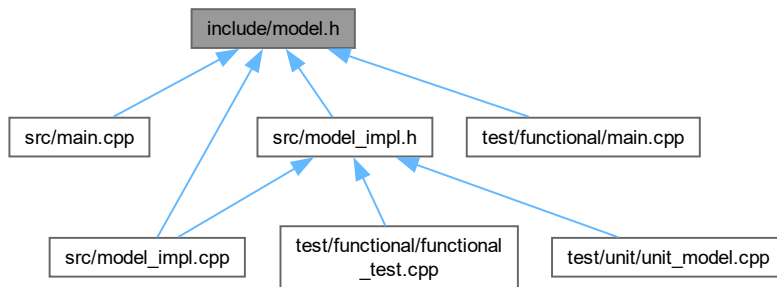
```
#include <vector>
```

```
#include "../src/flow_impl.h"
```

Include dependency graph for model.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- class [Model](#)

Interface abstrata que representa um modelo de dinâmica de sistemas.

### 8.3.1 Detailed Description

Declaração da interface abstrata [Model](#).

## 8.4 model.h

[Go to the documentation of this file.](#)

```

00001 #ifndef MODEL_H
00002 #define MODEL_H
00003
00004 #include <vector>
00005 #include "../src/flow_impl.h"
00006
00007 using namespace std;
00008
00013
00014 // Forward declarations
00015 class System;
00016 class Flow;
00017
00027 class Model {
00028 public:
00029
00039     static Model& createModel(string id = 0);
00040
00054     template <typename T_FLOW_IMPL>
00055     Flow& createFlow(string id, System* source = NULL, System* target = NULL) {
00056         Flow* flow = new T_FLOW_IMPL(id, source, target);
00057         add(flow);
00058         return *flow;
00059     }
00060
00071     virtual System& createSystem(string id, double) = 0;
00072
00076     virtual ~Model() = default;
00077
00084     virtual bool deleteFlow(Flow&) = 0;
00085
00092     virtual bool deleteSystem(System&) = 0;
00093
00100     virtual void setSource(Flow&, System&) = 0;
00101
00108     virtual void setTarget(Flow&, System&) = 0;
00109
00115     virtual void clearSource(Flow&) = 0;
00116
00122     virtual void clearTarget(Flow&) = 0;
00123
00134     virtual bool run(int t_init, int t_final) = 0;

```

```

00135
00139     virtual void showModel() const = 0;
00140 protected:
00146     virtual void add(Model*) = 0;
00147
00153     virtual void add(Flow*) = 0;
00154 };
00155
00156 #endif

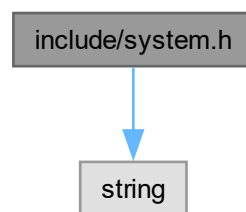
```

## 8.5 include/system.h File Reference

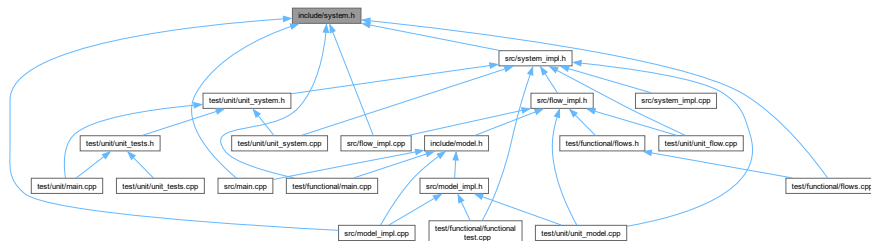
Declaração da interface abstrata [System](#).

#include <string>

Include dependency graph for system.h:



This graph shows which files directly or indirectly include this file:



### Data Structures

- class [System](#)

Interface abstrata que representa um sistema em um modelo de simulação.

### 8.5.1 Detailed Description

Declaração da interface abstrata [System](#).

## 8.6 system.h

[Go to the documentation of this file.](#)

```

00001 #ifndef SYSTEM_H
00002 #define SYSTEM_H
00003
00004 #include <string>
00005

```

```

00006 using namespace std;
00007
00012
00021 class System {
00022 public:
00023
00027     virtual ~System() = default;
00028
00034     virtual string getName() const = 0;
00035
00041     virtual void setName(const string& name) = 0;
00042
00048     virtual double getValue() const = 0;
00049
00055     virtual void setValue(double value) = 0;
00056 };
00057
00058 #endif

```

## 8.7 README.md File Reference

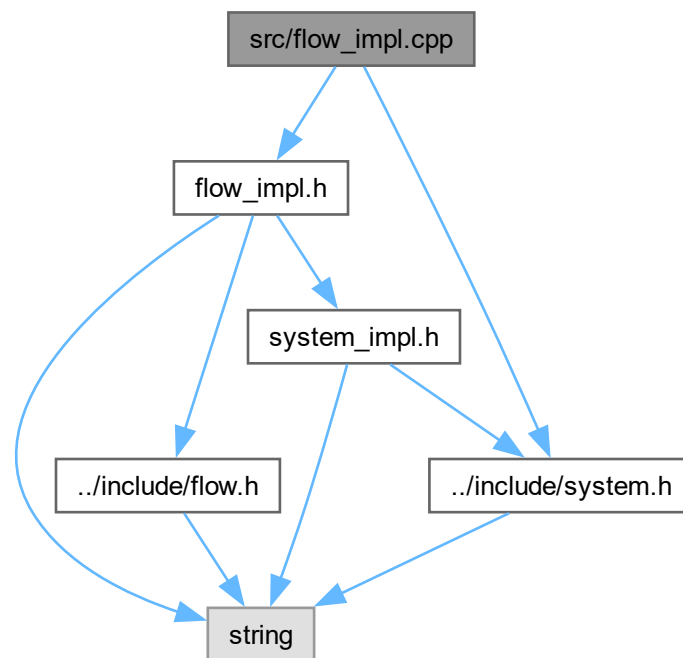
## 8.8 src/flow\_impl.cpp File Reference

Implementação da classe [Flow\\_Impl](#).

```
#include "flow_impl.h"
```

```
#include "../include/system.h"
```

Include dependency graph for flow\_impl.cpp:



### 8.8.1 Detailed Description

Implementação da classe [Flow\\_Impl](#).

Este arquivo contém os métodos responsáveis pela manipulação dos fluxos do simulador, incluindo construtores, destrutor, operador de atribuição e getters/setters.

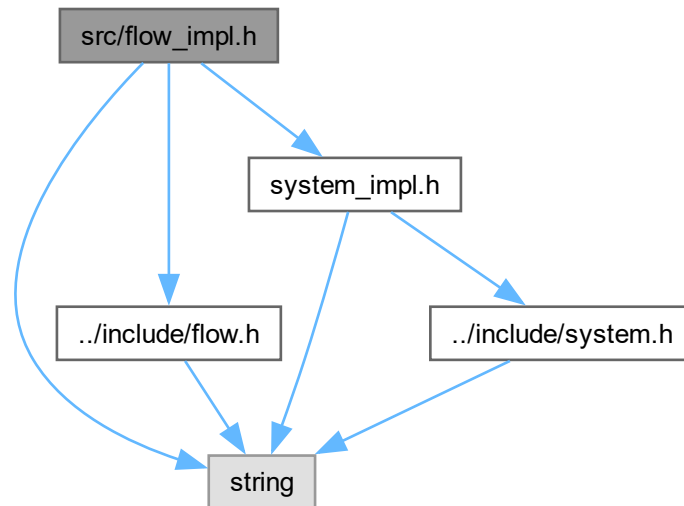
## 8.9 src/flow\_impl.h File Reference

```
#include <string>
```

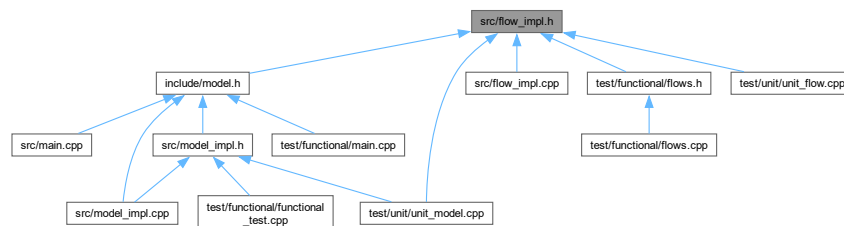
```
#include "../include/flow.h"
```

```
#include "system_impl.h"
```

Include dependency graph for flow\_impl.h:



This graph shows which files directly or indirectly include this file:



### Data Structures

- class [Flow\\_Impl](#)

## 8.10 flow\_impl.h

[Go to the documentation of this file.](#)

```

00001 #ifndef FLOW_IMPL_H
00002 #define FLOW_IMPL_H
00003
00004 #include <string>
00005 #include "../include/flow.h"
00006 #include "system_impl.h"
00007 using namespace std;
00008
00009 class System;

```

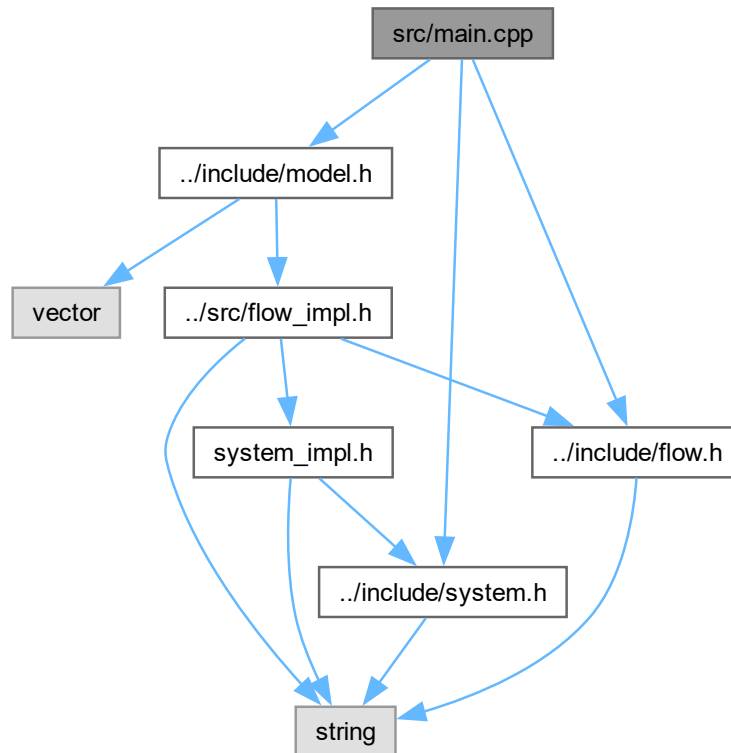
```
00010
00011 class Flow_Impl : public Flow {
00012 protected:
00013     string name;
00014     System* source;
00015     System* target;
00016
00017 public:
00018
00020     Flow_Impl();
00021
00029     Flow_Impl(const string& name,
00030               System* source,
00031               System* target);
00032
00038     Flow_Impl(const Flow_Impl& other);
00039
00043     virtual ~Flow_Impl();
00044
00053     Flow_Impl& operator=(const Flow_Impl& other);
00054
00056
00061     string getName() const;
00062
00068     void setName(const string& name);
00069
00075     System* getSource() const;
00076
00082     void setSource(System* source);
00083
00089     System* getTarget() const;
00090
00096     void setTarget(System* target);
00097
00098
00100
00107     virtual double execute() = 0;
00108
00109 };
00110 #endif
```

## 8.11 src/main.cpp File Reference

Ponto de entrada principal do simulador.

```
#include "../include/model.h"
#include "../include/system.h"
#include "../include/flow.h"
```

Include dependency graph for main.cpp:



#### Macros

- `#define` [MAIN](#)

#### Functions

- `int` [main](#) ()  
Função principal do simulador.

### 8.11.1 Detailed Description

Ponto de entrada principal do simulador.

Este arquivo contém a função principal responsável pela inicialização da aplicação do simulador de sistemas dinâmicos.

### 8.11.2 Macro Definition Documentation

#### 8.11.2.1 MAIN

```
#define MAIN
```

### 8.11.3 Function Documentation

#### 8.11.3.1 main()

```
int main ()
```



Função principal do simulador.

Ponto de entrada da aplicação. Responsável por inicializar e coordenar a execução do simulador.

Returns

0 caso o programa execute corretamente.

## 8.12 test/functional/main.cpp File Reference

Ponto de entrada dos testes funcionais do simulador.

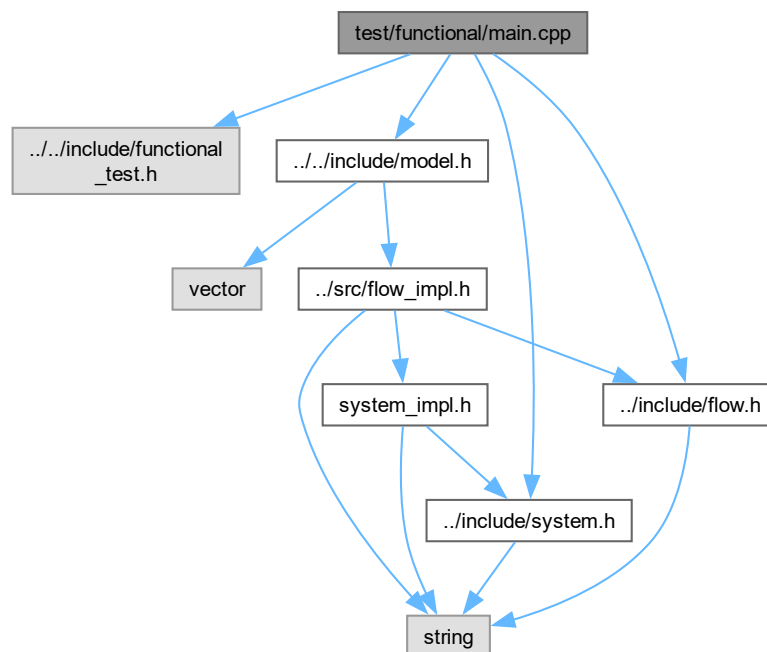
```
#include "../include/functional_test.h"
```

```
#include "../include/model.h"
```

```
#include "../include/system.h"
```

```
#include "../include/flow.h"
```

Include dependency graph for main.cpp:



Macros

- `#define` `MAIN_FUNCIONAL_TESTS`

Functions

- `int` `main ()`

Função principal dos testes funcionais.

### 8.12.1 Detailed Description

Ponto de entrada dos testes funcionais do simulador.

Este arquivo executa em sequência todos os testes funcionais do simulador de sistemas dinâmicos:↔ exponencial, logístico e complexo. Cada teste verifica se os valores finais da simulação correspondem aos resultados analíticos esperados.

## 8.12.2 Macro Definition Documentation

### 8.12.2.1 MAIN\_FUNCIONAL\_TESTS

```
#define MAIN_FUNCIONAL_TESTS
```

## 8.12.3 Function Documentation

### 8.12.3.1 main()

```
int main ()
```

Função principal dos testes funcionais.

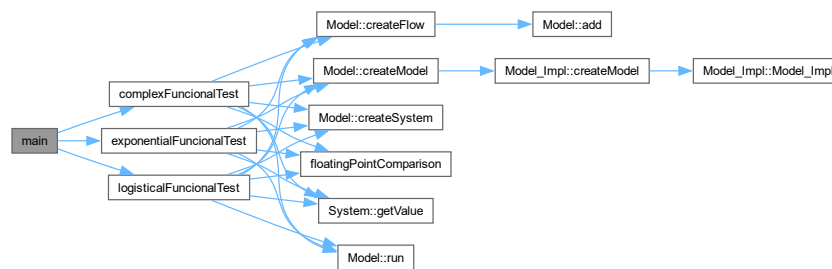
Executa os três testes funcionais do simulador em sequência:

- `exponentialFuncionalTest()`: verifica o modelo de crescimento exponencial.
- `logisticalFuncionalTest()`: verifica o modelo de crescimento logístico.
- `complexFuncionalTest()`: verifica o modelo com múltiplos sistemas e fluxos.

Returns

0 caso todos os testes sejam executados com sucesso.

Here is the call graph for this function:

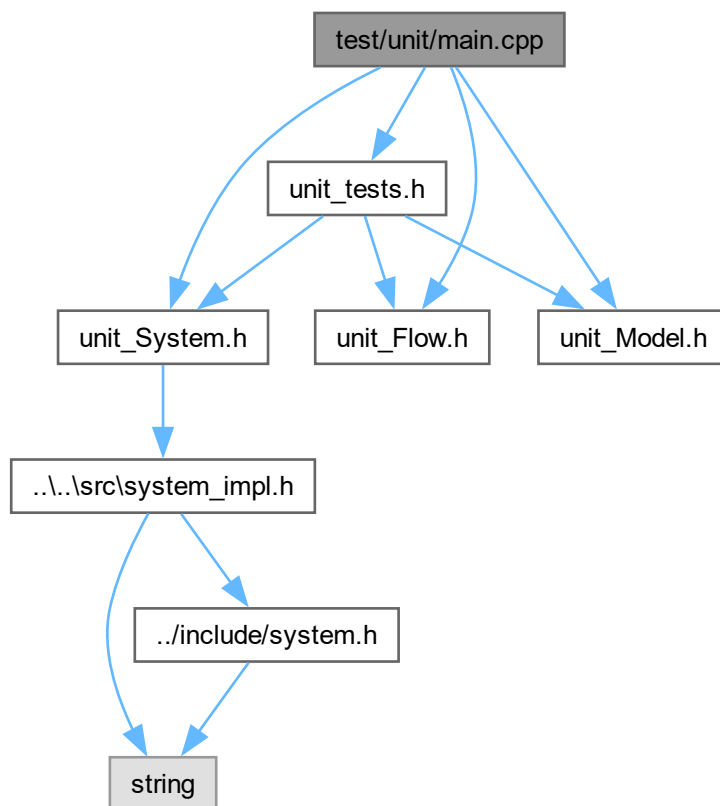


## 8.13 test/unit/main.cpp File Reference

Ponto de entrada da suíte de testes unitários.

```
#include "unit_tests.h"
#include "unit_System.h"
#include "unit_Flow.h"
#include "unit_Model.h"
```

Include dependency graph for main.cpp:



## Functions

- `int main ()`  
Função principal do executável de testes unitários.

### 8.13.1 Detailed Description

Ponto de entrada da suíte de testes unitários.

Arquivo principal responsável por iniciar a execução dos testes unitários do modelo de simulação. Por padrão, executa todos os testes globais via `run_unit_tests_globals()`. As chamadas individuais por módulo (`System`, `Flow`, `Model`) estão disponíveis e podem ser habilitadas removendo os comentários correspondentes.

### 8.13.2 Function Documentation

#### 8.13.2.1 main()

`int main ()`

Função principal do executável de testes unitários.

Invoca a suíte de testes unitários globais. As chamadas individuais por módulo estão comentadas e podem ser ativadas conforme necessário para depuração isolada de cada classe.

Returns

0 em caso de sucesso (todos os asserts aprovados).

Here is the call graph for this function:



## 8.14 src/model\_impl.cpp File Reference

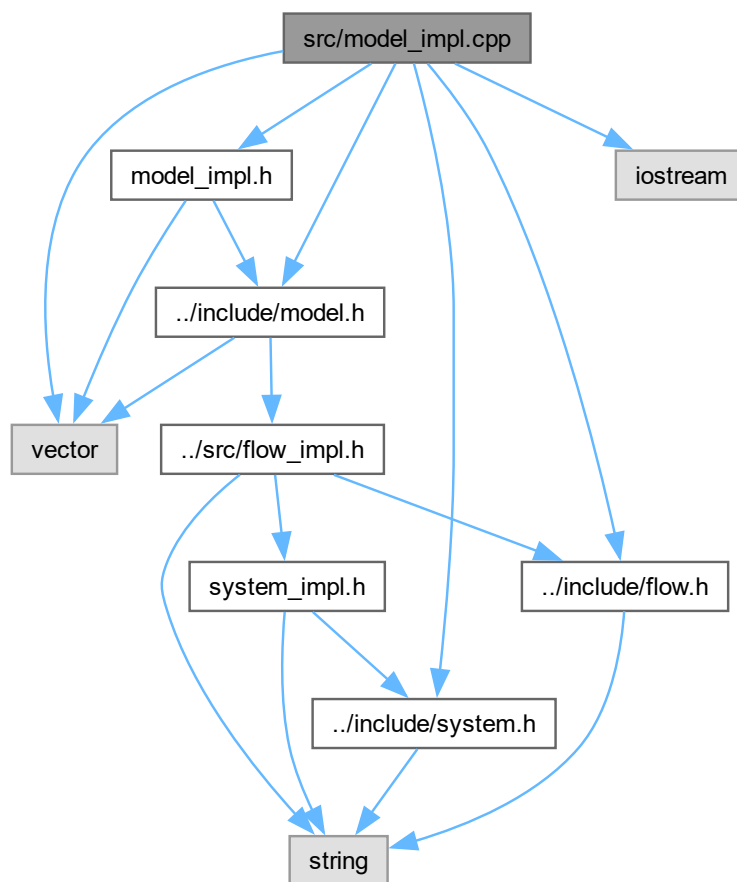
Implementação da classe [Model\\_Impl](#).

```

#include "model_impl.h"
#include "../include/system.h"
#include "../include/model.h"
#include "../include/flow.h"
#include <iostream>
#include <vector>

```

Include dependency graph for model\_impl.cpp:



### 8.14.1 Detailed Description

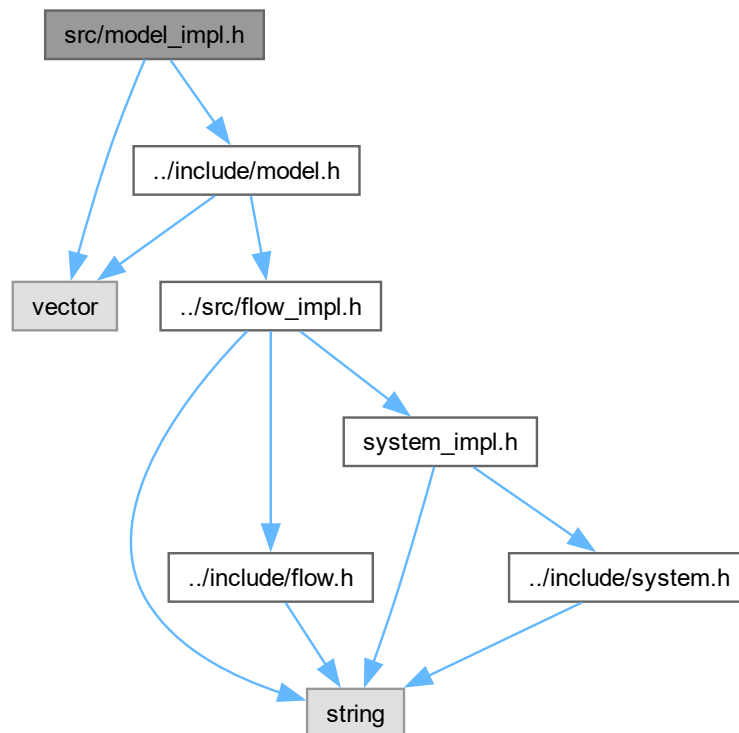
Implementação da classe [Model\\_Impl](#).

Este arquivo contém os métodos responsáveis pela manipulação e execução dos modelos de simulação de sistemas dinâmicos, incluindo construtores, destrutor, operador de atribuição, adição de sistemas e fluxos, execução da simulação e exibição do estado do modelo.

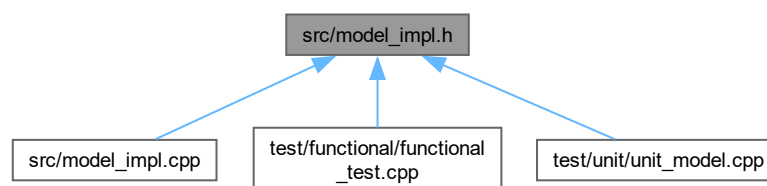
## 8.15 src/model\_impl.h File Reference

```
#include <vector>
#include "../include/model.h"
```

Include dependency graph for model\_impl.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- class [Model\\_Impl](#)  
Classe responsável por representar um modelo de simulação.

## 8.16 model\_impl.h

[Go to the documentation of this file.](#)

```

00001 #ifndef MODEL_IMPL_H
00002 #define MODEL_IMPL_H
00003

```

```

00004 #include <vector>
00005 #include "../include/model.h"
00006 using namespace std;
00007
00008 // Forward declaration
00009 class System;
00010 class Flow;
00011
00019 class Model_Impl : public Model {
00020 protected:
00021
00025     vector<System*> systems;
00026
00030     vector<Flow*> flows;
00031
00038     static vector<Model*> models;
00039
00040 public:
00041
00043     Model_Impl(string);
00047
00055     Model_Impl(const Model_Impl& other);
00056
00065     Model_Impl& operator=(const Model_Impl& other);
00066
00070     virtual ~Model_Impl();
00071
00081     static Model& createModel(string id);
00082
00093     System& createSystem(string id, double);
00094
00101     bool deleteSystem(System&);
00102
00109     bool deleteFlow(Flow&);
00110
00119     void setSource(Flow&, System&);
00120
00129     void setTarget(Flow&, System&);
00130
00136     void clearSource(Flow&);
00137
00143     void clearTarget(Flow&);
00144
00154     bool run(int t_init, int t_final);
00155
00159     void showModel() const;
00160
00161 private:
00162
00168     void add(Model*);
00169
00175     void add(Flow*);
00176
00182     void add(System*);
00183
00190     void add(Flow*, int i);
00191
00192 public:
00196     string id_;
00197
00198 };
00199
00200 #endif

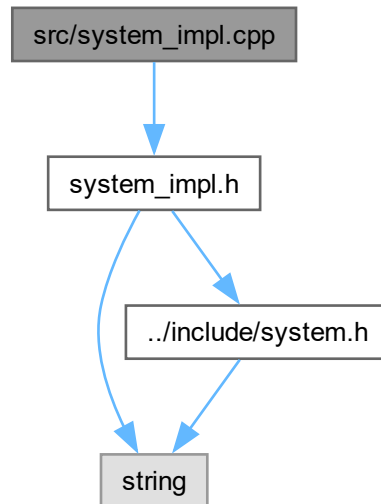
```

## 8.17 src/system\_impl.cpp File Reference

Implementação da classe [System\\_Impl](#).

```
#include "system_impl.h"
```

Include dependency graph for system\_impl.cpp:



### 8.17.1 Detailed Description

Implementação da classe [System\\_Impl](#).

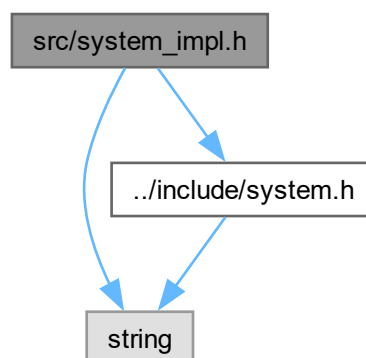
Este arquivo contém a implementação dos métodos responsáveis pela manipulação dos sistemas do simulador, incluindo construtores, destrutor, operador de atribuição e getters/setters de nome e valor.

## 8.18 src/system\_impl.h File Reference

```
#include <string>
```

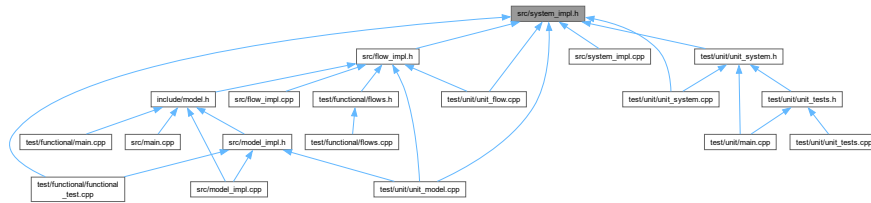
```
#include "../include/system.h"
```

Include dependency graph for system\_impl.h:





This graph shows which files directly or indirectly include this file:



## Data Structures

- class [System\\_Impl](#)  
Classe responsável por representar um sistema.

## 8.19 system\_impl.h

[Go to the documentation of this file.](#)

```

00001 #ifndef SYSTEM_IMPL_H
00002 #define SYSTEM_IMPL_H
00003
00004 #include <string>
00005 #include "../include/system.h"
00006
00007 using namespace std;
00008
00015 class System_Impl : public System {
00016 private:
00017
00021     double value;
00022
00026     string name;
00027
00028 public:
00030
00033     System_Impl();
00034
00041     System_Impl(const string& name, double value);
00042
00050     System_Impl(const System_Impl& other);
00051
00055     virtual ~System_Impl();
00056
00065     System_Impl& operator=(const System_Impl& other);
00066
00068
00073     string getName() const;
00074
00080     void setName(const string& name);
00081
00087     double getValue() const;
00088
00094     void setValue(double value);
00095 public:
00096     string id_;
00097
00098 };
00099
00100 #endif

```

## 8.20 test/functional/flows.cpp File Reference

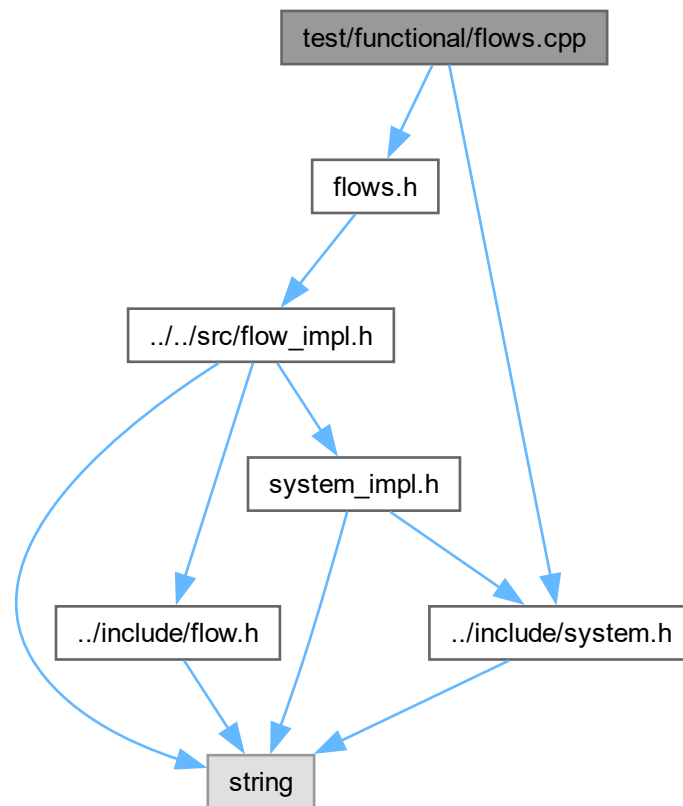
Implementação dos fluxos do simulador.

```

#include "flows.h"
#include "../include/system.h"

```

Include dependency graph for flows.cpp:



### 8.20.1 Detailed Description

Implementação dos fluxos do simulador.

Este arquivo contém as implementações dos fluxos:

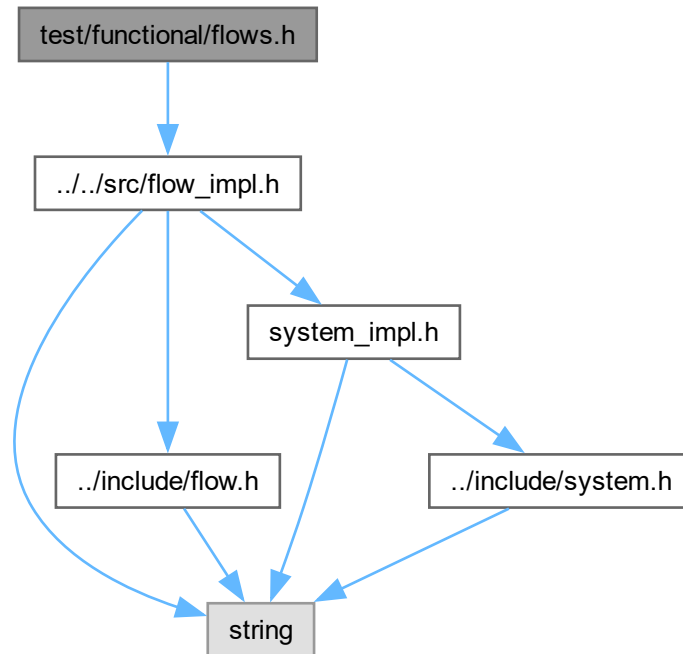
- exponencial
- logístico
- complexo

## 8.21 test/functional/flows.h File Reference

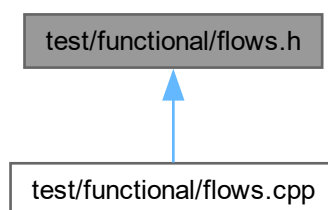
Declaração das classes de fluxo concretas: exponencial, logístico e complexo.

```
#include "../src/flow_impl.h"
```

Include dependency graph for flows.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- class [FlowExponencial](#)  
Fluxo baseado em crescimento exponencial.
- class [FlowLogistico](#)  
Fluxo baseado em crescimento logístico.
- class [FlowComplexo](#)  
Fluxo com equação de múltiplas interações entre sistemas.

### 8.21.1 Detailed Description

Declaração das classes de fluxo concretas: exponencial, logístico e complexo.

## 8.22 flows.h

[Go to the documentation of this file.](#)

```

00001 #ifndef FLOWS_H
00002 #define FLOWS_H
00003
00004 #include "../src/flow_impl.h"
00005
00010
00019 class FlowExponencial : public Flow__Impl {
00020 public:
00021
00025     FlowExponencial();
00026
00034     FlowExponencial(
00035         const string& name,
00036         System* source,
00037         System* target
00038     );
00039
00045     FlowExponencial(
00046         const FlowExponencial& other
00047     );
00048
00052     virtual ~FlowExponencial();
00053
00060     FlowExponencial& operator=(
00061         const FlowExponencial& other
00062     );
00063
00073     double execute() override;
00074 };
00075
00084 class FlowLogistico : public Flow__Impl {
00085 public:
00086
00090     FlowLogistico();
00091
00099     FlowLogistico(
00100         const string& name,
00101         System* source,
00102         System* target
00103     );
00104
00110     FlowLogistico(
00111         const FlowLogistico& other
00112     );
00113
00117     virtual ~FlowLogistico();
00118
00125     FlowLogistico& operator=(
00126         const FlowLogistico& other
00127     );
00128
00138     double execute() override;
00139 };
00140
00149 class FlowComplexo : public Flow__Impl {
00150 public:
00151
00155     FlowComplexo();
00156
00164     FlowComplexo(
00165         const string& name,
00166         System* source,
00167         System* target
00168     );
00169
00175     FlowComplexo(
00176         const FlowComplexo& other
00177     );
00178
00182     virtual ~FlowComplexo();
00183
00190     FlowComplexo& operator=(
00191         const FlowComplexo& other
00192     );
00193
00204     double execute() override;
00205 };

```

00206

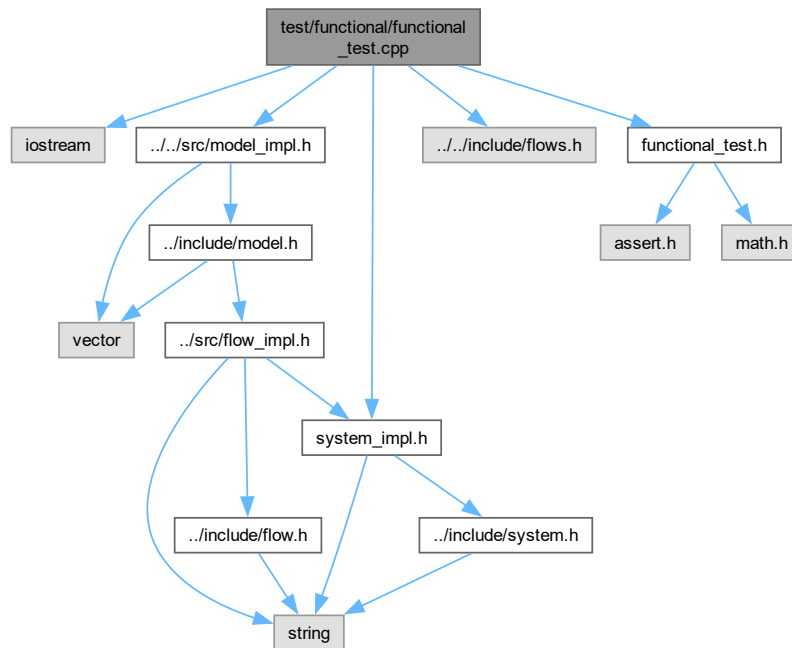
00207 `#endif`

## 8.23 test/functional/functional\_test.cpp File Reference

Implementação dos testes funcionais do simulador.

```
#include <iostream>
#include "../src/model_impl.h"
#include "../src/system_impl.h"
#include "../include/flows.h"
#include "functional_test.h"
```

Include dependency graph for functional\_test.cpp:



### Functions

- `bool floatingPointComparison (double a, double b)`  
Compara dois números de ponto flutuante com margem de tolerância.
- `void exponentialFuncionalTest ()`  
Teste Funcional Exponencial.
- `void logisticalFuncionalTest ()`  
Teste Funcional Logístico.
- `void complexFuncionalTest ()`  
Teste Funcional Complexo.

### 8.23.1 Detailed Description

Implementação dos testes funcionais do simulador.

Este arquivo contém os testes funcionais que verificam se os resultados produzidos pela simulação correspondem aos valores analíticos esperados para cada modelo:

- Exponencial

- Logístico
- Complexo

A comparação entre valores de ponto flutuante é feita com uma margem de tolerância de 0.0001 para evitar erros de precisão numérica.

## 8.23.2 Function Documentation

### 8.23.2.1 complexFuncionalTest()

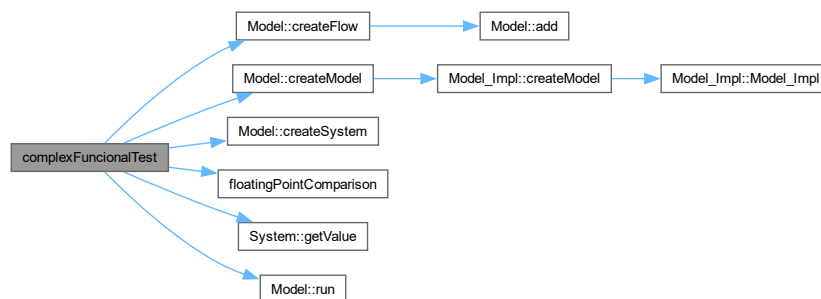
void complexFuncionalTest ()

Teste Funcional Complexo.

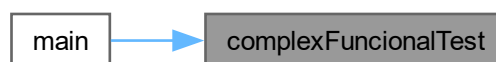
Executa o teste funcional do modelo complexo.

Teste Funcional Complexo.

Cria um modelo com múltiplos sistemas e fluxos interligados, executa a simulação por um intervalo de tempo pré-definido e verifica, via assert, se os valores finais de cada sistema correspondem aos resultados esperados. Here is the call graph for this function:



Here is the caller graph for this function:



### 8.23.2.2 exponentialFuncionalTest()

void exponentialFuncionalTest ()

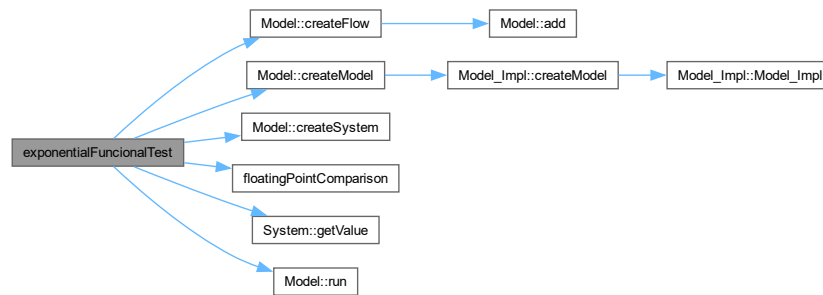
Teste Funcional Exponencial.

Executa o teste funcional do modelo exponencial.

Teste Funcional Exponencial.

Cria um modelo com dois sistemas e um fluxo exponencial, executa a simulação por um intervalo de tempo pré-definido e verifica, via assert, se os valores finais dos sistemas correspondem aos resultados

esperados analiticamente. Here is the call graph for this function:



Here is the caller graph for this function:



### 8.23.2.3 floatingPointComparison()

```
bool floatingPointComparison (
    double a,
    double b)
```

Compara dois números de ponto flutuante com margem de tolerância.

Utiliza uma margem de erro de 0.0001 para evitar falhas causadas por imprecisões de representação em ponto flutuante.

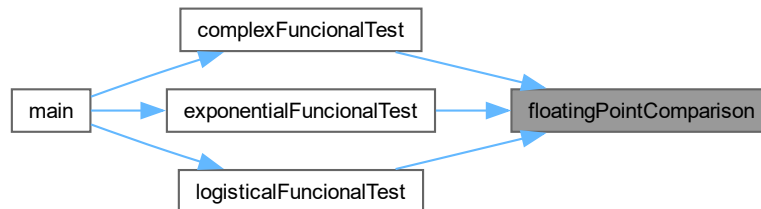
Parameters

|   |                            |
|---|----------------------------|
| a | Primeiro valor a comparar. |
| b | Segundo valor a comparar.  |

## Returns

true se a diferença absoluta entre a e b for menor que 0.0001.  
false caso contrário.

Here is the caller graph for this function:



## 8.23.2.4 logisticalFuncionalTest()

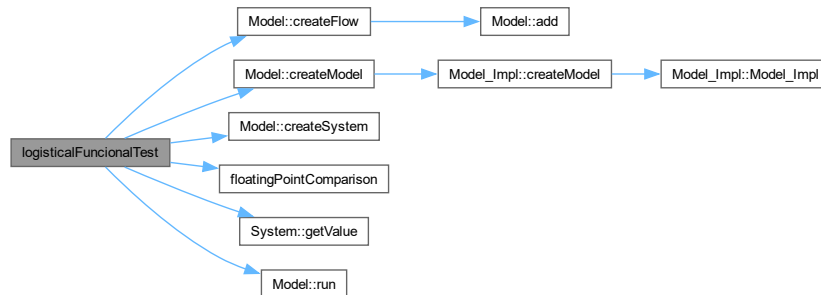
void logisticalFuncionalTest ()

Teste Funcional Logistico.

Executa o teste funcional do modelo logístico.

Teste Funcional Logistico.

Cria um modelo com dois sistemas e um fluxo logístico, executa a simulação por um intervalo de tempo pré-definido e verifica, via assert, se os valores finais dos sistemas correspondem aos resultados esperados analiticamente. Here is the call graph for this function:



Here is the caller graph for this function:





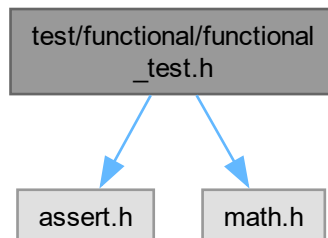
## 8.24 test/functional/functional\_test.h File Reference

Declaração dos testes funcionais dos modelos de simulação.

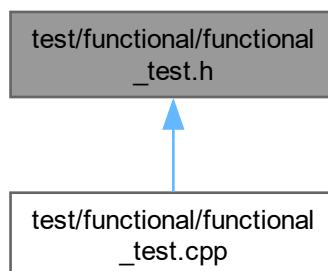
```
#include <assert.h>
```

```
#include <math.h>
```

Include dependency graph for functional\_test.h:



This graph shows which files directly or indirectly include this file:



### Functions

- void [exponentialFuncionalTest](#) ()  
Executa o teste funcional do modelo exponencial.
- void [logisticalFuncionalTest](#) ()  
Executa o teste funcional do modelo logístico.
- void [complexFuncionalTest](#) ()  
Executa o teste funcional do modelo complexo.

#### 8.24.1 Detailed Description

Declaração dos testes funcionais dos modelos de simulação.

Este arquivo contém os protótipos das funções de teste funcional responsáveis por verificar o comportamento esperado dos modelos exponencial, logístico e complexo após a execução da simulação.

## 8.24.2 Function Documentation

### 8.24.2.1 complexFuncionalTest()

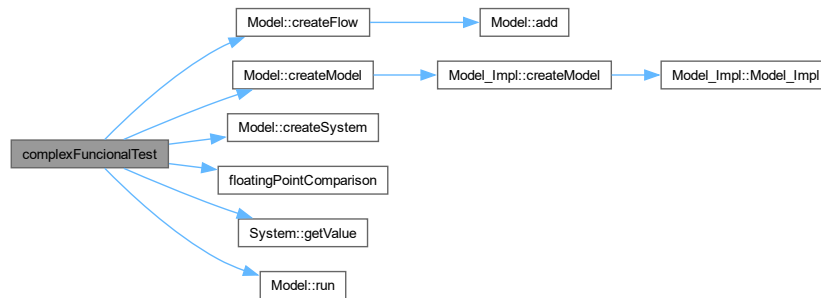
void complexFuncionalTest ()

Executa o teste funcional do modelo complexo.

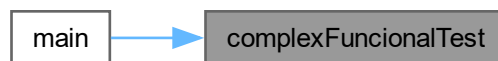
Teste Funcional Complexo.

Cria um modelo com múltiplos sistemas e fluxos interligados, executa a simulação por um intervalo de tempo pré-definido e verifica, via assert, se os valores finais de cada sistema correspondem aos resultados esperados.

Executa o teste funcional do modelo complexo. Here is the call graph for this function:



Here is the caller graph for this function:



### 8.24.2.2 exponentialFuncionalTest()

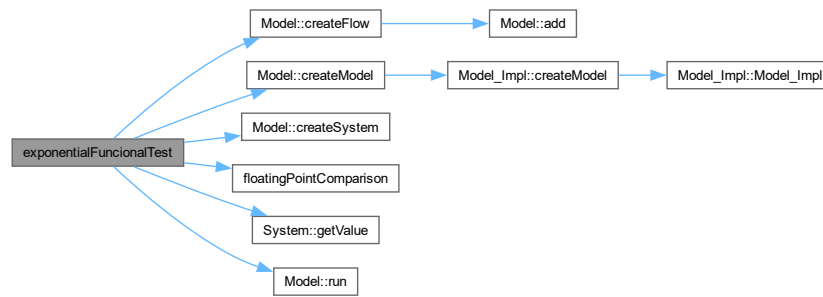
void exponentialFuncionalTest ()

Executa o teste funcional do modelo exponencial.

Teste Funcional Exponencial.

Cria um modelo com dois sistemas e um fluxo exponencial, executa a simulação por um intervalo de tempo pré-definido e verifica, via assert, se os valores finais dos sistemas correspondem aos resultados esperados analiticamente.

Executa o teste funcional do modelo exponencial. Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.24.2.3 logisticalFuncionalTest()

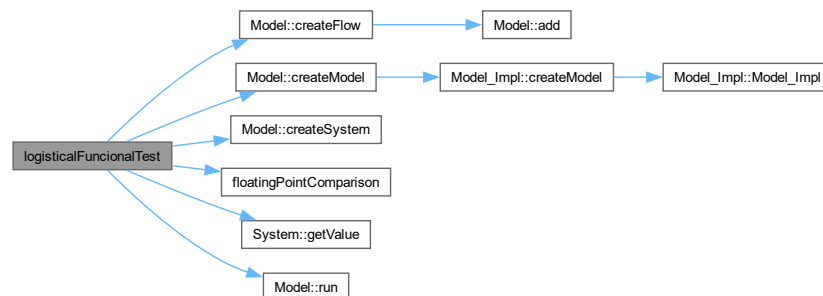
```
void logisticalFuncionalTest ()
```

Executa o teste funcional do modelo logístico.

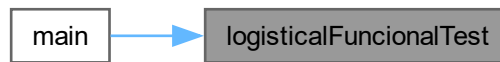
Teste Funcional Logistico.

Cria um modelo com dois sistemas e um fluxo logístico, executa a simulação por um intervalo de tempo pré-definido e verifica, via assert, se os valores finais dos sistemas correspondem aos resultados esperados analiticamente.

Executa o teste funcional do modelo logístico. Here is the call graph for this function:



Here is the caller graph for this function:



## 8.25 functional\_test.h

[Go to the documentation of this file.](#)

```

00001 #ifndef FUNCTIONAL_TESTS_H
00002 #define FUNCTIONAL_TESTS_H
00003
00004 #include <assert.h>
00005 #include <math.h>
00006
00015
00024 void exponentialFuncionalTest();
00025
00034 void logisticalFuncionalTest();
00035
00044 void complexFuncionalTest();
00045
00046 #endif
  
```

## 8.26 test/unit/unit\_flow.cpp File Reference

Implementação dos testes unitários da classe [Flow](#).

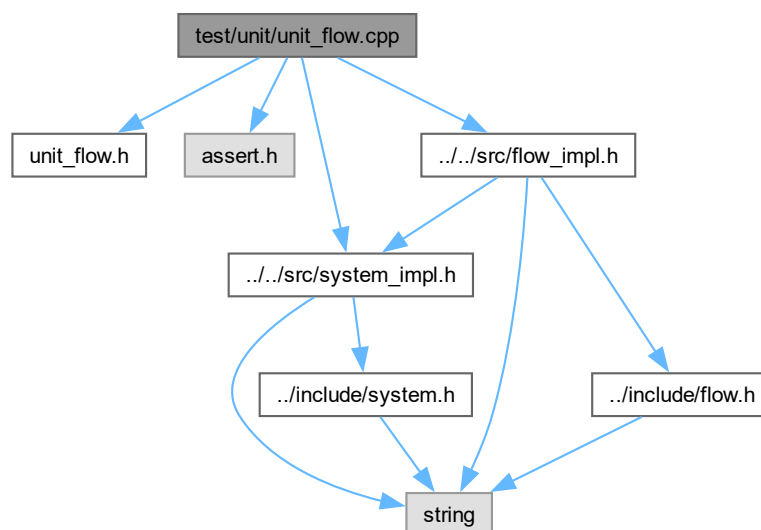
```
#include "unit_flow.h"
```

```
#include <assert.h>
```

```
#include "../src/system_impl.h"
```

```
#include "../src/flow_impl.h"
```

Include dependency graph for unit\_flow.cpp:



## Data Structures

- class [Flow\\_Test](#)  
Implementação concreta de [Flow\\_Impl](#) utilizada nos testes unitários.

## Functions

- void [unit\\_Flow\\_constructor](#) (void)  
Testa o construtor parametrizado da classe [Flow](#).
- void [unit\\_Flow\\_destructor](#) (void)  
Testa o destrutor da classe [Flow](#).
- void [unit\\_Flow\\_getName](#) (void)  
Testa o método getName() da classe [Flow](#).
- void [unit\\_Flow\\_setName](#) (void)  
Testa o método setName() da classe [Flow](#).
- void [unit\\_Flow\\_getSource](#) (void)  
Testa o método getSource() da classe [Flow](#).
- void [unit\\_Flow\\_setSource](#) (void)  
Testa o método setSource() da classe [Flow](#).
- void [unit\\_Flow\\_getTarget](#) (void)  
Testa o método getTarget() da classe [Flow](#).
- void [unit\\_Flow\\_setTarget](#) (void)  
Testa o método setTarget() da classe [Flow](#).
- void [unit\\_Flow\\_execute](#) (void)  
Testa o método execute() da classe [Flow](#).
- void [run\\_unit\\_tests\\_Flow](#) (void)  
Executa todos os testes unitários da classe [Flow](#) em sequência.

### 8.26.1 Detailed Description

Implementação dos testes unitários da classe [Flow](#).

Contém uma classe de fluxo concreta auxiliar ([Flow\\_Test](#)) utilizada exclusivamente nos testes, além das implementações de cada função de teste unitário declarada em [unit\\_Flow.h](#).

### 8.26.2 Function Documentation

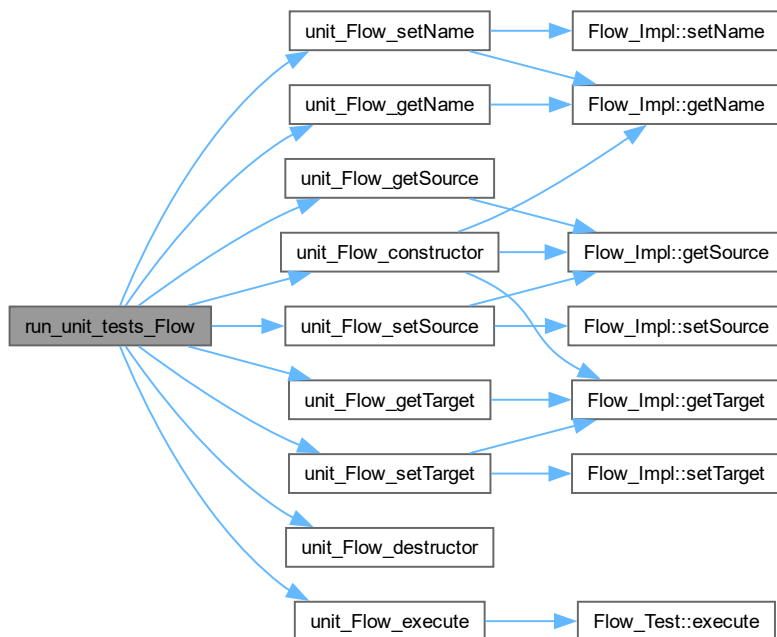
#### 8.26.2.1 run\_unit\_tests\_Flow()

```
void run_unit_tests_Flow (  
    void )
```

Executa todos os testes unitários da classe [Flow](#) em sequência.

Chama cada função de teste unitário da classe [Flow](#) e exibe uma mensagem de confirmação caso todos

sejam aprovados. Here is the call graph for this function:



Here is the caller graph for this function:



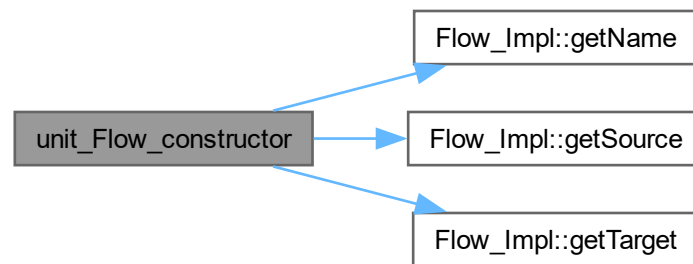
### 8.26.2.2 unit\_Flow\_constructor()

```
void unit_Flow_constructor (
    void )
```

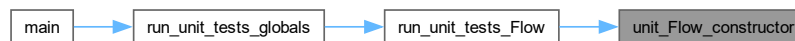
Testa o construtor parametrizado da classe [Flow](#).

Verifica se, ao criar um fluxo com nome, sistema de origem e sistema de destino, os atributos são corre-

tamente inicializados. Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.26.2.3 unit\_Flow\_destructor()

```
void unit_Flow_destructor (
    void )
```

Testa o destrutor da classe [Flow](#).

Verifica se o objeto é destruído sem erros ou vazamentos de memória. Here is the caller graph for this function:



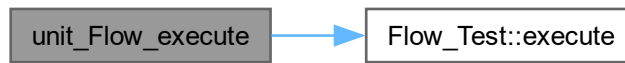
#### 8.26.2.4 unit\_Flow\_execute()

```
void unit_Flow_execute (
    void )
```

Testa o método `execute()` da classe [Flow](#).

Verifica se o valor retornado por `execute()` corresponde ao resultado esperado para a equação do fluxo de

teste. Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.26.2.5 unit\_Flow\_getName()

```
void unit_Flow_getName (  
    void )
```

Testa o método `getName()` da classe [Flow](#).

Verifica se o nome retornado corresponde ao valor definido na construção do objeto. Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.26.2.6 unit\_Flow\_getSource()

```
void unit_Flow_getSource (  
    void )
```

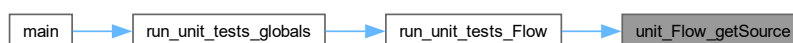
Testa o método `getSource()` da classe [Flow](#).



Verifica se o ponteiro para o sistema de origem retornado corresponde ao sistema definido na construção do objeto. Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.26.2.7 unit\_Flow\_getTarget()

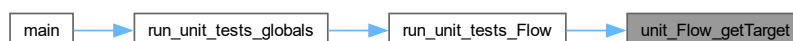
```
void unit_Flow_getTarget (  
    void )
```

Testa o método `getTarget()` da classe [Flow](#).

Verifica se o ponteiro para o sistema de destino retornado corresponde ao sistema definido na construção do objeto. Here is the call graph for this function:



Here is the caller graph for this function:

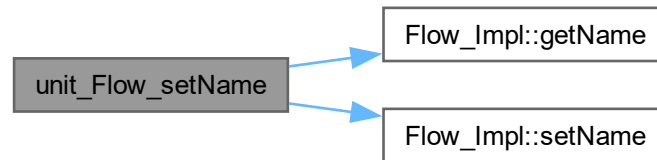


#### 8.26.2.8 unit\_Flow\_setName()

```
void unit_Flow_setName (  
    void )
```

Testa o método `setName()` da classe [Flow](#).

Verifica se o nome do fluxo é atualizado corretamente após a chamada de setName(). Here is the call graph for this function:



Here is the caller graph for this function:

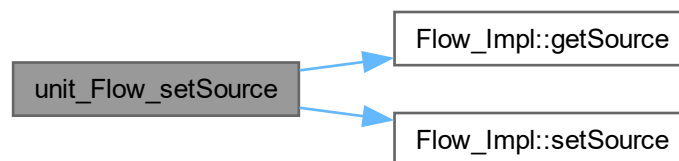


#### 8.26.2.9 unit\_Flow\_setSource()

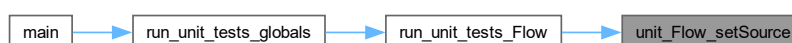
```
void unit_Flow_setSource (
    void )
```

Testa o método `setSource()` da classe [Flow](#).

Verifica se o sistema de origem é atualizado corretamente após a chamada de `setSource()`. Here is the call graph for this function:



Here is the caller graph for this function:

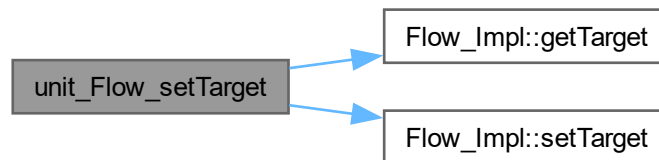


## 8.26.2.10 unit\_Flow\_setTarget()

```
void unit_Flow_setTarget (
    void )
```

Testa o método setTarget() da classe [Flow](#).

Verifica se o sistema de destino é atualizado corretamente após a chamada de setTarget(). Here is the call graph for this function:



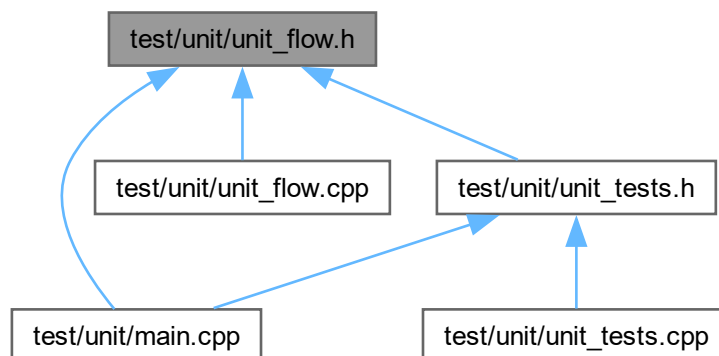
Here is the caller graph for this function:



## 8.27 test/unit/unit\_flow.h File Reference

Declaração dos testes unitários da classe [Flow](#).

This graph shows which files directly or indirectly include this file:



## Functions

- void [unit\\_Flow\\_constructor](#) (void)

- Testa o construtor parametrizado da classe [Flow](#).
- void [unit\\_Flow\\_destructor](#) (void)  
Testa o destrutor da classe [Flow](#).
- void [unit\\_Flow\\_getName](#) (void)  
Testa o método getName() da classe [Flow](#).
- void [unit\\_Flow\\_setName](#) (void)  
Testa o método setName() da classe [Flow](#).
- void [unit\\_Flow\\_getSource](#) (void)  
Testa o método getSource() da classe [Flow](#).
- void [unit\\_Flow\\_setSource](#) (void)  
Testa o método setSource() da classe [Flow](#).
- void [unit\\_Flow\\_getTarget](#) (void)  
Testa o método getTarget() da classe [Flow](#).
- void [unit\\_Flow\\_setTarget](#) (void)  
Testa o método setTarget() da classe [Flow](#).
- void [unit\\_Flow\\_execute](#) (void)  
Testa o método execute() da classe [Flow](#).
- void [run\\_unit\\_tests\\_Flow](#) (void)  
Executa todos os testes unitários da classe [Flow](#) em sequência.

### 8.27.1 Detailed Description

Declaração dos testes unitários da classe [Flow](#).

Este arquivo contém os protótipos das funções de teste unitário responsáveis por verificar individualmente cada método da classe [Flow](#) e sua implementação concreta [Flow\\_Impl](#).

### 8.27.2 Function Documentation

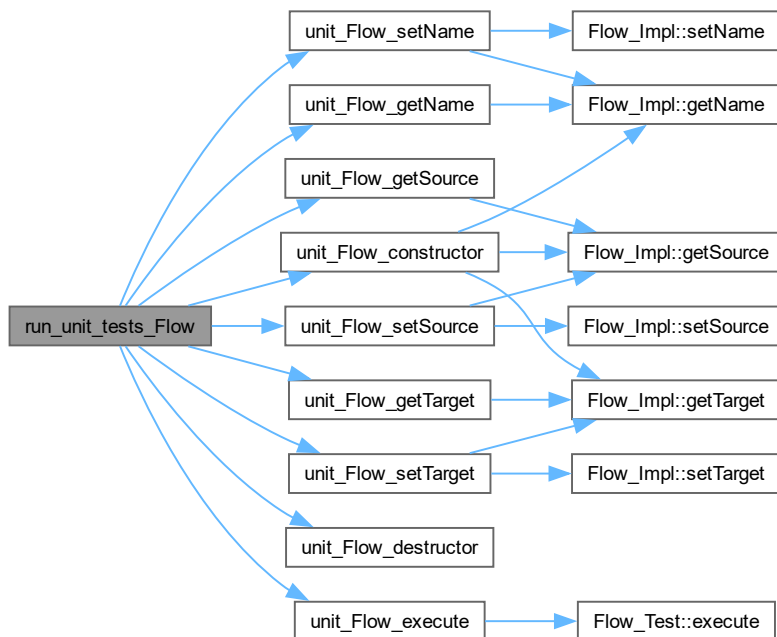
#### 8.27.2.1 [run\\_unit\\_tests\\_Flow\(\)](#)

```
void run_unit_tests_Flow (  
    void )
```

Executa todos os testes unitários da classe [Flow](#) em sequência.

Chama cada função de teste unitário da classe [Flow](#) e exibe uma mensagem de confirmação caso todos

sejam aprovados. Here is the call graph for this function:



Here is the caller graph for this function:



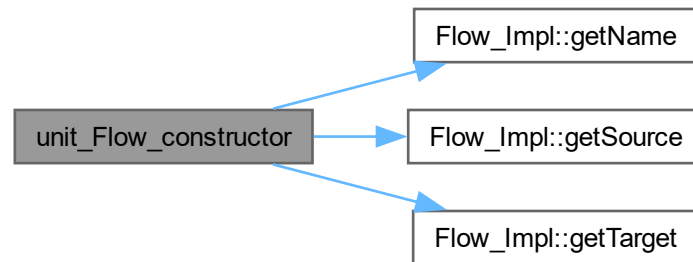
### 8.27.2.2 unit\_Flow\_constructor()

```
void unit_Flow_constructor (
    void )
```

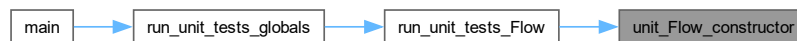
Testa o construtor parametrizado da classe [Flow](#).

Verifica se, ao criar um fluxo com nome, sistema de origem e sistema de destino, os atributos são corre-

tamente inicializados. Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.27.2.3 unit\_Flow\_destructor()

```
void unit_Flow_destructor (
    void )
```

Testa o destrutor da classe [Flow](#).

Verifica se o objeto é destruído sem erros ou vazamentos de memória. Here is the caller graph for this function:



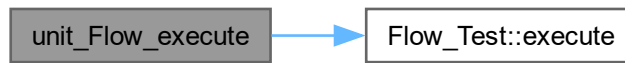
#### 8.27.2.4 unit\_Flow\_execute()

```
void unit_Flow_execute (
    void )
```

Testa o método `execute()` da classe [Flow](#).

Verifica se o valor retornado por `execute()` corresponde ao resultado esperado para a equação do fluxo de

teste. Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.27.2.5 unit\_Flow\_getName()

```
void unit_Flow_getName (  
    void )
```

Testa o método `getName()` da classe [Flow](#).

Verifica se o nome retornado corresponde ao valor definido na construção do objeto. Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.27.2.6 unit\_Flow\_getSource()

```
void unit_Flow_getSource (  
    void )
```

Testa o método `getSource()` da classe [Flow](#).

Verifica se o ponteiro para o sistema de origem retornado corresponde ao sistema definido na construção do objeto. Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.27.2.7 unit\_Flow\_getTarget()

```
void unit_Flow_getTarget (
    void )
```

Testa o método `getTarget()` da classe [Flow](#).

Verifica se o ponteiro para o sistema de destino retornado corresponde ao sistema definido na construção do objeto. Here is the call graph for this function:



Here is the caller graph for this function:



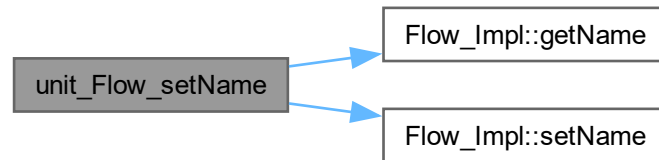
#### 8.27.2.8 unit\_Flow\_setName()

```
void unit_Flow_setName (
    void )
```

Testa o método `setName()` da classe [Flow](#).



Verifica se o nome do fluxo é atualizado corretamente após a chamada de setName(). Here is the call graph for this function:



Here is the caller graph for this function:

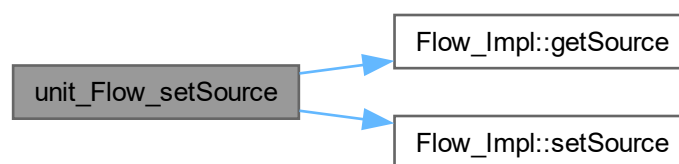


#### 8.27.2.9 unit\_Flow\_setSource()

```
void unit_Flow_setSource (
    void )
```

Testa o método `setSource()` da classe [Flow](#).

Verifica se o sistema de origem é atualizado corretamente após a chamada de `setSource()`. Here is the call graph for this function:



Here is the caller graph for this function:

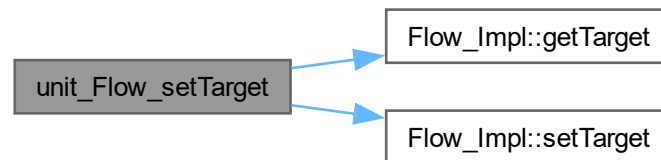


### 8.27.2.10 unit\_Flow\_setTarget()

```
void unit_Flow_setTarget (
    void )
```

Testa o método setTarget() da classe [Flow](#).

Verifica se o sistema de destino é atualizado corretamente após a chamada de setTarget(). Here is the call graph for this function:



Here is the caller graph for this function:



## 8.28 unit\_flow.h

[Go to the documentation of this file.](#)

```

00001 #ifndef UNIT_FLOW_H
00002 #define UNIT_FLOW_H
00003
00012
00019 void unit_Flow_constructor(void);
00020
00026 void unit_Flow_destructor(void);
00027
00034 void unit_Flow_getName(void);
00035
00042 void unit_Flow_setName(void);
00043
00050 void unit_Flow_getSource(void);
00051
00058 void unit_Flow_setSource(void);
00059
00066 void unit_Flow_getTarget(void);
00067
00074 void unit_Flow_setTarget(void);
00075
00082 void unit_Flow_execute(void);
00083
00090 void run_unit_tests_Flow(void);
00091
00092 #endif
```

## 8.29 test/unit/unit\_model.cpp File Reference

Implementação dos testes unitários da classe [Model](#).

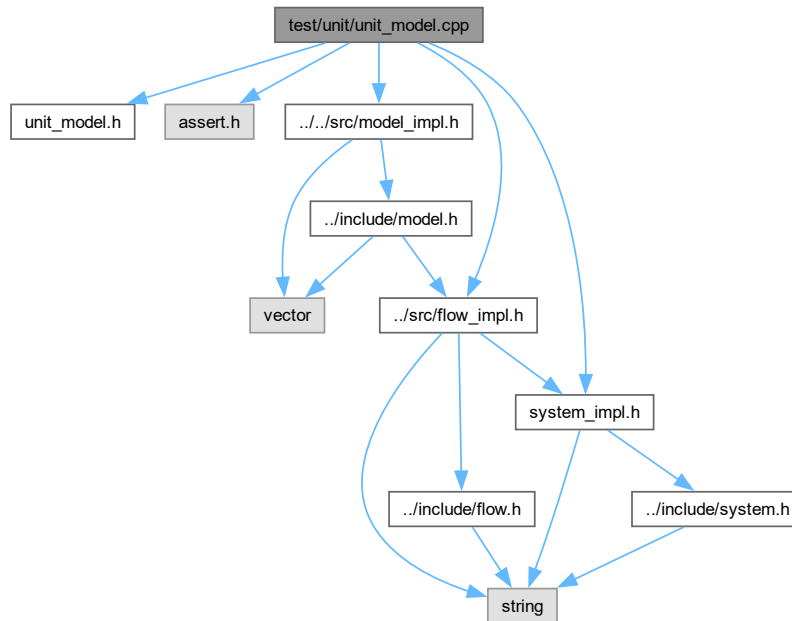
```

#include "unit_model.h"
#include <assert.h>
#include "../src/model_impl.h"
```

```
#include "../src/system_impl.h"
```

```
#include "../src/flow_impl.h"
```

Include dependency graph for unit\_model.cpp:



## Data Structures

- class [FlowTest](#)  
Implementação concreta de [Flow\\_Impl](#) utilizada nos testes do [Model](#).

## Functions

- void [unit\\_Model\\_constructor](#) (void)  
Testa o construtor da classe [Model](#).
- void [unit\\_Model\\_destructor](#) (void)  
Testa o destrutor da classe [Model](#).
- void [unit\\_Model\\_addSystem](#) (void)  
Testa o método add(System\*) da classe [Model](#).
- void [unit\\_Model\\_addFlow](#) (void)  
Testa o método add(Flow\*) da classe [Model](#).
- void [unit\\_Model\\_run](#) (void)  
Testa o método run() da classe [Model](#).
- void [unit\\_Model\\_showModel](#) (void)  
Testa o método showModel() da classe [Model](#).
- void [run\\_unit\\_tests\\_Model](#) (void)  
Executa todos os testes unitários da classe [Model](#) em sequência.

### 8.29.1 Detailed Description

Implementação dos testes unitários da classe [Model](#).

Contém uma classe de fluxo concreta auxiliar ([FlowTest](#)) utilizada exclusivamente nos testes, além das implementações de cada função de teste unitário declarada em [unit\\_Model.h](#).

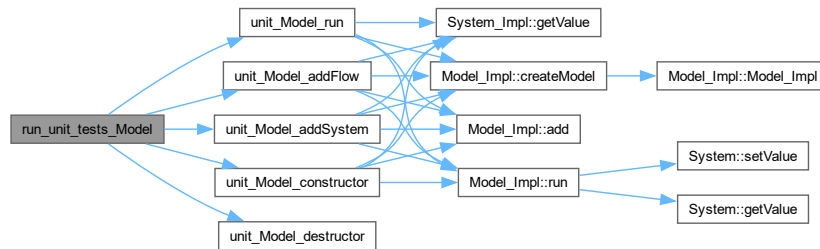
## 8.29.2 Function Documentation

### 8.29.2.1 run\_unit\_tests\_Model()

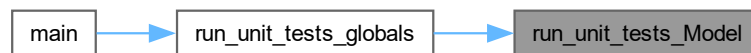
```
void run_unit_tests_Model (
    void )
```

Executa todos os testes unitários da classe [Model](#) em sequência.

Chama cada função de teste unitário da classe [Model](#) e exibe uma mensagem de confirmação caso todos sejam aprovados. Here is the call graph for this function:



Here is the caller graph for this function:

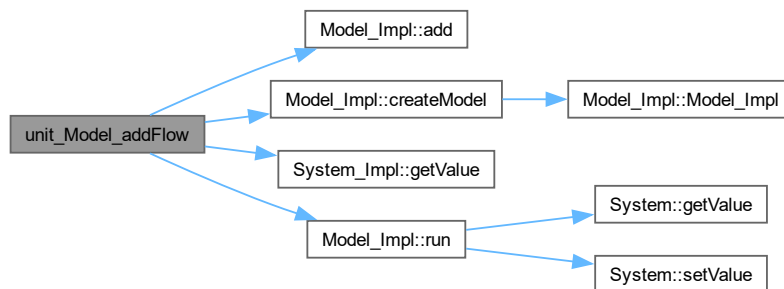


### 8.29.2.2 unit\_Model\_addFlow()

```
void unit_Model_addFlow (
    void )
```

Testa o método `add(Flow*)` da classe [Model](#).

Verifica o comportamento do modelo com e sem fluxo adicionado: sem fluxo os sistemas não se alteram; com fluxo a transferência ocorre corretamente a cada passo de tempo. Here is the call graph for this function:



Here is the caller graph for this function:

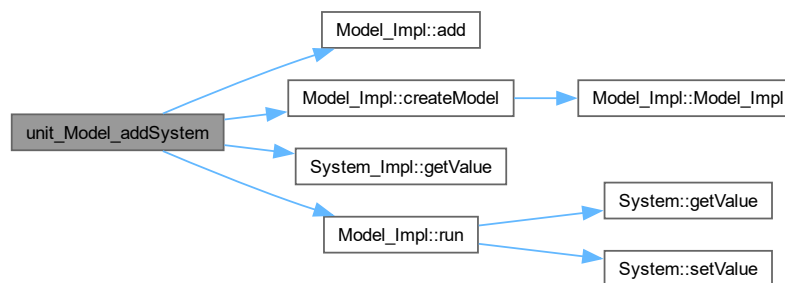


### 8.29.2.3 unit\_Model\_addSystem()

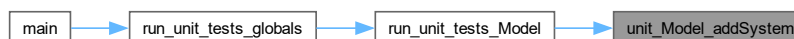
```
void unit_Model_addSystem (
    void )
```

Testa o método add(System\*) da classe [Model](#).

Verifica se sistemas adicionados ao modelo participam corretamente da simulação, tendo seus valores atualizados após a execução dos fluxos. Here is the call graph for this function:



Here is the caller graph for this function:



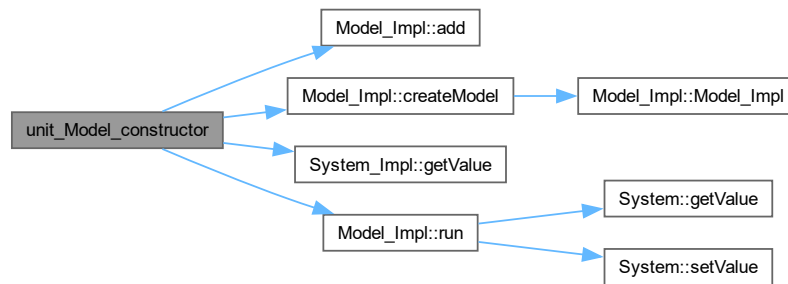
### 8.29.2.4 unit\_Model\_constructor()

```
void unit_Model_constructor (
    void )
```

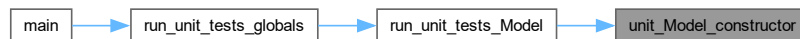
Testa o construtor da classe [Model](#).

Cria um modelo, adiciona sistemas e um fluxo de teste, executa a simulação por um passo e verifica se

os valores dos sistemas foram atualizados corretamente. Here is the call graph for this function:



Here is the caller graph for this function:

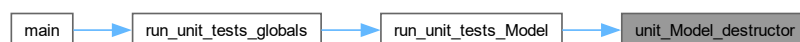


#### 8.29.2.5 unit\_Model\_destructor()

```
void unit_Model_destructor (
    void )
```

Testa o destrutor da classe [Model](#).

Verifica se o objeto é destruído sem erros ou vazamentos de memória. Here is the caller graph for this function:



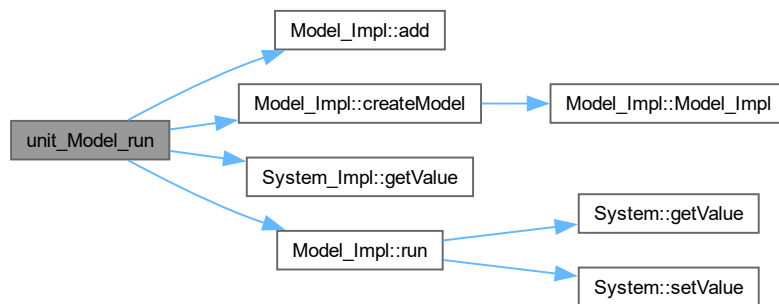
#### 8.29.2.6 unit\_Model\_run()

```
void unit_Model_run (
    void )
```

Testa o método `run()` da classe [Model](#).

Executa a simulação por múltiplos passos de tempo e verifica se os valores finais dos sistemas correspondem

ao total acumulado de transferências esperado. Here is the call graph for this function:



Here is the caller graph for this function:

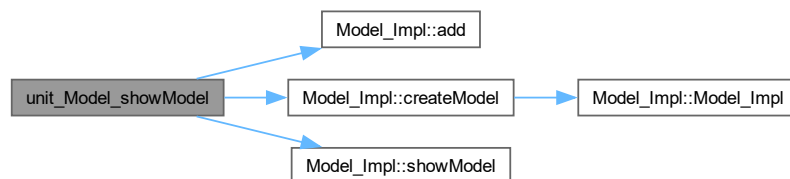


#### 8.29.2.7 unit\_Model\_showModel()

```
void unit_Model_showModel (
    void )
```

Testa o método `showModel()` da classe [Model](#).

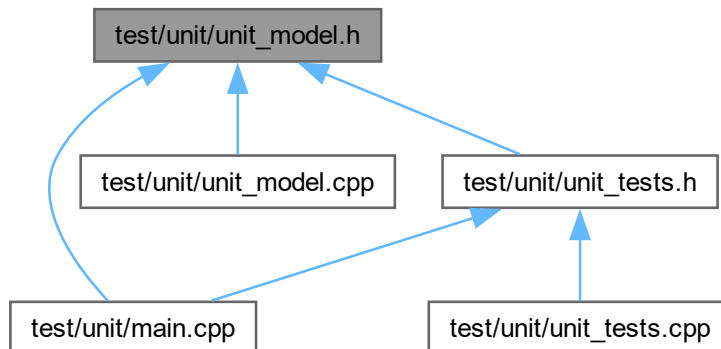
Verifica se a exibição das informações do modelo é executada sem erros. A correção visual da saída deve ser verificada manualmente. Here is the call graph for this function:



## 8.30 test/unit/unit\_model.h File Reference

Declaração dos testes unitários da classe [Model](#).

This graph shows which files directly or indirectly include this file:



## Functions

- void [unit\\_Model\\_constructor](#) (void)  
Testa o construtor da classe [Model](#).
- void [unit\\_Model\\_destructor](#) (void)  
Testa o destrutor da classe [Model](#).
- void [unit\\_Model\\_addSystem](#) (void)  
Testa o método add(System\*) da classe [Model](#).
- void [unit\\_Model\\_addFlow](#) (void)  
Testa o método add(Flow\*) da classe [Model](#).
- void [unit\\_Model\\_run](#) (void)  
Testa o método run() da classe [Model](#).
- void [unit\\_Model\\_showModel](#) (void)  
Testa o método showModel() da classe [Model](#).
- void [run\\_unit\\_tests\\_Model](#) (void)  
Executa todos os testes unitários da classe [Model](#) em sequência.

### 8.30.1 Detailed Description

Declaração dos testes unitários da classe [Model](#).

Este arquivo contém os protótipos das funções de teste unitário responsáveis por verificar individualmente cada método da classe [Model](#) e sua implementação concreta [Model\\_Impl](#).

### 8.30.2 Function Documentation

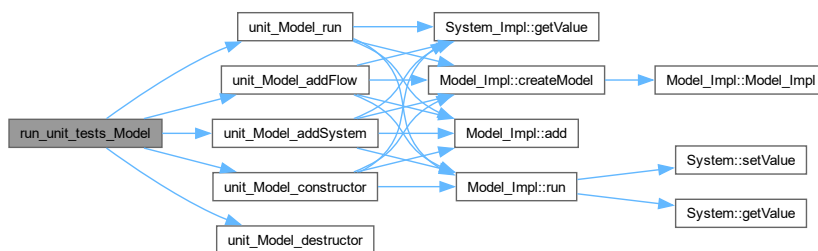
#### 8.30.2.1 run\_unit\_tests\_Model()

```
void run_unit_tests_Model (
    void )
```

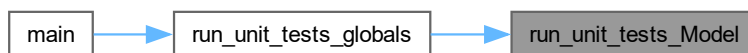
Executa todos os testes unitários da classe [Model](#) em sequência.



Chama cada função de teste unitário da classe [Model](#) e exibe uma mensagem de confirmação caso todos sejam aprovados. Here is the call graph for this function:



Here is the caller graph for this function:

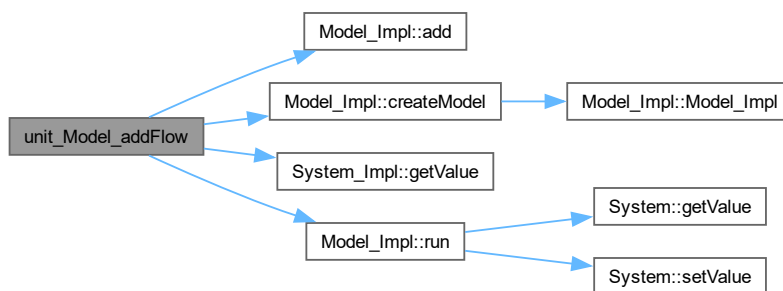


### 8.30.2.2 unit\_Model\_addFlow()

```
void unit_Model_addFlow (
    void )
```

Testa o método `add(Flow*)` da classe [Model](#).

Verifica o comportamento do modelo com e sem fluxo adicionado: sem fluxo os sistemas não se alteram; com fluxo a transferência ocorre corretamente a cada passo de tempo. Here is the call graph for this function:



Here is the caller graph for this function:

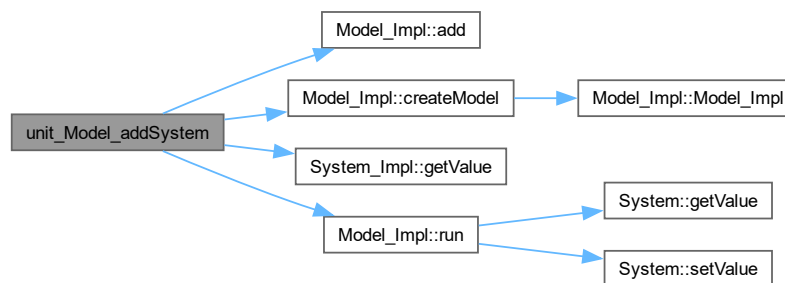


### 8.30.2.3 unit\_Model\_addSystem()

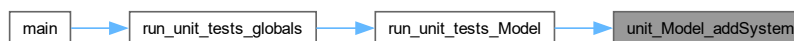
```
void unit_Model_addSystem (
    void )
```

Testa o método `add(System*)` da classe [Model](#).

Verifica se sistemas adicionados ao modelo participam corretamente da simulação, tendo seus valores atualizados após a execução dos fluxos. Here is the call graph for this function:



Here is the caller graph for this function:



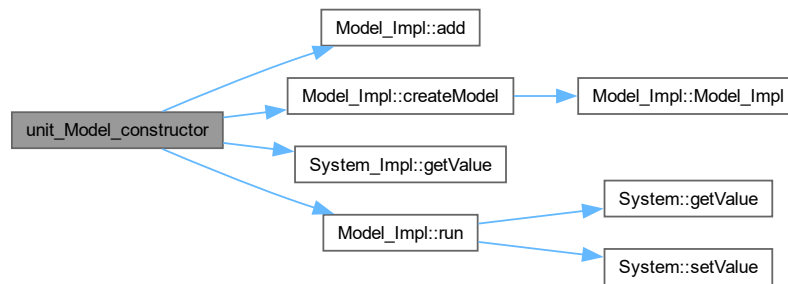
### 8.30.2.4 unit\_Model\_constructor()

```
void unit_Model_constructor (
    void )
```

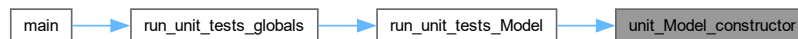
Testa o construtor da classe [Model](#).

Cria um modelo, adiciona sistemas e um fluxo de teste, executa a simulação por um passo e verifica se

os valores dos sistemas foram atualizados corretamente. Here is the call graph for this function:



Here is the caller graph for this function:

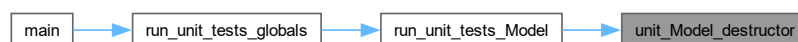


#### 8.30.2.5 unit\_Model\_destructor()

```
void unit_Model_destructor (
    void )
```

Testa o destrutor da classe [Model](#).

Verifica se o objeto é destruído sem erros ou vazamentos de memória. Here is the caller graph for this function:



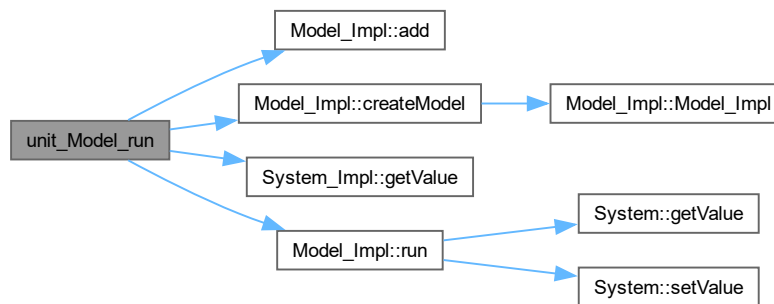
#### 8.30.2.6 unit\_Model\_run()

```
void unit_Model_run (
    void )
```

Testa o método `run()` da classe [Model](#).

Executa a simulação por múltiplos passos de tempo e verifica se os valores finais dos sistemas correspondem

ao total acumulado de transferências esperado. Here is the call graph for this function:



Here is the caller graph for this function:

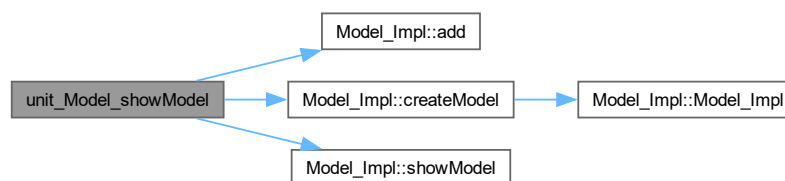


### 8.30.2.7 unit\_Model\_showModel()

```
void unit_Model_showModel (
    void )
```

Testa o método showModel() da classe [Model](#).

Verifica se a exibição das informações do modelo é executada sem erros. A correção visual da saída deve ser verificada manualmente. Here is the call graph for this function:



## 8.31 unit\_model.h

[Go to the documentation of this file.](#)

```
00001 #ifndef UNIT_MODEL_H
00002 #define UNIT_MODEL_H
00003
00012
00020 void unit_Model_constructor(void);
00021
00027 void unit_Model_destructor(void);
00028
00035 void unit_Model_addSystem(void);
```

```

00036
00044 void unit_Model_addFlow(void);
00045
00053 void unit_Model_run(void);
00054
00061 void unit_Model_showModel(void);
00062
00069 void run_unit_tests_Model(void);
00070
00071 #endif

```

## 8.32 test/unit/unit\_system.cpp File Reference

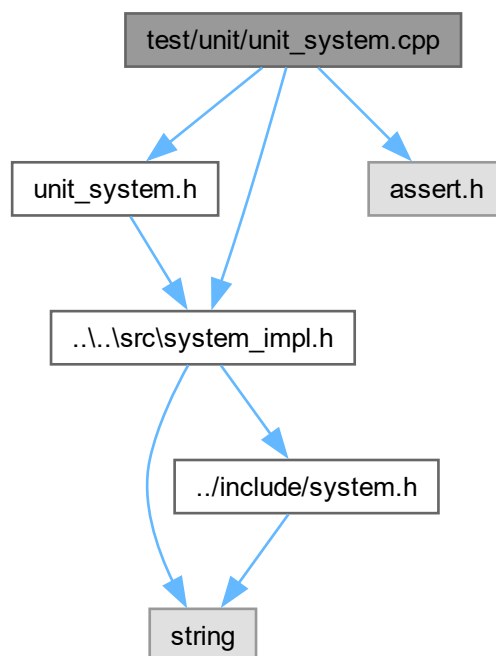
Implementação dos testes unitários da classe [System](#).

```
#include "unit_system.h"
```

```
#include <assert.h>
```

```
#include "../src/system_impl.h"
```

Include dependency graph for unit\_system.cpp:



### Functions

- void [unit\\_System\\_constructor](#) (void)  
Testa o construtor parametrizado da classe [System](#).
- void [unit\\_System\\_destructor](#) (void)  
Testa o destrutor da classe [System](#).
- void [unit\\_System\\_getValue](#) (void)  
Testa o método `getValue()` da classe [System](#).
- void [unit\\_System\\_setValue](#) (void)  
Testa o método `setValue()` da classe [System](#).
- void [run\\_unit\\_tests\\_System](#) (void)  
Executa todos os testes unitários da classe [System](#) em sequência.

### 8.32.1 Detailed Description

Implementação dos testes unitários da classe [System](#).

Contém as implementações de cada função de teste unitário declarada em [unit\\_System.h](#), verificando o comportamento da classe [System\\_Impl](#) para construção, destruição e manipulação de valores.

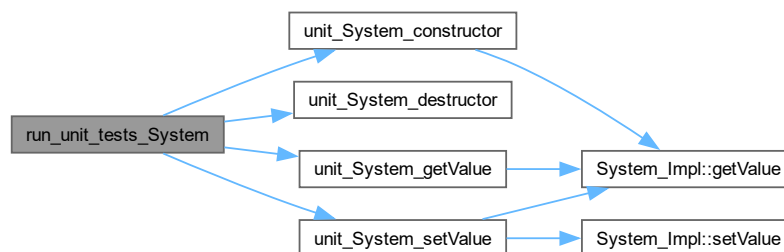
### 8.32.2 Function Documentation

#### 8.32.2.1 run\_unit\_tests\_System()

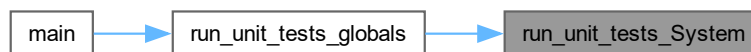
```
void run_unit_tests_System (
    void )
```

Executa todos os testes unitários da classe [System](#) em sequência.

Chama cada função de teste unitário da classe [System](#) e exibe uma mensagem de confirmação caso todos sejam aprovados. Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.32.2.2 unit\_System\_constructor()

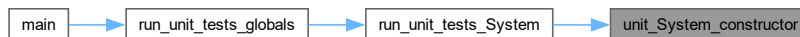
```
void unit_System_constructor (
    void )
```

Testa o construtor parametrizado da classe [System](#).

Verifica se, ao criar sistemas com valores iniciais distintos, o valor armazenado corresponde ao informado na construção. Here is the call graph for this function:



Here is the caller graph for this function:

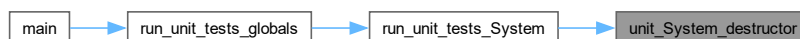


#### 8.32.2.3 unit\_System\_destructor()

```
void unit_System_destructor (  
    void )
```

Testa o destrutor da classe [System](#).

Verifica se o objeto é destruído sem erros ou vazamentos de memória. Here is the caller graph for this function:



#### 8.32.2.4 unit\_System\_getValue()

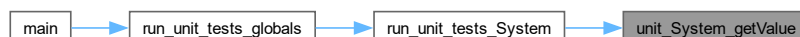
```
void unit_System_getValue (  
    void )
```

Testa o método `getValue()` da classe [System](#).

Verifica se o valor retornado corresponde ao valor definido na construção do objeto. Here is the call graph for this function:



Here is the caller graph for this function:

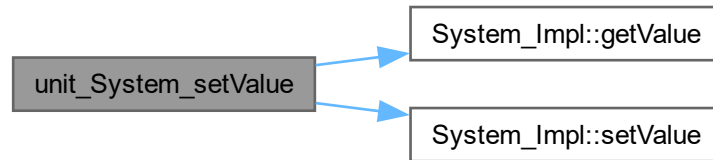


#### 8.32.2.5 unit\_System\_setValue()

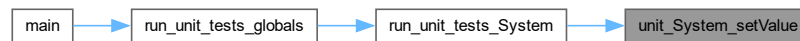
```
void unit_System_setValue (  
    void )
```

Testa o método `setValue()` da classe [System](#).

Verifica se o valor do sistema é atualizado corretamente após a chamada de `setValue()`, sendo refletido por uma chamada subsequente a `getValue()`. Here is the call graph for this function:



Here is the caller graph for this function:

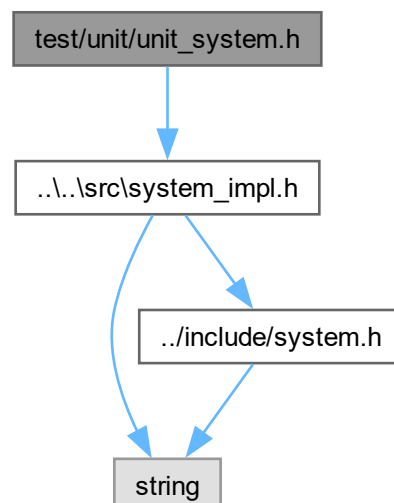


### 8.33 test/unit/unit\_system.h File Reference

Declaração dos testes unitários da classe [System](#).

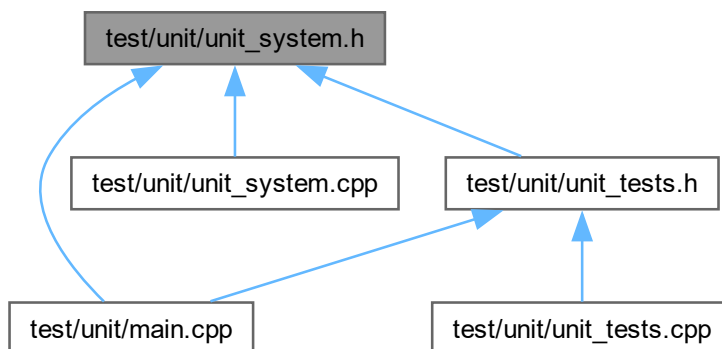
`#include "...\src\system_impl.h"`

Include dependency graph for `unit_system.h`:





This graph shows which files directly or indirectly include this file:



## Functions

- void [unit\\_System\\_constructor](#) (void)  
Testa o construtor parametrizado da classe [System](#).
- void [unit\\_System\\_destructor](#) (void)  
Testa o destrutor da classe [System](#).
- void [unit\\_System\\_getValue](#) (void)  
Testa o método getValue() da classe [System](#).
- void [unit\\_System\\_setValue](#) (void)  
Testa o método setValue() da classe [System](#).
- void [run\\_unit\\_tests\\_System](#) (void)  
Executa todos os testes unitários da classe [System](#) em sequência.

### 8.33.1 Detailed Description

Declaração dos testes unitários da classe [System](#).

Este arquivo contém os protótipos das funções de teste unitário responsáveis por verificar individualmente cada método da classe [System](#) e sua implementação concreta [System\\_Impl](#).

### 8.33.2 Function Documentation

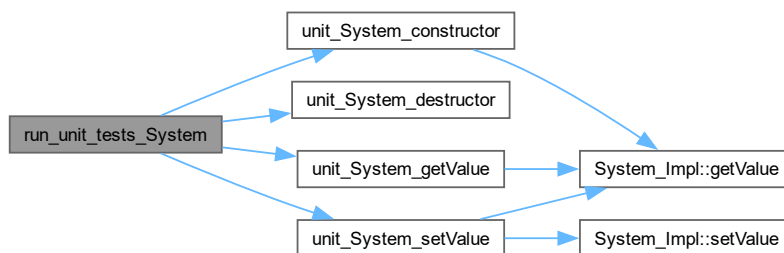
#### 8.33.2.1 run\_unit\_tests\_System()

```
void run_unit_tests_System (
    void )
```

Executa todos os testes unitários da classe [System](#) em sequência.

Chama cada função de teste unitário da classe [System](#) e exibe uma mensagem de confirmação caso todos

sejam aprovados. Here is the call graph for this function:



Here is the caller graph for this function:



### 8.33.2.2 unit\_System\_constructor()

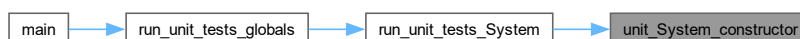
```
void unit_System_constructor (
    void )
```

Testa o construtor parametrizado da classe [System](#).

Verifica se, ao criar sistemas com valores iniciais distintos, o valor armazenado corresponde ao informado na construção. Here is the call graph for this function:



Here is the caller graph for this function:

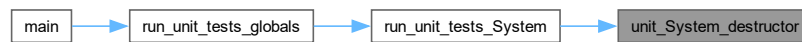


### 8.33.2.3 unit\_System\_destructor()

```
void unit_System_destructor (
    void )
```

Testa o destrutor da classe [System](#).

Verifica se o objeto é destruído sem erros ou vazamentos de memória. Here is the caller graph for this function:



#### 8.33.2.4 unit\_System\_getValue()

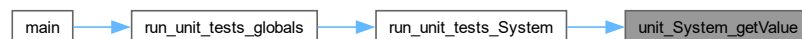
```
void unit_System_getValue (  
    void )
```

Testa o método getValue() da classe [System](#).

Verifica se o valor retornado corresponde ao valor definido na construção do objeto. Here is the call graph for this function:



Here is the caller graph for this function:



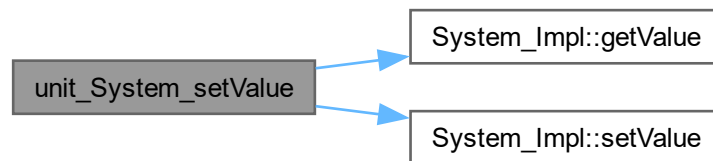
#### 8.33.2.5 unit\_System\_setValue()

```
void unit_System_setValue (  
    void )
```

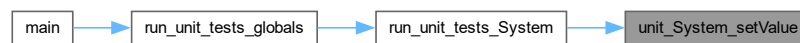
Testa o método setValue() da classe [System](#).

Verifica se o valor do sistema é atualizado corretamente após a chamada de setValue(), sendo refletido

por uma chamada subsequente a `getValue()`. Here is the call graph for this function:



Here is the caller graph for this function:



## 8.34 unit\_system.h

[Go to the documentation of this file.](#)

```

00001 #ifndef UNIT_SYSTEM
00002 #define UNIT_SYSTEM
00003
00012
00013 #include "../src/system_impl.h"
00014
00021 void unit_System_constructor(void);
00022
00028 void unit_System_destructor(void);
00029
00036 void unit_System_getValue(void);
00037
00045 void unit_System_setValue(void);
00046
00053 void run_unit_tests_System(void);
00054
00055 #endif
  
```

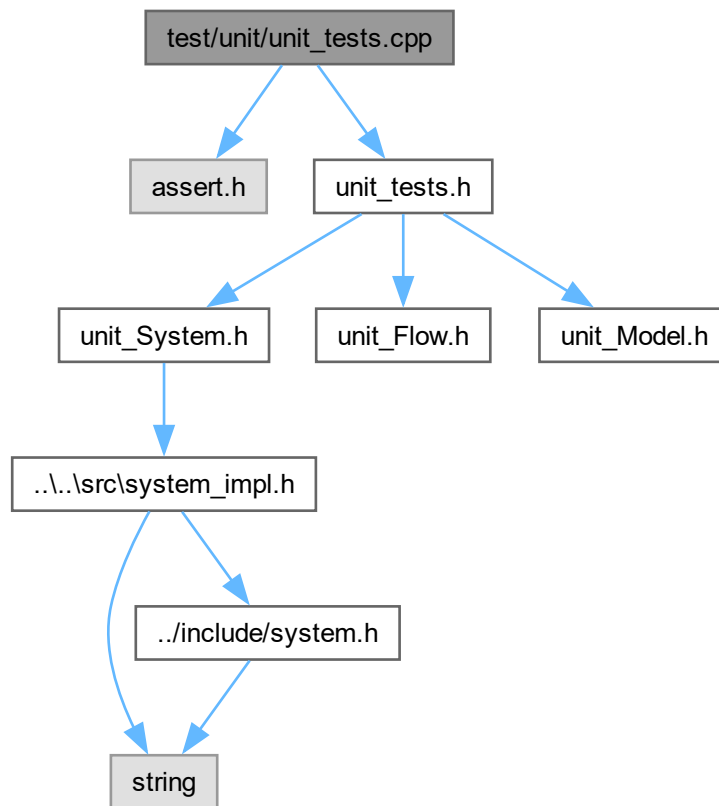
## 8.35 test/unit/unit\_tests.cpp File Reference

Implementação da função orquestradora dos testes unitários globais.

```

#include <assert.h>
#include "unit_tests.h"
  
```

Include dependency graph for unit\_tests.cpp:



## Functions

- void [run\\_unit\\_tests\\_globals](#) (void)  
Executa todos os testes unitários do projeto em sequência.

### 8.35.1 Detailed Description

Implementação da função orquestradora dos testes unitários globais.

Contém a implementação de [run\\_unit\\_tests\\_globals\(\)](#), que centraliza a execução de todos os testes unitários do projeto, chamando as suítes de teste de cada classe individualmente.

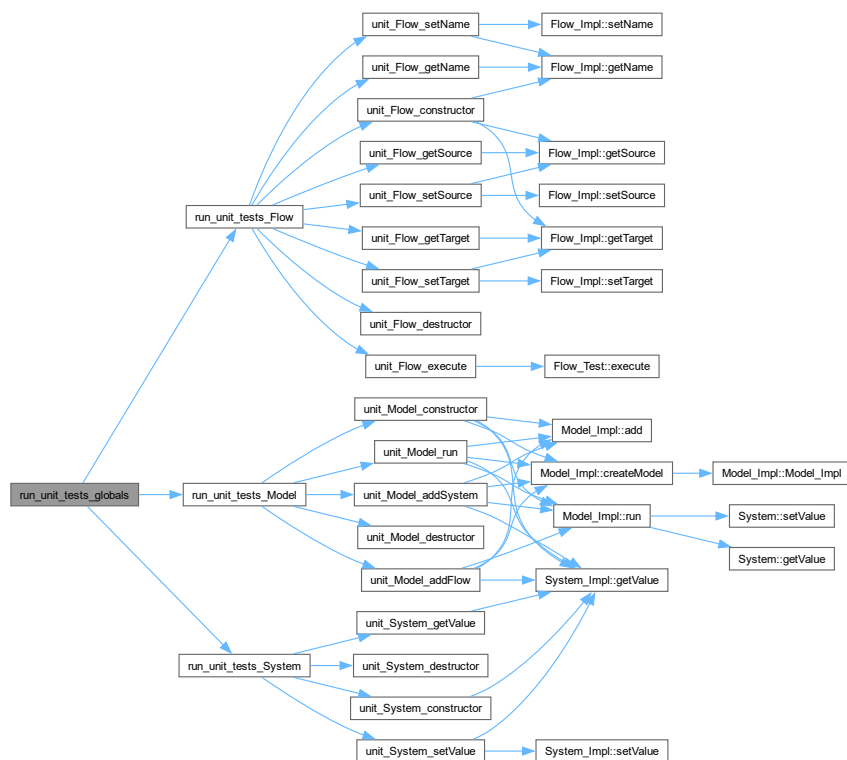
### 8.35.2 Function Documentation

#### 8.35.2.1 run\_unit\_tests\_globals()

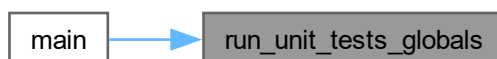
```
void run_unit_tests_globals (
    void )
```

Executa todos os testes unitários do projeto em sequência.

Chama [run\\_unit\\_tests\\_System\(\)](#), [run\\_unit\\_tests\\_Flow\(\)](#) e [run\\_unit\\_tests\\_Model\(\)](#), cobrindo os testes unitários de todas as classes do modelo de simulação. Here is the call graph for this function:



Here is the caller graph for this function:



### 8.36 test/unit/unit\_tests.h File Reference

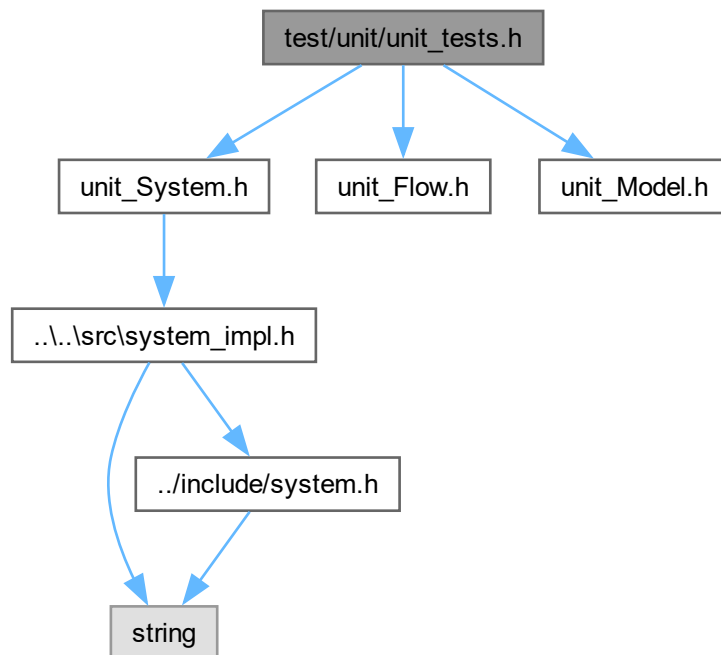
Declaração da função de entrada dos testes unitários globais.

```

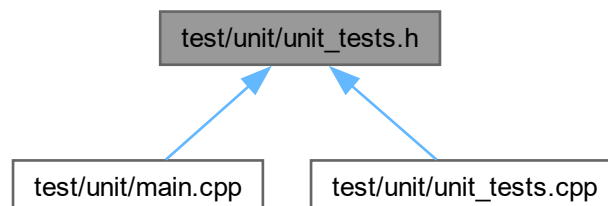
#include "unit_System.h"
#include "unit_Flow.h"
#include "unit_Model.h"

```

Include dependency graph for unit\_tests.h:



This graph shows which files directly or indirectly include this file:



## Functions

- void [run\\_unit\\_tests\\_globals](#) (void)  
Executa todos os testes unitários do projeto em sequência.

### 8.36.1 Detailed Description

Declaração da função de entrada dos testes unitários globais.

Este arquivo agrega os cabeçalhos de todos os módulos de teste unitário ([System](#), [Flow](#) e [Model](#)) e declara a função orquestradora que os executa em sequência.

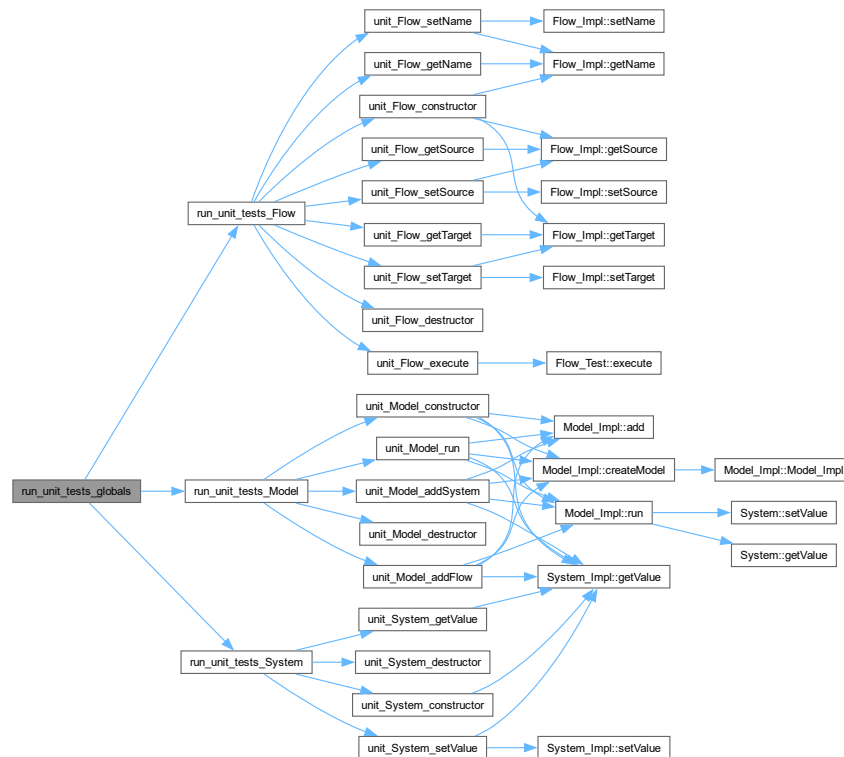
## 8.36.2 Function Documentation

### 8.36.2.1 run\_unit\_tests\_globals()

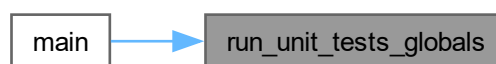
```
void run_unit_tests_globals (
    void )
```

Executa todos os testes unitários do projeto em sequência.

Chama [run\\_unit\\_tests\\_System\(\)](#), [run\\_unit\\_tests\\_Flow\(\)](#) e [run\\_unit\\_tests\\_Model\(\)](#), cobrindo os testes unitários de todas as classes do modelo de simulação. Here is the call graph for this function:



Here is the caller graph for this function:



## 8.37 unit\_tests.h

[Go to the documentation of this file.](#)

```
00001 #ifndef UNIT_TESTS
00002 #define UNIT_TESTS
00003
00012
00013 #include "unit_System.h"
00014 #include "unit_Flow.h"
```



```
00015 #include "unit_Model.h"
00016
00024 void run_unit_tests_globals(void);
00025
00026 #endif
```



# Index

- ~Flow
  - Flow, [16](#)
- ~FlowComplexo
  - FlowComplexo, [36](#)
- ~FlowExponencial
  - FlowExponencial, [42](#)
- ~FlowLogistico
  - FlowLogistico, [48](#)
- ~Flow\_Impl
  - Flow\_Impl, [23](#)
- ~Model
  - Model, [56](#)
- ~Model\_Impl
  - Model\_Impl, [67](#)
- ~System
  - System, [77](#)
- ~System\_Impl
  - System\_Impl, [83](#)
- add
  - Model, [56](#)
  - Model\_Impl, [67](#), [68](#)
- clearSource
  - Model, [57](#)
  - Model\_Impl, [68](#)
- clearTarget
  - Model, [57](#)
  - Model\_Impl, [69](#)
- complexFuncionalTest
  - functional\_test.cpp, [108](#)
  - functional\_test.h, [112](#)
- createFlow
  - Model, [57](#)
- createModel
  - Model, [58](#)
  - Model\_Impl, [69](#)
- createSystem
  - Model, [59](#)
  - Model\_Impl, [70](#)
- deleteFlow
  - Model, [60](#)
  - Model\_Impl, [71](#)
- deleteSystem
  - Model, [60](#)
  - Model\_Impl, [71](#)
- execute
  - Flow, [17](#)
  - Flow\_Impl, [23](#)
  - Flow\_Test, [31](#)
  - FlowComplexo, [36](#)
  - FlowExponencial, [42](#)
  - FlowLogistico, [48](#)
  - FlowTest, [53](#)
- exponentialFuncionalTest
  - functional\_test.cpp, [108](#)
  - functional\_test.h, [112](#)
- floatingPointComparison
  - functional\_test.cpp, [109](#)
- Flow, [15](#)
  - ~Flow, [16](#)
  - execute, [17](#)
  - getName, [17](#)
  - getSource, [17](#)
  - getTarget, [17](#)
  - setName, [17](#)
  - setSource, [17](#)
  - setTarget, [18](#)
- Flow\_Impl, [18](#)
  - ~Flow\_Impl, [23](#)
  - execute, [23](#)
  - Flow\_Impl, [21](#), [22](#)
  - getName, [23](#)
  - getSource, [24](#)
  - getTarget, [24](#)
  - name, [27](#)
  - operator=, [24](#)
  - setName, [25](#)
  - setSource, [26](#)
  - setTarget, [26](#)
  - source, [27](#)
  - target, [27](#)
- Flow\_Test, [27](#)
  - execute, [31](#)
  - Flow\_Test, [30](#), [31](#)
- FlowComplexo, [32](#)
  - ~FlowComplexo, [36](#)
  - execute, [36](#)
  - FlowComplexo, [35](#), [36](#)
  - operator=, [37](#)
- FlowExponencial, [38](#)
  - ~FlowExponencial, [42](#)
  - execute, [42](#)
  - FlowExponencial, [41](#), [42](#)
  - operator=, [43](#)
- FlowLogistico, [44](#)
  - ~FlowLogistico, [48](#)

- execute, 48
- FlowLogistico, 47, 48
- operator=, 49
- flows
  - Model\_Impl, 75
- FlowTest, 50
  - execute, 53
  - FlowTest, 52, 53
- functional\_test.cpp
  - complexFuncionalTest, 108
  - exponentialFuncionalTest, 108
  - floatingPointComparison, 109
  - logisticalFuncionalTest, 110
- functional\_test.h
  - complexFuncionalTest, 112
  - exponentialFuncionalTest, 112
  - logisticalFuncionalTest, 113
- getName
  - Flow, 17
  - Flow\_Impl, 23
  - System, 77
  - System\_Impl, 83
- getSource
  - Flow, 17
  - Flow\_Impl, 24
- getTarget
  - Flow, 17
  - Flow\_Impl, 24
- getValue
  - System, 77
  - System\_Impl, 83
- id\_
  - Model\_Impl, 75
  - System\_Impl, 85
- include Directory Reference, 12
- include/flow.h, 87, 88
- include/model.h, 88, 89
- include/system.h, 90
- logisticalFuncionalTest
  - functional\_test.cpp, 110
  - functional\_test.h, 113
- MAIN
  - main.cpp, 94
- main
  - main.cpp, 94, 96, 97
- main.cpp
  - MAIN, 94
  - main, 94, 96, 97
  - MAIN\_FUNCIONAL\_TESTS, 96
- MAIN\_FUNCIONAL\_TESTS
  - main.cpp, 96
- Model, 53
  - ~Model, 56
  - add, 56
  - clearSource, 57
  - clearTarget, 57
  - createFlow, 57
  - createModel, 58
  - createSystem, 59
  - deleteFlow, 60
  - deleteSystem, 60
  - run, 60
  - setSource, 61
  - setTarget, 61
  - showModel, 62
- Model\_Impl, 62
  - ~Model\_Impl, 67
  - add, 67, 68
  - clearSource, 68
  - clearTarget, 69
  - createModel, 69
  - createSystem, 70
  - deleteFlow, 71
  - deleteSystem, 71
  - flows, 75
  - id\_, 75
  - Model\_Impl, 66
  - models, 75
  - operator=, 72
  - run, 72
  - setSource, 74
  - setTarget, 74
  - showModel, 75
  - systems, 75
- models
  - Model\_Impl, 75
- MyVensim, 1
- name
  - Flow\_Impl, 27
  - System\_Impl, 85
- operator=
  - Flow\_Impl, 24
  - FlowComplexo, 37
  - FlowExponencial, 43
  - FlowLogistico, 49
  - Model\_Impl, 72
  - System\_Impl, 84
- README.md, 91
- run
  - Model, 60
  - Model\_Impl, 72
- run\_unit\_tests\_Flow
  - unit\_flow.cpp, 115
  - unit\_flow.h, 122
- run\_unit\_tests\_globals
  - unit\_tests.cpp, 147
  - unit\_tests.h, 150
- run\_unit\_tests\_Model
  - unit\_model.cpp, 130
  - unit\_model.h, 134
- run\_unit\_tests\_System

- unit\_system.cpp, [140](#)
- unit\_system.h, [143](#)
- setName
  - Flow, [17](#)
  - Flow\_Impl, [25](#)
  - System, [78](#)
  - System\_Impl, [85](#)
- setSource
  - Flow, [17](#)
  - Flow\_Impl, [26](#)
  - Model, [61](#)
  - Model\_Impl, [74](#)
- setTarget
  - Flow, [18](#)
  - Flow\_Impl, [26](#)
  - Model, [61](#)
  - Model\_Impl, [74](#)
- setValue
  - System, [78](#)
  - System\_Impl, [85](#)
- showModel
  - Model, [62](#)
  - Model\_Impl, [75](#)
- source
  - Flow\_Impl, [27](#)
- src Directory Reference, [12](#)
- src/flow\_impl.cpp, [91](#)
- src/flow\_impl.h, [92](#)
- src/main.cpp, [93](#)
- src/model\_impl.cpp, [98](#)
- src/model\_impl.h, [99](#), [100](#)
- src/system\_impl.cpp, [101](#)
- src/system\_impl.h, [102](#), [103](#)
- System, [76](#)
  - ~System, [77](#)
  - getName, [77](#)
  - getValue, [77](#)
  - setName, [78](#)
  - setValue, [78](#)
- System\_Impl, [79](#)
  - ~System\_Impl, [83](#)
  - getName, [83](#)
  - getValue, [83](#)
  - id\_, [85](#)
  - name, [85](#)
  - operator=, [84](#)
  - setName, [85](#)
  - setValue, [85](#)
  - System\_Impl, [82](#), [83](#)
  - value, [85](#)
- systems
  - Model\_Impl, [75](#)
- target
  - Flow\_Impl, [27](#)
- test Directory Reference, [13](#)
- test/functional Directory Reference, [11](#)
- test/functional/flows.cpp, [103](#)
- test/functional/flows.h, [104](#), [106](#)
- test/functional/functional\_test.cpp, [107](#)
- test/functional/functional\_test.h, [111](#), [114](#)
- test/functional/main.cpp, [95](#)
- test/unit Directory Reference, [13](#)
- test/unit/main.cpp, [96](#)
- test/unit/unit\_flow.cpp, [114](#)
- test/unit/unit\_flow.h, [121](#), [128](#)
- test/unit/unit\_model.cpp, [128](#)
- test/unit/unit\_model.h, [133](#), [138](#)
- test/unit/unit\_system.cpp, [139](#)
- test/unit/unit\_system.h, [142](#), [146](#)
- test/unit/unit\_tests.cpp, [146](#)
- test/unit/unit\_tests.h, [148](#), [150](#)
- unit\_flow.cpp
  - run\_unit\_tests\_Flow, [115](#)
  - unit\_Flow\_constructor, [116](#)
  - unit\_Flow\_destructor, [117](#)
  - unit\_Flow\_execute, [117](#)
  - unit\_Flow\_getName, [118](#)
  - unit\_Flow\_getSource, [118](#)
  - unit\_Flow\_getTarget, [119](#)
  - unit\_Flow\_setName, [119](#)
  - unit\_Flow\_setSource, [120](#)
  - unit\_Flow\_setTarget, [120](#)
- unit\_flow.h
  - run\_unit\_tests\_Flow, [122](#)
  - unit\_Flow\_constructor, [123](#)
  - unit\_Flow\_destructor, [124](#)
  - unit\_Flow\_execute, [124](#)
  - unit\_Flow\_getName, [125](#)
  - unit\_Flow\_getSource, [125](#)
  - unit\_Flow\_getTarget, [126](#)
  - unit\_Flow\_setName, [126](#)
  - unit\_Flow\_setSource, [127](#)
  - unit\_Flow\_setTarget, [127](#)
- unit\_Flow\_constructor
  - unit\_flow.cpp, [116](#)
  - unit\_flow.h, [123](#)
- unit\_Flow\_destructor
  - unit\_flow.cpp, [117](#)
  - unit\_flow.h, [124](#)
- unit\_Flow\_execute
  - unit\_flow.cpp, [117](#)
  - unit\_flow.h, [124](#)
- unit\_Flow\_getName
  - unit\_flow.cpp, [118](#)
  - unit\_flow.h, [125](#)
- unit\_Flow\_getSource
  - unit\_flow.cpp, [118](#)
  - unit\_flow.h, [125](#)
- unit\_Flow\_getTarget
  - unit\_flow.cpp, [119](#)
  - unit\_flow.h, [126](#)
- unit\_Flow\_setName
  - unit\_flow.cpp, [119](#)
  - unit\_flow.h, [126](#)
- unit\_Flow\_setSource

- unit\_flow.cpp, [120](#)
  - unit\_flow.h, [127](#)
- unit\_Flow\_setTarget
  - unit\_flow.cpp, [120](#)
  - unit\_flow.h, [127](#)
- unit\_model.cpp
  - run\_unit\_tests\_Model, [130](#)
  - unit\_Model\_addFlow, [130](#)
  - unit\_Model\_addSystem, [131](#)
  - unit\_Model\_constructor, [131](#)
  - unit\_Model\_destructor, [132](#)
  - unit\_Model\_run, [132](#)
  - unit\_Model\_showModel, [133](#)
- unit\_model.h
  - run\_unit\_tests\_Model, [134](#)
  - unit\_Model\_addFlow, [135](#)
  - unit\_Model\_addSystem, [136](#)
  - unit\_Model\_constructor, [136](#)
  - unit\_Model\_destructor, [137](#)
  - unit\_Model\_run, [137](#)
  - unit\_Model\_showModel, [138](#)
- unit\_Model\_addFlow
  - unit\_model.cpp, [130](#)
  - unit\_model.h, [135](#)
- unit\_Model\_addSystem
  - unit\_model.cpp, [131](#)
  - unit\_model.h, [136](#)
- unit\_Model\_constructor
  - unit\_model.cpp, [131](#)
  - unit\_model.h, [136](#)
- unit\_Model\_destructor
  - unit\_model.cpp, [132](#)
  - unit\_model.h, [137](#)
- unit\_Model\_run
  - unit\_model.cpp, [132](#)
  - unit\_model.h, [137](#)
- unit\_Model\_showModel
  - unit\_model.cpp, [133](#)
  - unit\_model.h, [138](#)
- unit\_system.cpp
  - run\_unit\_tests\_System, [140](#)
  - unit\_System\_constructor, [140](#)
  - unit\_System\_destructor, [141](#)
  - unit\_System\_getValue, [141](#)
  - unit\_System\_setValue, [141](#)
- unit\_system.h
  - run\_unit\_tests\_System, [143](#)
  - unit\_System\_constructor, [144](#)
  - unit\_System\_destructor, [144](#)
  - unit\_System\_getValue, [145](#)
  - unit\_System\_setValue, [145](#)
- unit\_System\_constructor
  - unit\_system.cpp, [140](#)
  - unit\_system.h, [144](#)
- unit\_System\_destructor
  - unit\_system.cpp, [141](#)
  - unit\_system.h, [144](#)
- unit\_System\_getValue
  - unit\_system.cpp, [141](#)
  - unit\_system.h, [145](#)
- unit\_System\_setValue
  - unit\_system.cpp, [141](#)
  - unit\_system.h, [145](#)
- unit\_tests.cpp
  - run\_unit\_tests\_globals, [147](#)
- unit\_tests.h
  - run\_unit\_tests\_globals, [150](#)
- value
  - System\_Impl, [85](#)